Student Guide: GCP Deployment and CI/CD

Version 1.1 — 28 Oct 2026

How to use this guide

Each episode lists what must be done first, the issue it closes, the commands to run, and a **"Done" when** checked.

Replace these placeholders in every command with your team's values. None of them are **secret**.

Placeholder	Meaning	Example
`YOUR_PROJECT_ID`	GCP Project ID, set in Episode 1	`cpe334-sec1-team07`
`YOUR_REGION`	GCP region	`asia-southeast1`
`YOUR_GITHUB`	GitHub user or organization that owns the repo	`team07-cpe334`
`YOUR_REPO`	Repository name	`toktickit-webapp`
`staging`	Your sprint's staging branch	`staging`
`KIT`	Folder where you unzipped the Starter Kit, outside the repo	`~/kit/toktickit-lab5-starter`

Examples use the TokTickIT app. If your app uses different names, endpoints or scripts, adjust them.

Rules for the whole lab

- Never paste passwords, connection strings or keys into an AI chat, the repo, a screenshot or a group chat.
- Read every `gcloud` command an AI suggests before you run it. Never let an AI change billing, IAM or the database on its own.
- Cloud SQL is the main cost. It is billed for every hour it is running.

Content

How to use this guide
Guide/Episode 0: Overview
Guide/Episode 1: GCP account and cost control
Guide/Episode 2: From local to cloud – what changes
Guide/Episode 3: Docker
Guide/Episode 4: CI on pull requests
Guide/Episode 5: First deployment to GCP
Guide/Episode 6: Connect GitHub to GCP and set the merge gate
Guide/Episode 7: Auto deploy and smoke test
Guide/Episode 8: Persistence and rollback
Guide/Episode 9: Pause when you are not using it
Guide/Episode 10: Teardown at the end of term

Guide/Episode 0: Overview

Every change goes through a pull request and CI before it can reach `main`. A merge to `main` then builds an image and automatically deploys it to Cloud Run. If a release is bad, you redeploy an older image using its commit SHA.

Service	What it does
Docker	Packages the web UI, API and migration tools into one image
Artifact Registry	Stores every image version
Cloud Run	Runs the app over HTTPS and scales to 0 instances when idle
Cloud SQL (PostgreSQL)	Keeps data outside the container, so it survives redeploys
Secret Manager	Stores `DATABASE_URL` and the admin password
GitHub Actions + Workload Identity Federation	Tests and deploys automatically, with no service account key

0.1 Team Workflow Suggestion

1. Every task has an "**Issue**".
2. Create a branch from `staging`.
3. Open a pull request into `staging`. A teammate reviews and approves it, then you merge.
4. Open a release pull request from `staging` into `main`.
5. Work done in the Console or in GitHub Settings has no branch, but it still has an Issue. Close that Issue with a screenshot as evidence.

Note: Never commit directly to `staging` or `main`. Authors cannot approve their own pull requests. Deploys happen from `main` only.

0.2 What your app can use on GCP

Your team chose its own stack for the term project. For Lab 5, it must fit these rules. Your team adapts the kit to your stack yourselves.

Required (project spec)

- The app is packaged with **Docker** and deployed on **GCP**.
- Every relevant PR runs **build and automated tests**, and a failed required check **blocks the merge**.
- Releases deploy **from `main`** through a **repeatable pipeline**.
- Data **survives a restart or redeployment**, so it lives outside the container.
- Configuration is documented **without real secrets**.

Recommended (the kit is built for this)

- **Cloud Run** for hosting, **Artifact Registry** for images, **GitHub Actions** with **Workload Identity Federation** for CI/CD, and **Cloud SQL** for the database.
- If you use something else, your team adapts the kit yourselves and explains the choice in your System-Level SDS.

0.3 What the Starter Kit gives you

The Starter Kit has ready-made files for TokTickIT. Copy them into your repo in the episode shown, then adapt them to your app (see 0.4 What to change for your project). Unzip the kit outside your repo.

Episode	Branch (example)	Kit files	What they do
3	`build/3-docker`	`Dockerfile`, `.dockerignore`, `docker-compose.yml`, `server/docker-entrypoint.sh`, `client/nginx.conf`, `server/Dockerfile`, `client/Dockerfile`	Build one container with the UI and API, and run it locally with a database
4	`ci/4-pr-checks`	`.github/workflows/ci-docker.yml`, the whole `deploy/` folder	Run build and tests on every PR. CI also checks the deploy scripts, so `deploy/` comes in now
5	none	`deploy/deploy.sh`	One-time setup and first deploy to GCP, run from Cloud Shell
7	`ci/7-deploy-pipeline`	`.github/workflows/deploy.yml` (uses `deploy/release.sh` and `deploy/smoke_test.py`)	Deploy automatically on merge to `main`, run the smoke test, and roll back with `image_tag`
End of term	none	`deploy/teardown.sh`	Delete the cloud resources

0.4 What to change for your project

The kit files are written for TokTickIT: a Node.js app with PostgreSQL and Prisma, with the UI and API in one container. Before you use a kit file, change the parts below to match your app. Search your repo for `TokticKIT` to find every name you need to change.

What	Change	Episode
Resource names	Service `toktickit`, job `toktickit-migrate`, registry `toktickit`, Cloud SQL `toktickit-pg`, database `toktickit`, secrets `toktickit-*`, service accounts `toktickit-runner,` and `toktickit-deployer`. Rename them the same way everywhere, including the commands in this guide.	3–7
`Dockerfile`, `server/docker-entrypoint.sh`	The base image, build commands, files copied into the final image, and the port are all specified. The final image must contain every file the server and the seed need at run time.	3
`docker-compose.yml`	Services, ports, and environment variables for running locally	3
`ci-docker.yml`	Install, migrate, seed, test, and build commands for your stack. If you rename the job `test-and-docker`, select the new name in the ruleset (Episode 6).	4, 6
`deploy/deploy.sh`, `deploy/release.sh`	The migration and seed commands, and the secrets your app reads. The kit creates Cloud SQL for **PostgreSQL**. If you use MySQL or SQL Server, change the database version and connection settings (Cloud SQL supports both). If your team uses a different database, you will need to replace this part yourself.	5, 7
`deploy/smoke_test.py`	The endpoints it checks and the text it expects. Keep it read-only and never use the admin account.	7

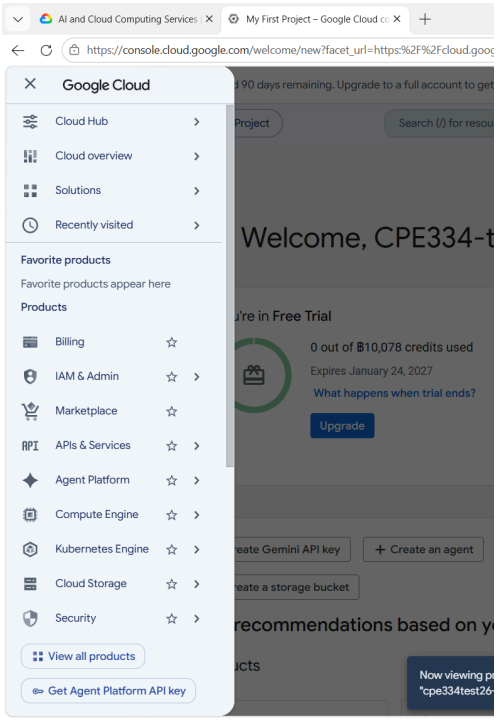
Keep these as they are unless you know why you are changing them:

- Workload Identity Federation instead of JSON keys
- The full commit SHA as the image tag, and deploying by image digest
- The `if` conditions on the `test` and `deploy` jobs in `deploy.yml` (they make rollback work)
- Migrations that only add tables or columns, so an older image still works after a rollback

0.5 Console menus you will use

Go to the Google Cloud Console at console.cloud.google.com. Open the menu with ☰ at the top left. Click the star ☆ next to a menu to pin it. If you cannot find a menu, type its name in the Search bar at the top.

Menu	Used for	Episodes
Billing → Budgets & caps	Track cost and set alerts	1
IAM & Admin → IAM	Who or which service can do what	1, 5
IAM & Admin → Service Accounts	Accounts for programs, not people	5, 6
IAM & Admin → Workload Identity Federation	Lets GitHub Actions sign in to GCP without a key	6
APIs & Services	Each service must be enabled before use	5
Cloud SQL	The PostgreSQL database	5
Security → Secret Manager	Passwords and connection strings	5
Cloud Run	Runs the app; shows each revision	5, 7, 8
Artifact Registry (search)	Docker images	5, 7
Logging (search)	Errors from a failed deploy or migration	5



Guide/Episode 1: GCP account and cost control

Videos: LEB2 / Learn It / Student Guide: GCP Deployment and CI/CD (Video) / Episode 1

Prerequisite: none

Issue #1 (no branch)

Criterion: GCP deployment and persistence

1.1 Get access to GCP

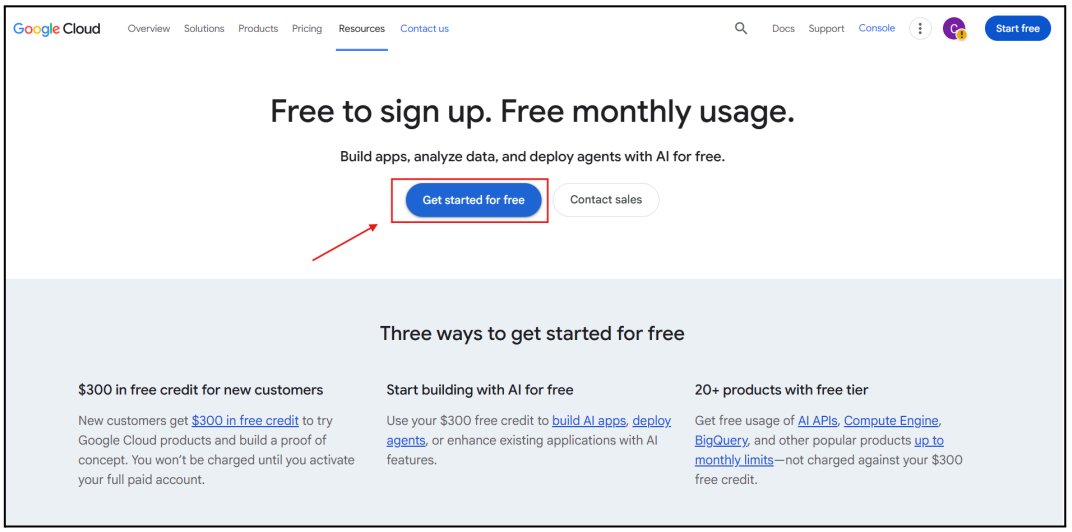
Use the access method your instructor announces. If your team signs up for the Free Trial itself, follow these steps.

Before you start

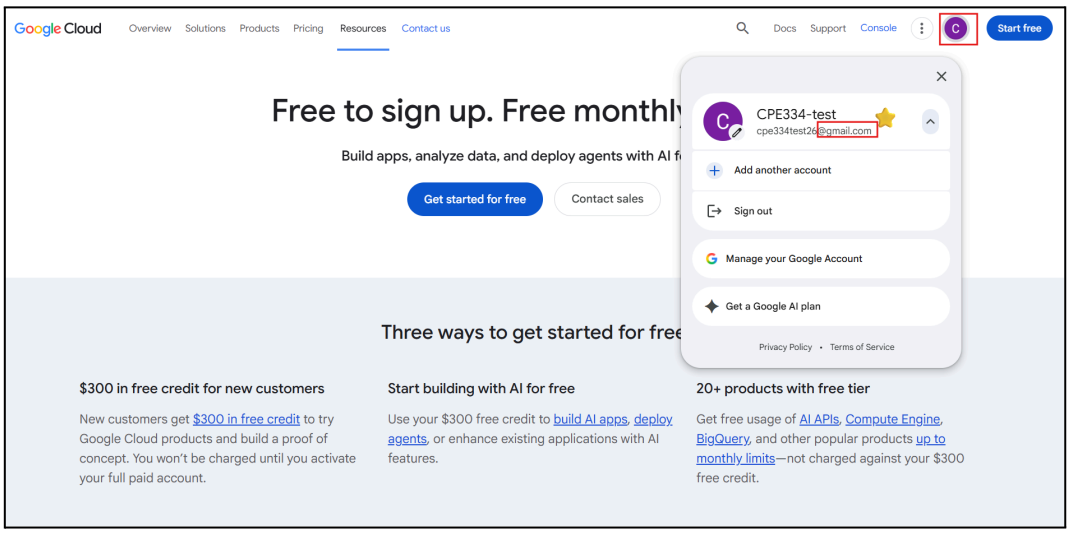
- Use a **personal Gmail account** that has never used the Free Trial.
- **Do not use your university account (@mail.kmutt.ac.th).** Signing up with it can end in a prepayment request instead of the free credits.
- Have a credit or debit card ready. Google places a temporary **authorization of about 900 THB** on the card to verify it. This amount is not a charge; it is released within about a week. A debit card needs enough balance.

Steps

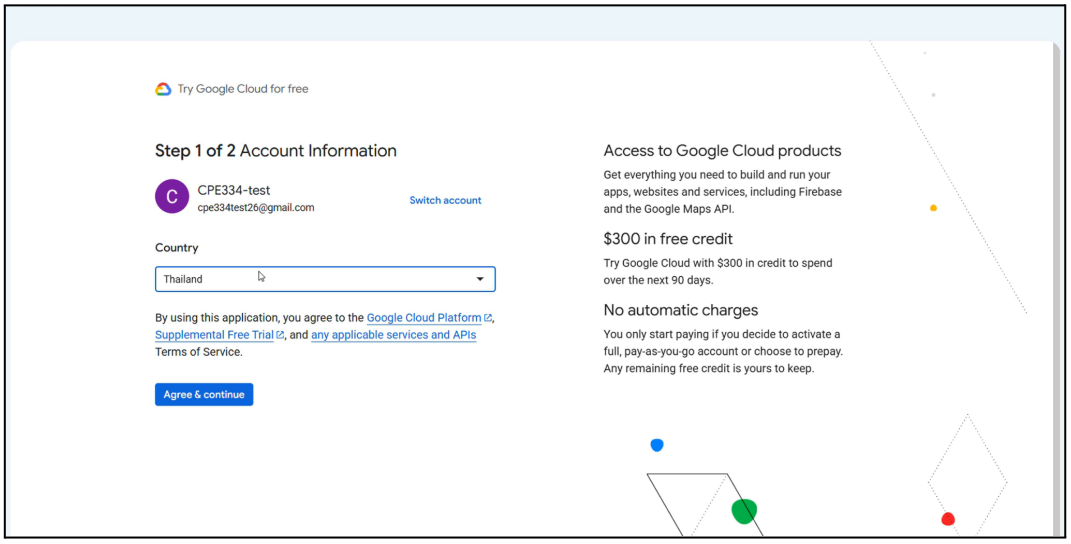
1. Go to cloud.google.com/free and click on **Get started for free**.



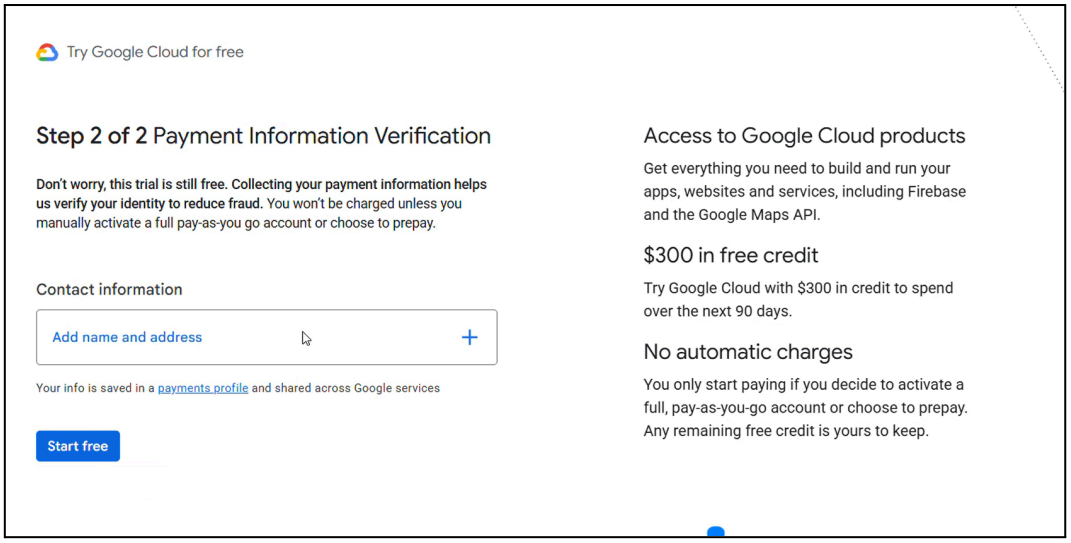
2. Sign in with your personal Gmail account. Check the account shown at the top right before you continue.



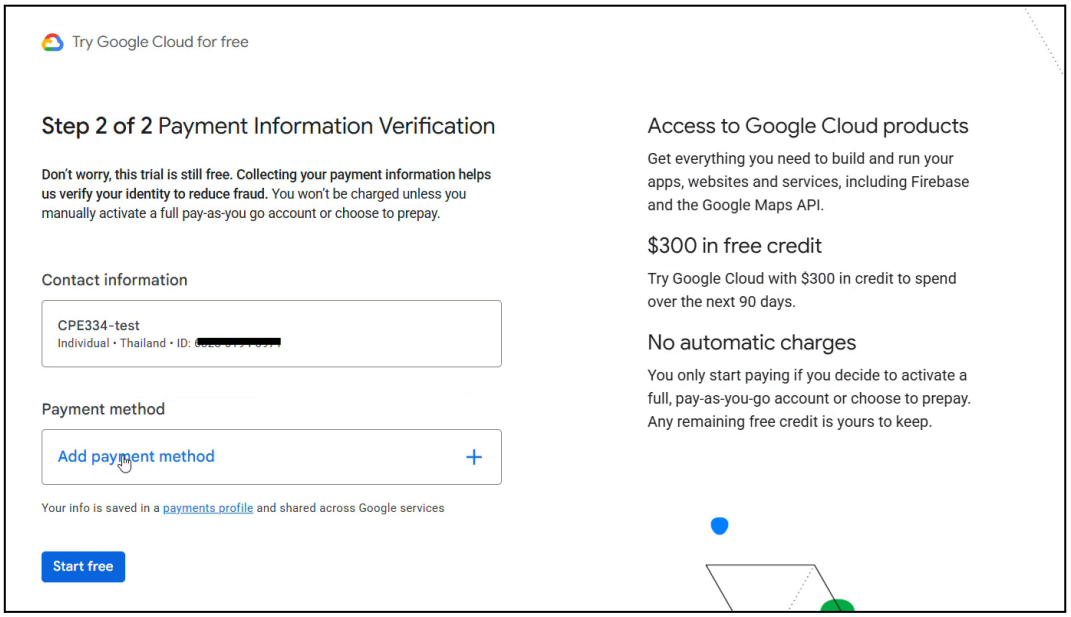
3. Choose your country (**Thailand**), read and accept the Terms of Service, and click **Continue**.



4. On Step 2 of 2 Payment Information Verification, fill in your Contact information: click Add name and address, enter your name and address, and save. If the form asks for an account type, choose **Individual**.

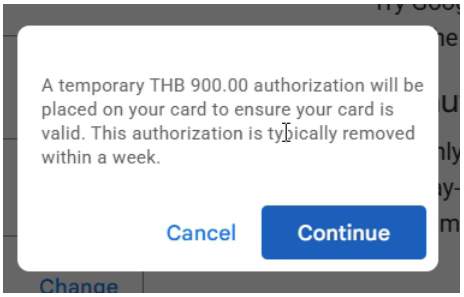


5. Under Payment method, click Add payment method, enter your card details, and save. Confirm with the OTP from your bank if asked.



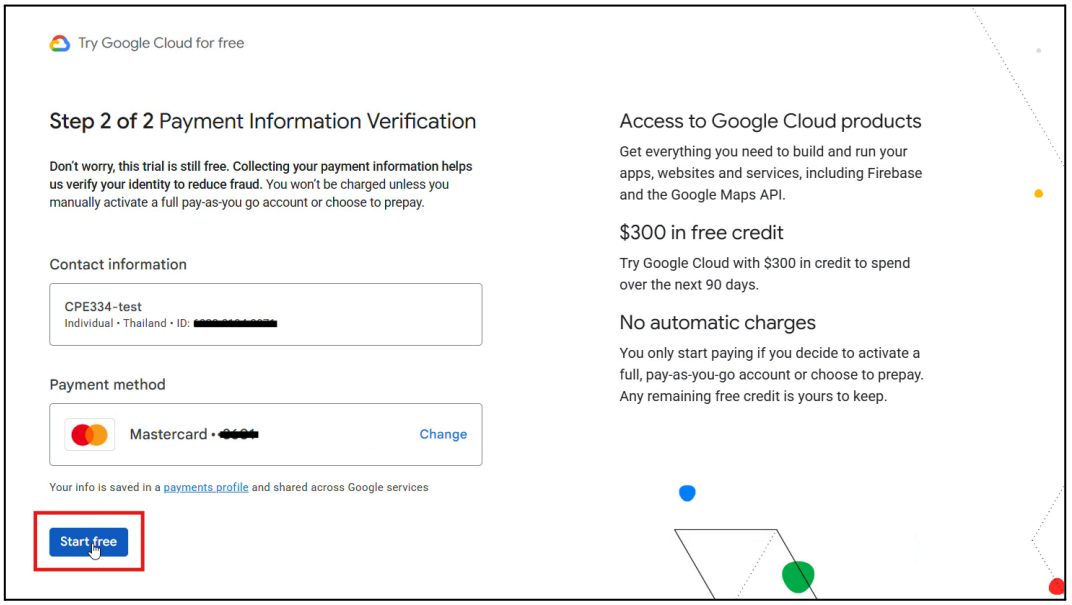
Note: card verification and prepayment

- Your bank may show a **temporary hold of about 900 THB**, or it may show nothing. Both are normal. The hold is a verification, not a charge, and is released within about a week. A debit card needs enough balance to cover it.

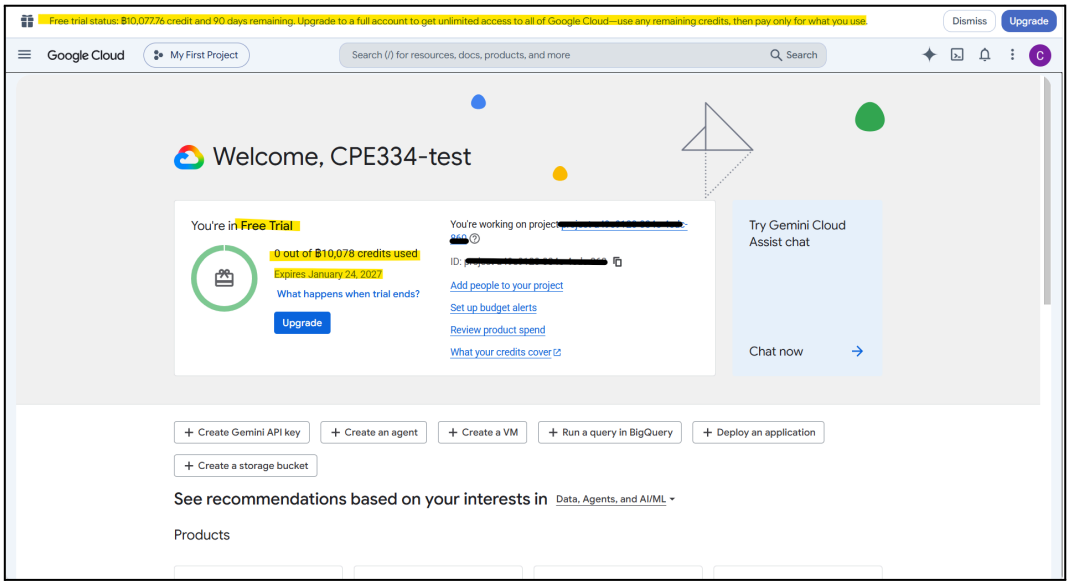


- Use a card that has **never been used for a Google Cloud Free Trial**. A card that has been used before can stop you from getting the free credits.
- If GCP asks for a **prepayment** instead of starting the free trial, **do not pay**. Close the page and sign up again with a different personal Gmail account and a card that has not been used for a Free Trial.

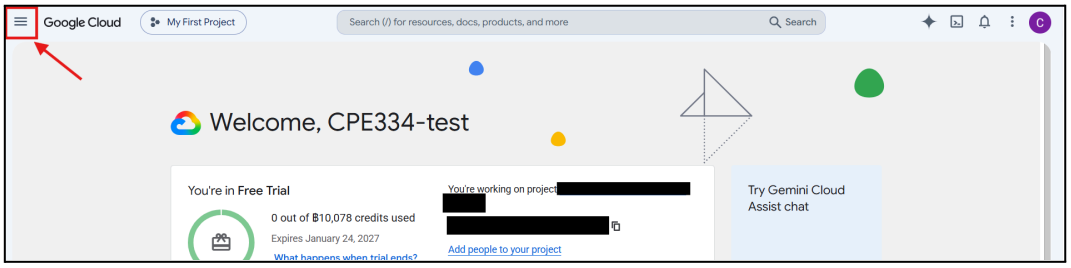
6. Click **Start free**.

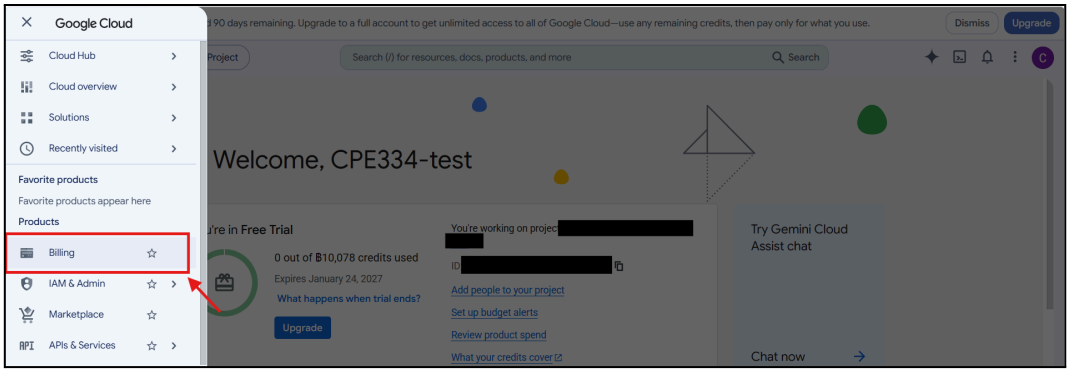


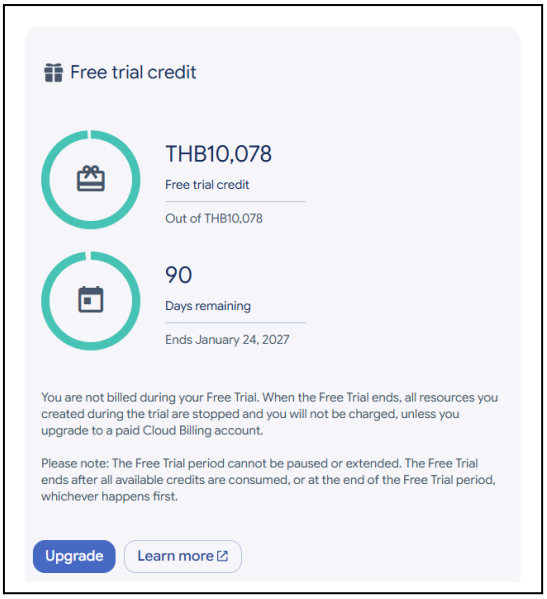
The Google Cloud Console opens.



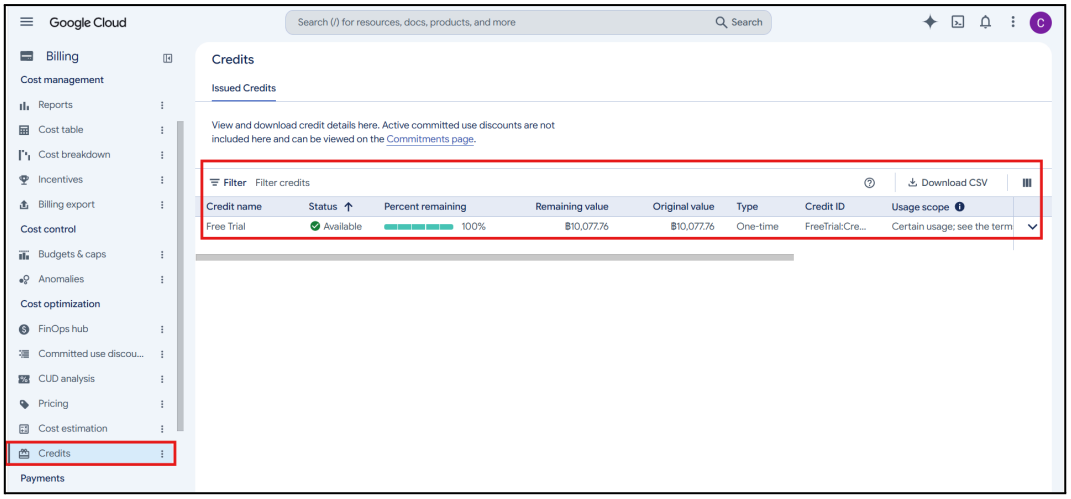
7. Open ☰ → **Billing** → **Overview**. There must be no red banner.







8. Open **Billing** → **Credits**. You should see **USD 300** and the expiry date.



If something goes wrong

- Do not click **Upgrade** unless your team has agreed to pay.
- If GCP asks for a **prepayment** or says the billing account is closed, do not transfer money. Take a screenshot and tell your TA.

Done when: Billing → Credits shows USD 300 and Billing → Overview has no red banner.

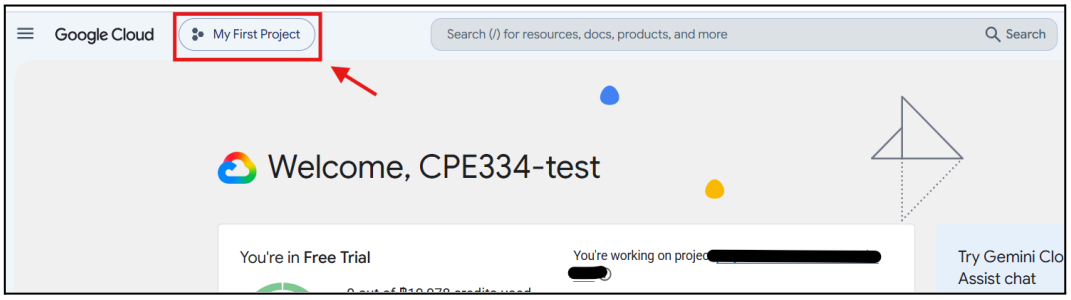
References: Google Cloud Free Trial FAQs · Free Trial and free features

1.2 Create a project

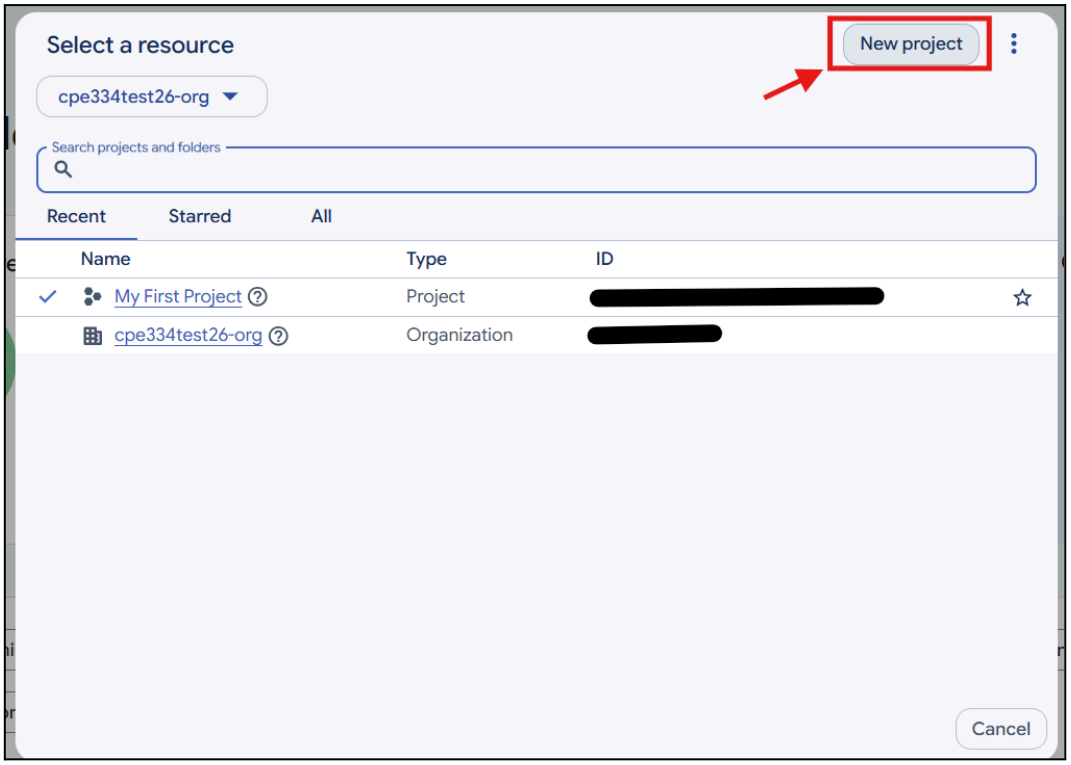
Start at console.cloud.google.com and sign in with the same personal Gmail account you used for the Free Trial.

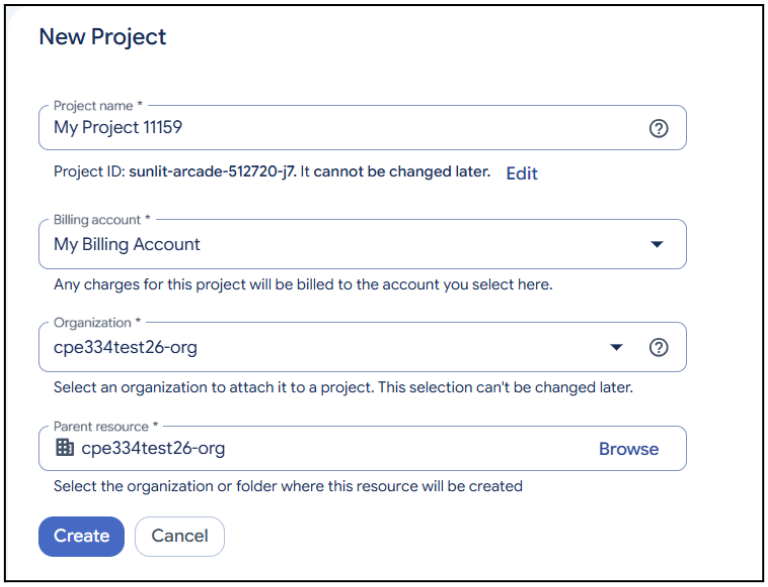
Steps

1. In the top bar, click the **project picker** next to the Google Cloud logo. It shows the current project name, for example “My First Project”.

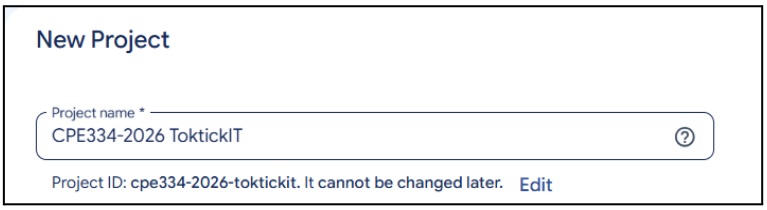


2. In the dialog that opens, click **New Project** at the top right. Shortcut: you can also go straight to console.cloud.google.com/projectcreate.

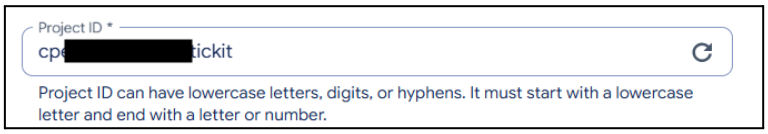




3. In **Project name**, type a name that includes the **course code** and either your **team name** or your **project app name**, so everyone can tell which project it is. For example:
    - `CPE334 Team07`
    - `CPE334 TokTickIT`



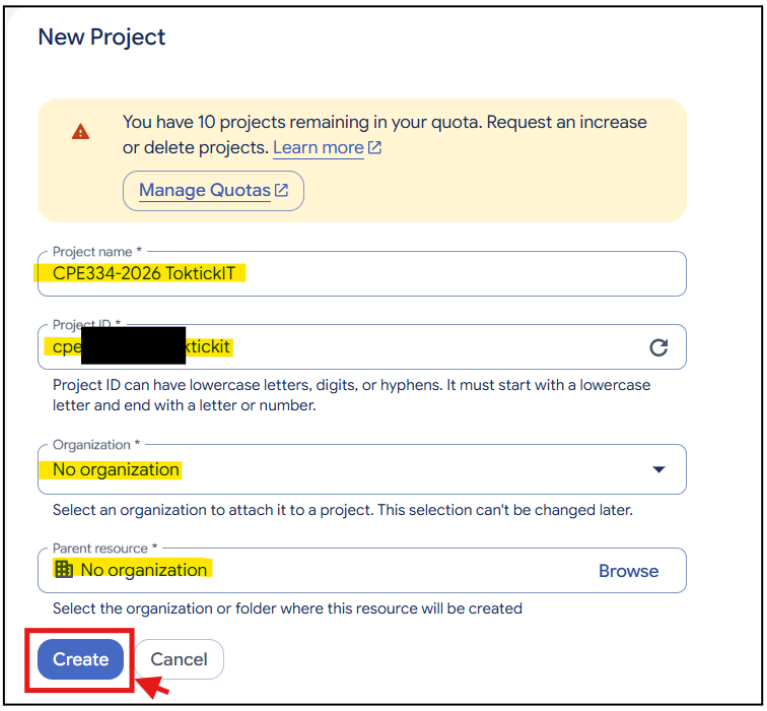
4. Under the name, the form shows the **Project ID** it will use. Click **Edit** and set an ID that matches your project name, for example `cpe334-team07` or `cpe334-toktickit`.
    - 6 to 30 characters: lowercase letters, numbers and hyphens only.
    - It must start with a letter and cannot end with a hyphen.
    - It must be unique across all of Google Cloud. If it is taken, add a number at the end.
    - **The Project ID cannot be changed after you click Create.** You will type it in every command in this lab.

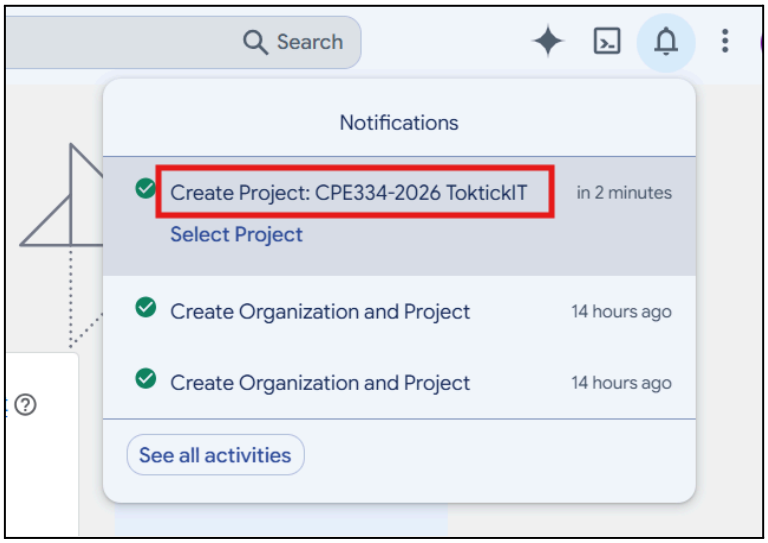


5. Under **Billing account**, choose the account created by your Free Trial. It is usually called **My Billing Account**.
6. Under **Location**, a personal account shows **No organization**. Leave it as it is.

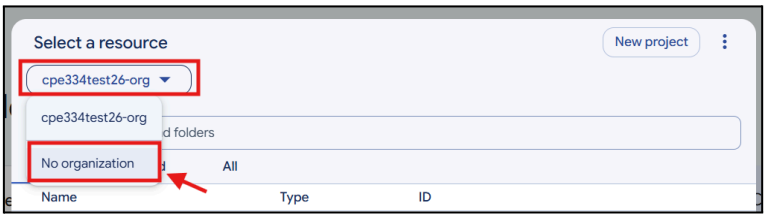


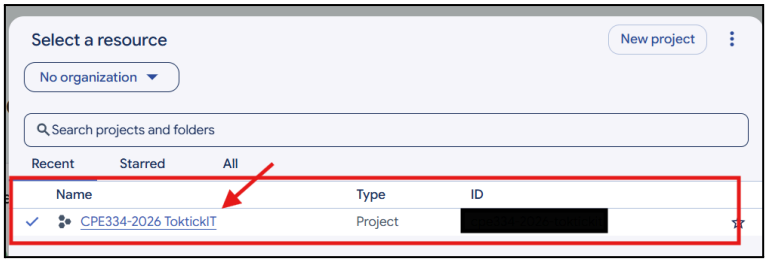
7. Click **Create**. Wait for the notification (bell icon, top right) to say the project was created. This usually takes under a minute.

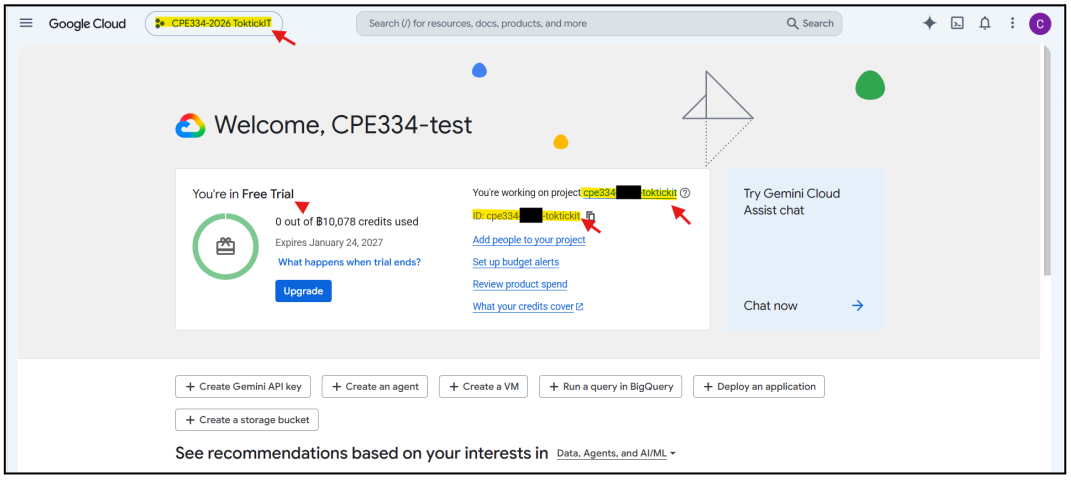




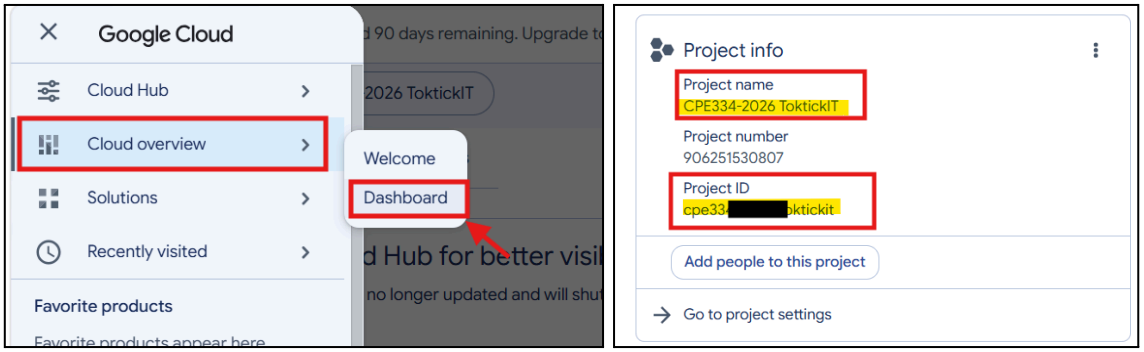
8. Open the project picker again and **select your new project**. Check that the top bar now shows its name.







9. Open ☰ → **Cloud overview** → **Dashboard**. In the **Project info** card, check the Project name, Project ID and Project number. Write down the **Project ID** for your team.



Note: You can rename “My First Project” instead, but its Project ID is random and cannot be changed, so a new project is recommended. Your team uses this one project from Lab 5 through the Final Project. **One team member creates it and adds the others in 1.3.**

Done when: the top bar shows your new project and the Project ID in the Dashboard matches the one your team agreed on.

Reference: Creating and managing projects

1.3 Add your teammates

IAM & Admin → IAM → **Grant access** → enter a teammate's email → role **Editor** → Save.

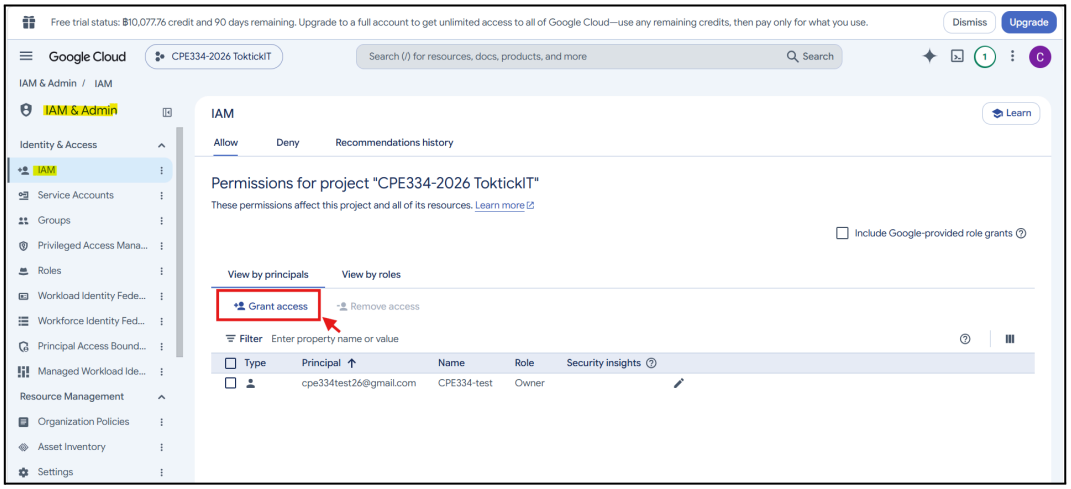
Before you begin

- Only the team member who created the project does this step.
- Collect each teammate's personal Gmail address. Use the accounts they will use for the lab, not @mail.kmutt.ac.th.
- Check the project picker in the top bar. It must show your team's project, not "My First Project".

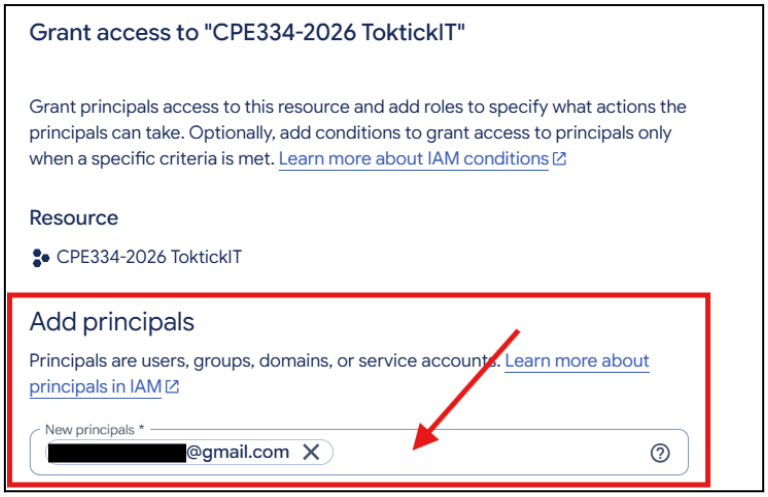
Steps

Start at console.cloud.google.com/iam-admin/iam.

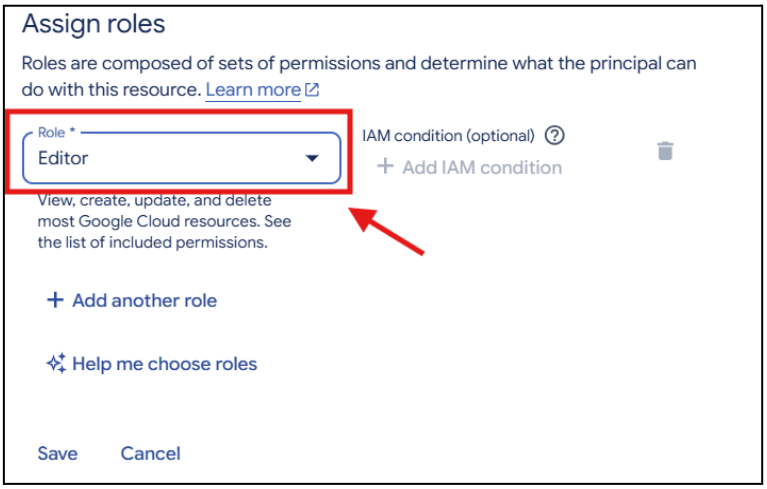
1. Click **Grant access** at the top of the IAM page.



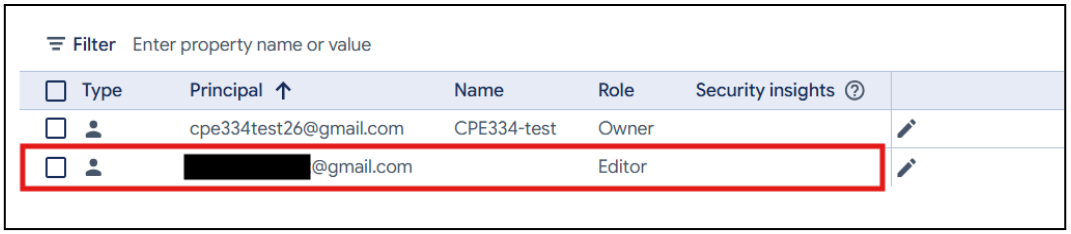
2. Under **Add principals**, in **New principals**, enter your teammates' Gmail addresses, one at a time or separated by commas.



3. Under **Assign roles**, click **Select a role**, type **Editor**, and then choose **Editor** (under Basic).



4. Click **Save**.
5. Back on the IAM page, open **View by principals** and check that each teammate is listed with the **Editor** role.



6. Ask each teammate to open **console.cloud.google.com**, select your project in the project picker, and confirm they can see it.

Note:

- Add only the teammates who work on this project. Do not add friends from other teams or anyone outside your team.
- Give **Editor**, not **Owner**. Editor can create and change resources but cannot change who has access or manage billing. Owner can also add or remove people and delete the project, so keep Owner with the one person who created the project.
- When someone leaves the team, remove them: IAM page → select the person → **Remove access**.

Done when: every teammate appears on the IAM page with **Editor** and can open the project from their own account.

Reference: Quickstart: Grant roles in the Google Cloud console

1.4 Open Cloud Shell and check billing.

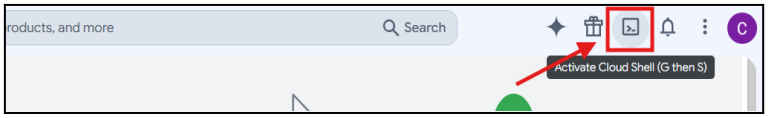
Cloud Shell is a terminal in your browser with gcloud, git, and Docker already installed. You will use it later.

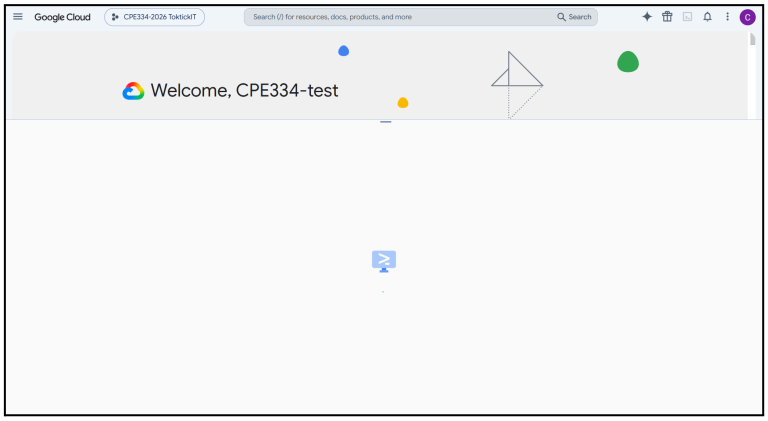
Before you begin

- Finish 1.2. The project picker in the top bar must show your team's project.
- Paste into Cloud Shell with **Ctrl+Shift+V** or right-click → **Paste**.
- Variables such as `PROJECT_ID` are lost each time Cloud Shell closes. Run the `export` line again every time you open a new session.

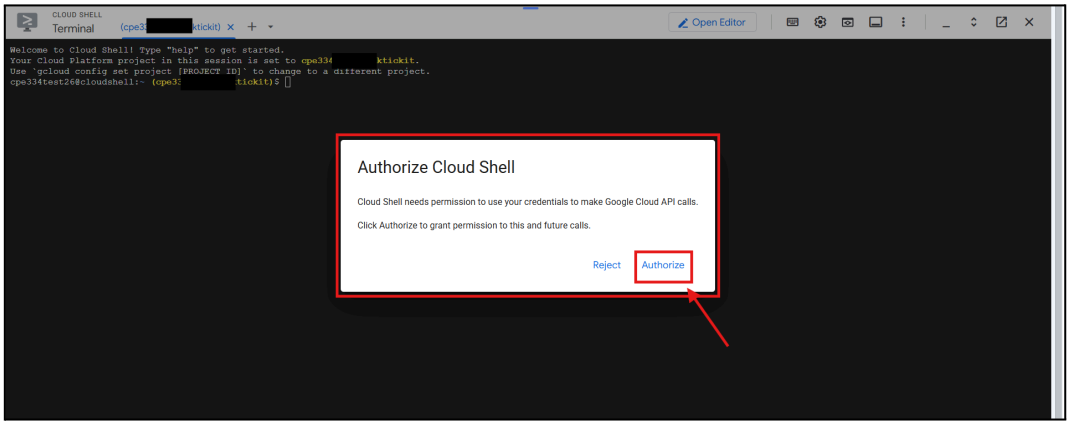
Steps

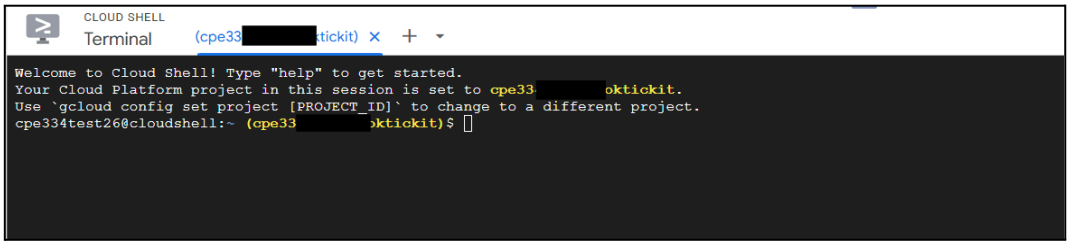
1. Click the **Activate Cloud Shell** button (**>_**) at the top right of the Console. A terminal opens at the bottom of the page. The first start can take about a minute.





2. If a pop-up asks you to **Authorize Cloud Shell**, click **Authorize**. The output must be the same Gmail account as the avatar at the top right of the Console. If not, switch accounts before you continue.

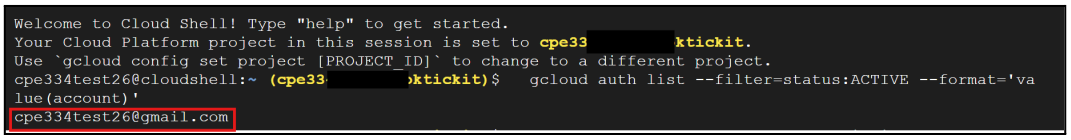




If the welcome message already says your project is set, continue anyway. Steps 3–4 make sure the terminal uses the right project.

3. Check which account the terminal uses:

```
gcloud auth list --filter=status:ACTIVE --format='value(account)'
```

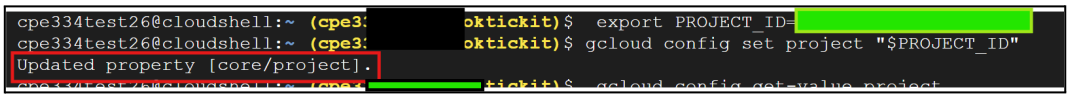


4. Set your project. Replace `YOUR_PROJECT_ID` with your team's Project ID from 1.2:

```
export PROJECT_ID=YOUR_PROJECT_ID
```

```
gcloud config set project "$PROJECT_ID"
```

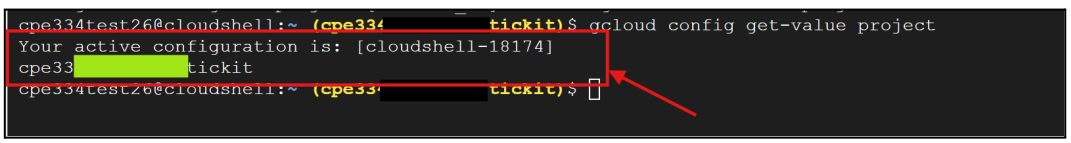
The output says `Updated property [core/project]`. The prompt then shows your Project ID in yellow.



4. Check that the terminal and the Console use the same project:

```
gcloud config get-value project
```

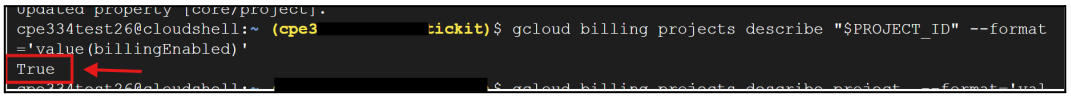
The output first shows your active configuration, then your Project ID. The Project ID must match the project shown in the project picker in the top bar.



5. Check that billing is enabled for the project:

```
gcloud billing projects describe "$PROJECT_ID" --format='value(billingEnabled)'
```

The output must be `True`. If it is `False`, go to ☰ → **Billing** → **Account management** and link the project to your Free Trial billing account, then run the command again.



Done when: step 2 shows your account, step 4 matches the top bar, and step 5 prints `True`. Keep the screenshot of `True` for Issue #1.

Reference: Launch Cloud Shell

1.5 Set a budget

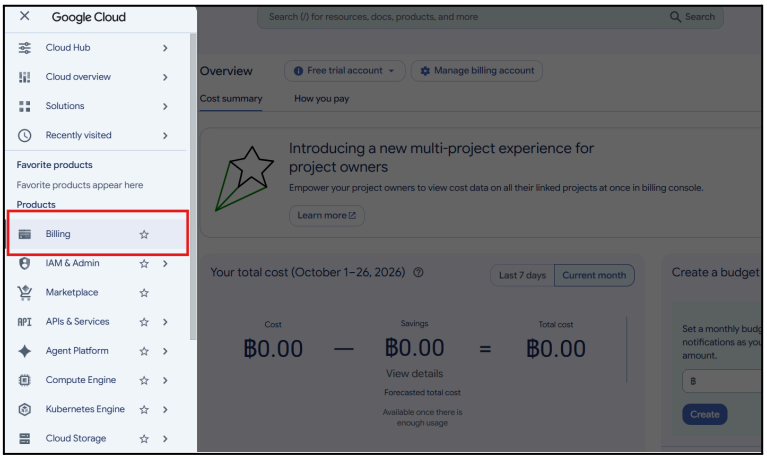
A budget sends email alerts when your spending reaches a set amount. A budget **only sends alerts**; It **does not stop spending**. Expect about USD 14–18 per month per project, mostly Cloud SQL (Oct 2026 estimate, before tax and credits).

Before you begin

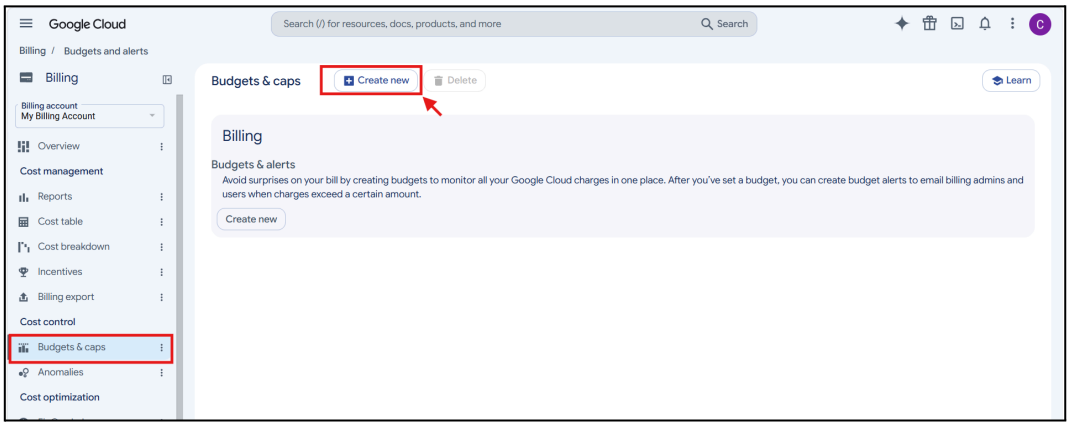
- Finish 1.4 Billing must print `True`.
- Agree with your team on a monthly amount, for example, USD 10, and who should receive alerts.

Steps

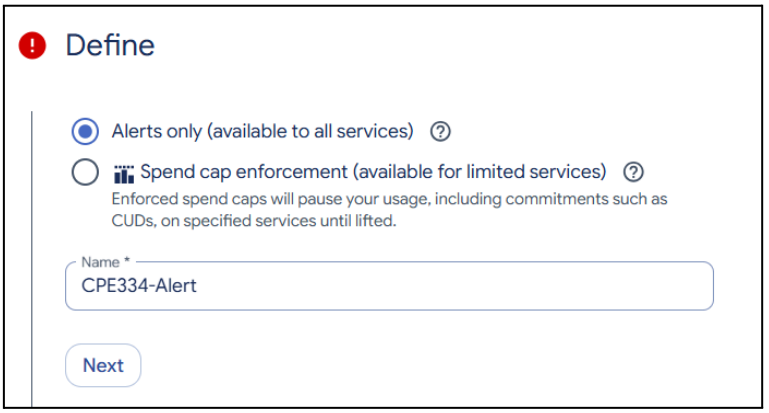
Start at **console.cloud.google.com/billing** and choose your Free Trial billing account (usually **My Billing Account**).



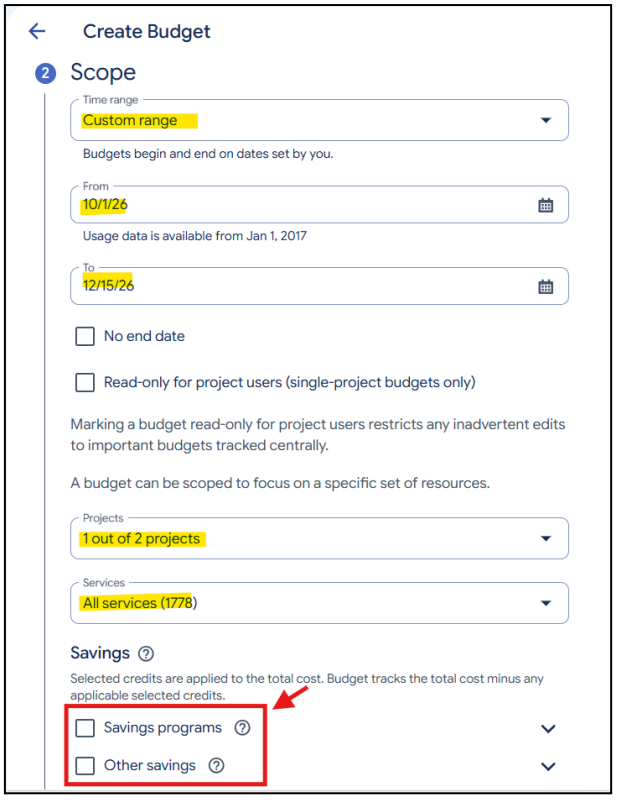
1. In the left menu, under **Cost control**, click **Budgets & caps**, then click **Create new**. If it asks what to create, choose an **alerts-only budget**.



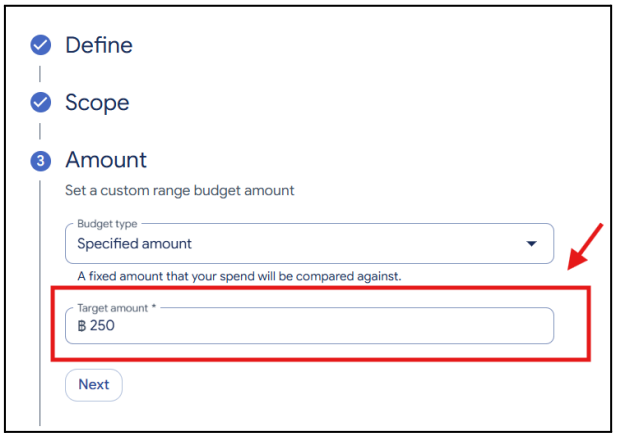
2. **Define**
   - Choose **Alerts only (available to all services)**. Do **not** choose **Spend cap enforcement**. It pauses services when the cap is reached, and your app could stop during class or while it is being graded.
   - **Name**: for example, `CPE334-Alert`
   - Click **Next**.



3. **Scope**
   - **Time range**: **Custom range**. Start date: today. End date: **15 December 2026** (or your Final Project deadline).
   - **Projects**: select **only your team's project**.
   - **Services**: leave **All services**.
   - **Savings** (discounts and credits): **clear all of them**, so the budget counts your actual usage while the free credit pays the bill.
   - Click **Next**.



4. **Amount**
    - **Budget type**: **Specified amount**.
    - **Target amount**: choose an amount that suits your team. **The values below are an example, not a rule.**
    - Click **Next**.

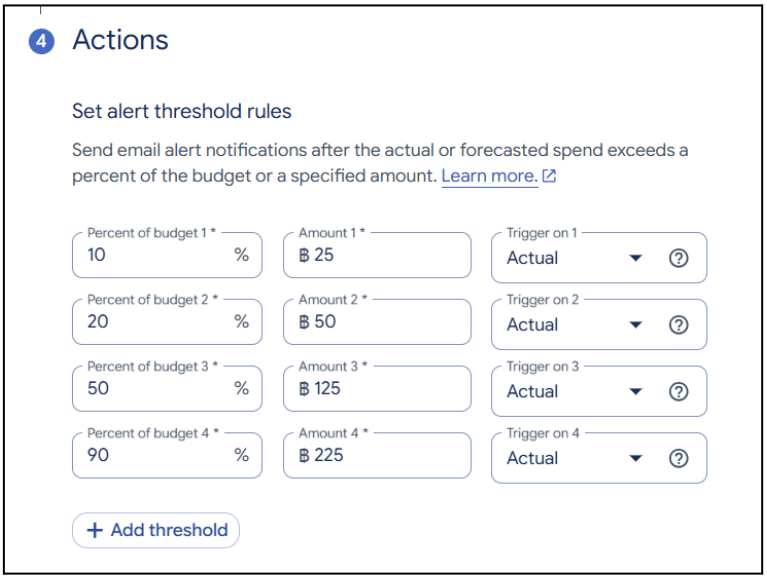


    - **Where the example of USD 250 comes from**
        i. The Free Trial gives **USD 300** of credit for 90 days.
        ii. This project runs until mid-December, about 1.5 months. At about USD 14–18 per month, it should cost about **USD 21–27** in total.
        iii. USD 250 is most of the credit, with about USD 50 kept in reserve. If spending reaches 90% (USD 225), you still have time to find the problem before the credit runs out.

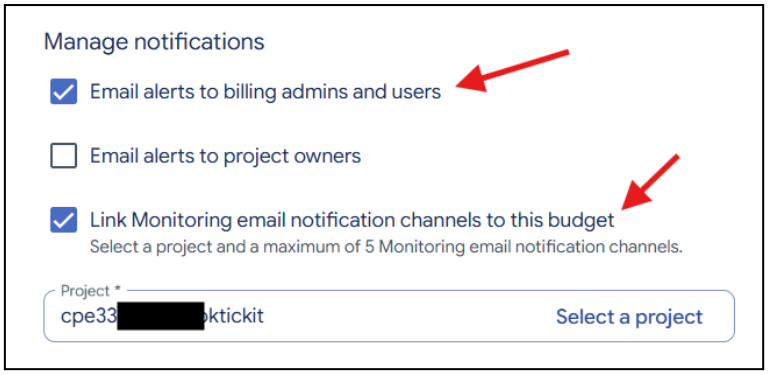
iv. **Your team can choose a lower amount**, such as USD 30–50, if you want the percentages to be closer to what you expect to spend. Your app, region or resources may also cost more or less than this example.

5. **Actions**
   - Set thresholds on **Actual** cost. Example for a USD 250 budget:

Threshold	Amount	What it means
10%	**USD 25**	About what the whole project should cost
20%	**USD 50**	About twice the expected cost: check what is running
50%	**USD 125**	Clearly unusual: find the cause now
90%	**USD 225**	The credit is close to running out



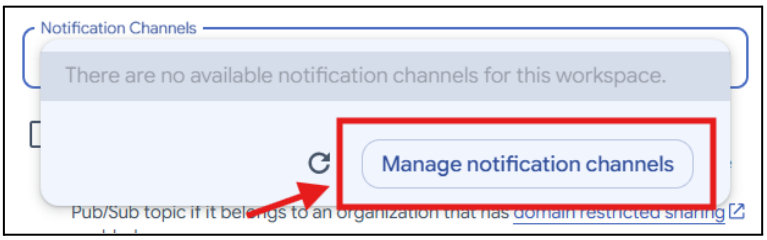
   - If you choose a different amount, adjust the percentages so the first alert is near your expected total cost.
   - **Manage notifications**
     - **Email alerts to billing admins and users**: keep it ticked. This sends alerts to you, the person who created the Free Trial billing account.
     - **Email alerts to project owners**: optional. In this lab you are the only owner, so this sends you the same email again.
     - **Link Monitoring email notification channels to this budget**: tick this to alert your teammates. They have the Editor role (1.3), so the two options above do not email them. Select your project, then up to **5** email channels, one per teammate. If there are no channels yet, see below.
     - **Connect a Pub/Sub topic to this budget**: leave it unticked. It is for sending alerts to programs, not people.



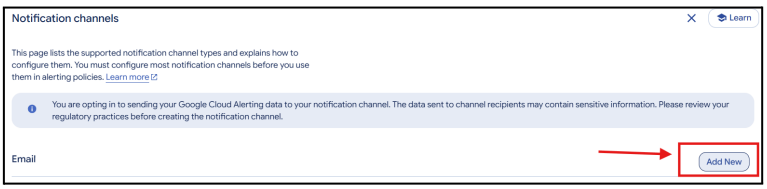
- **Create email channels for your teammates (once)**

  If the list says **There are no available notification channels for this workspace**, create them first:

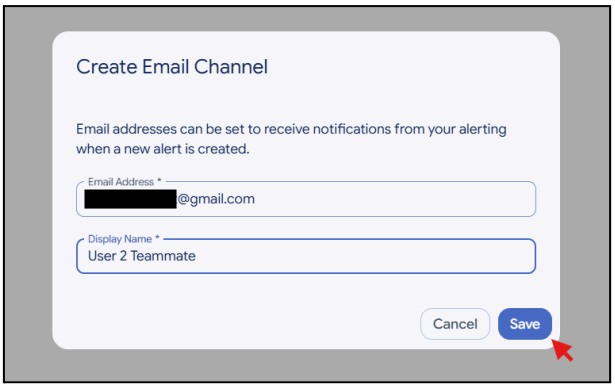
  - Click **Manage notification channels**. A new tab opens in Cloud Monitoring. The first visit can take a moment to load.



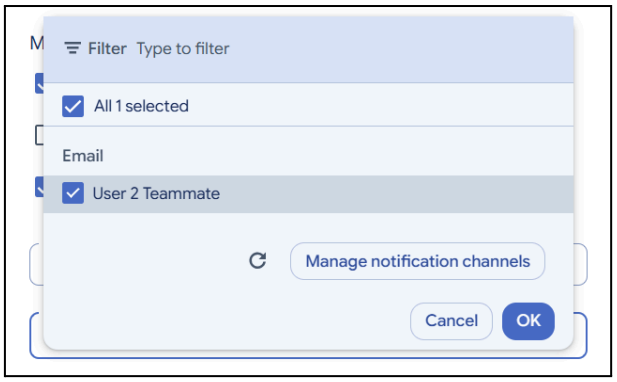
  - Next to **Email**, click **Add new**.



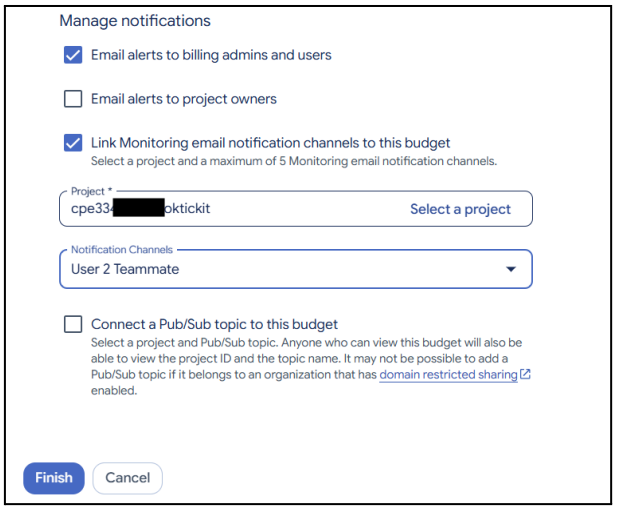
  - Enter a teammate's Gmail address and a display name, then click **Save**. Repeat for each teammate (up to 5).



- Go back to the budget tab and click the refresh button (↻) next to Manage notification channels.
- Select your teammates' channels in the list.



6. Click **Finish**. The budget appears in the list.



Done when: the budget appears in the list with your project and amount. Close Issue #1 with screenshots of Credits (1.1), the budget, and the `True` result (1.4).

Reference: Create, edit, or delete budgets and budget alerts

Guide/Episode 2: From local to cloud – what changes

Videos: none (self-check)

Prerequisite: the Sprint 1 app runs locally

Issue #2 Branch: refactor/2-cloud-ready (example)

Criterion: Docker and reproducible configuration

In Labs 1–4, you learned to build an app that runs on your laptop. In Lab 5, you take your team's term project app to Google Cloud. This guide uses TokTickIT, the course example app, to show each step; apply the same steps to your own app.

Cloud Run works differently from your laptop: each new deploy starts a fresh container, files saved inside the container disappear, Cloud Run chooses the port, and anyone on the internet can reach the app. This episode checks six things that work locally but break in the cloud.

2.1 Create the branch

Before you begin

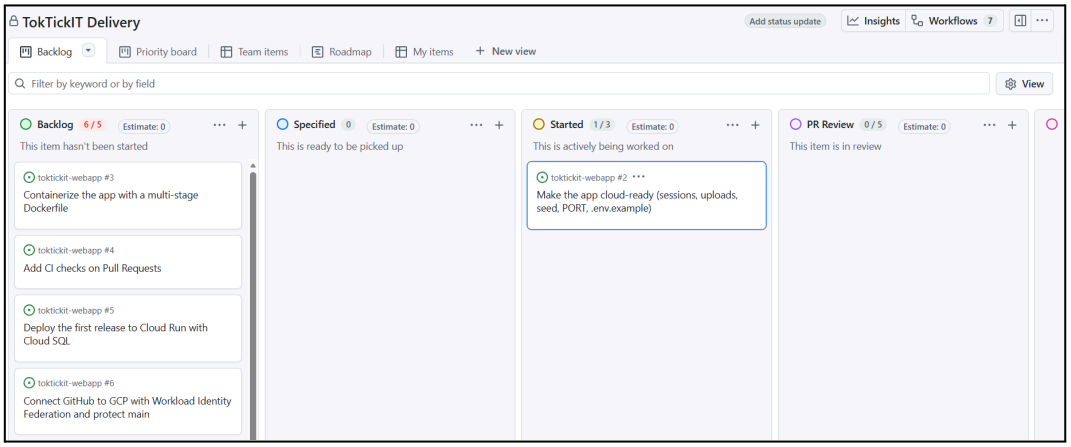
- Open your team's repo in VS Code, then open a terminal (**Terminal** → **New Terminal**). The terminal must be in the root folder of the repo.
- Check that you have no unsaved work:

```
git status
```

It should say `nothing to commit, working tree clean`. If not, commit or stash your changes first.

Steps

1. On your project board, move **Issue #2** to **In Progress** and assign it to yourself.



2. Update `staging` and create a branch from it:

```
git switch staging
git pull
git switch -c refactor/2-cloud-ready
```

`refactor/2-cloud-ready` is an example name. Use your team's naming style, and keep the Issue number in it.

3. Check that you are on the new branch:

```
git branch --show-current
```

The output must be the branch name you just created, for example `refactor/2-cloud-ready`. Always branch from `staging`, not `main`. Your pull request goes back into `staging` in 2.5.

2.2 Check six points

Goal: find and fix what works on your laptop but breaks, or becomes unsafe, on Cloud Run. It is much easier to fix now than after you deploy. Check each point in your own code, fix what does not match in this branch, and copy this checklist into your pull request (2.5).

Checklist

- ☐ **1. Uploaded files are not saved in a local folder.**
  Store them in the database or Cloud Storage. Files saved inside the container disappear on every deploy.

  In *TokTickIT: already OK. Files are saved as bytes in the database (`multer.memoryStorage()` in `app.ts`).*

- ☐ **2. The seed is safe for production.**
  Demo accounts are allowed (you will need them for UAT in Lab 6), but their passwords come from environment variables, never from code, and are not shown on screen. The seed is safe to rerun.

  In *TokTickIT: already OK (`server/prisma/seed.ts`).*

- ☐ **3. No dev shortcuts in production.**
  The Development Requester selector (or any way to pick a user without logging in) is off, and no default password is shown on the login page. If admins can set a default password for new users, the user must change it at first login.

  In *TokTickIT: already OK. The legacy user header is off by default, and the default password for new users must be changed at first login.*

☐ **4. Sessions are stored in the database, not in server memory.**
Otherwise, users are logged out on every deploy or when Cloud Run adds an instance.

In *TokTickIT: already OK (*`server/src/auth.ts`*).*

☐ **5. The server reads its port from `PORT`.**
Cloud Run chooses the port. Do not bind only to `localhost`.

In *TokTickIT: already OK (*`server/src/index.ts`*).*

☐ **6. `/api/ready` exists.**
It returns 503 if the database cannot be reached. The smoke test uses it after every deploy.

In *TokTickIT: already OK (*`server/src/app.ts`*).*

If your app has none, see the example below.

If your app has no `/api/ready`, add one. Example for Express and Prisma:

app.get("/api/ready", async (_req, res) => {
  try {
  await prisma.$queryRaw`SELECT 1`;
  res.json({ status: "ready", db: "connected" });
  } catch {
  res.status(503).json({ status: "not_ready" });
  }
});

Test it with your app running: open `http://localhost:3000/api/ready` in the browser. It should show `ready`.

2.3 Complete .env.example

Goal: anyone can copy `.env.example` to `.env` and know every setting the app needs, without seeing real secrets. Cloud Run will set these same variables.

1. Find every environment variable your code reads. In Node.js, search for `process.env` (and `import.meta.env` in a Vite client). In other languages, search for the equivalent, for example, `os.environ` in Python or `System.getenv` in Java.
2. Make sure each one is listed in `server/.env.example` or `client/.env.example`, with a brief comment and an example value.
3. Never put real passwords or connection strings in this file.

In *TokTickIT, the* server reads 7 variables, but `server/.env.example` listed only `DATABASE_URL` and `PORT`. The fix adds the rest as comments, so local runs behave as before:

```
# Runtime mode. The cloud uses "production."
# NODE_ENV=development
```

```
# First admin password for the seed. Required when NODE_ENV=production.
# ADMIN_PASSWORD=change-me

# Demo users and sample tickets. Default: on locally, off in production.
# SEED_DEMO_DATA=false
# DEMO_USER_PASSWORD=change-me

# Allowed web origins, comma-separated (optional).
# CORS_ORIGIN=http://localhost:5173

# Old x-requester-id header for legacy tests. Never enable it in the cloud.
# ALLOW_LEGACY_REQUESTER_HEADER=false
```

Each line starts with #, so it is a comment: **copying this file to .env changes nothing on your laptop. Remove the # only when you want to set that variable.** Values such as change-me are placeholders, never real passwords. When you deploy to Cloud Run, Cloud Run sets `NODE_ENV` to `production` and reads `ADMIN_PASSWORD` from Secret Manager.

2.4 Run your tests.

Goal: make sure your changes did not break anything.

Start your local database the way your team normally does. Run your app's tests and build it. All must pass before you open a pull request.

2.5 Submit

1. Commit and push your branch:

```
 git add -A
git commit -m "Prepare the app for the cloud (#2)"
git push -u origin refactor/2-cloud-ready
```

2. Open a pull request into **`staging`**. In the description, write `Part of #2` and paste the checklist from 2.2. Tick each point and add a short note: what you changed, or "already OK".
3. A teammate reviews and approves it. Then merge.
4. On the project board, move Issue #2 to **Done**.

Done when: all six points are ticked, `.env.example` lists every variable the app reads, the tests pass, and the pull request is merged into `staging`.

References: Cloud Run container runtime contract (port, listening address, and why files in the container are lost) · The Twelve-Factor App: Config (why settings go in environment variables)

Guide/Episode 3: Docker

Videos: LEB2 / Learn It / Student Guide: GCP Deployment and CI/CD (Video) / Episode 3

Prerequisite: Episode 2

Issue #3 Branch: build/3-docker (example name; keep the Issue number)

Criterion: Docker and reproducible configuration

Goal: Package your app in Docker files and, if you can, run it on your computer with Docker Compose, so you know the container setup works before CI (Episode 4) and Google Cloud (Episode 5) use it. Nothing is deployed in this episode and nothing costs money.

3.1 What is required and what is recommended.

Required: the Docker files are in your repository and the Pull Request is open (steps 3.2, 3.3 and 3.6).

Recommended: run them on your computer (steps 3.4 and 3.5). If you cannot install Docker, you can skip these two steps, but then you find problems later, in CI or on Google Cloud.

Adapt to your project. The files come from the TokTickIT example (Node.js, React, PostgreSQL). If your team uses another stack, keep the same idea and change only the commands inside the files. Step 3.3 lists the lines to change.

Before you begin

- Episode 2 is merged into `staging`.
- Your terminal is open in the root folder of your repository.
- You have unzipped the Starter Kit **outside** your repository.
- For steps 3.3 and 3.4 only: Docker is installed and running. Check with `docker --version` and `docker compose version`. If you have no Docker Desktop, use Google Cloud Shell, which already has both.

3.2 Create the branch and copy the kit files

Before you begin: `git status` is clean.

Steps

1. Get the latest `staging`, which includes Episode 2:

```
git switch staging
git pull
```

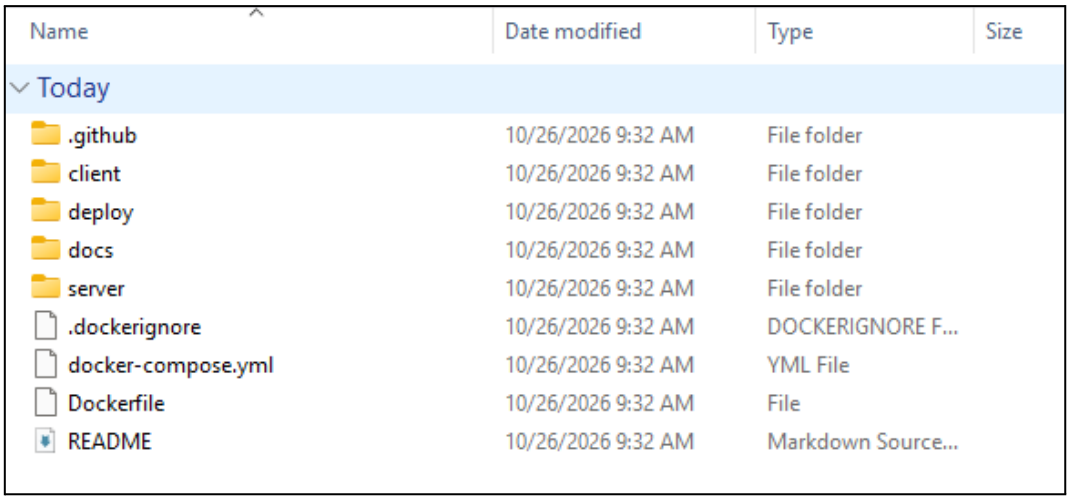
2. Create the branch:

```
git switch -c build/3-docker
```

```
PS C:\Users\CPE KMUTT\Documents\GitHub\toktickit-webapp> git switch -c buil
d/3-docker
Switched to a new branch 'build/3-docker'
```

3. Copy these 7 files from the Starter Kit into the same places in your repository:

File	What it does
`docker-compose.yml`	Starts the database, the setup job, the API and the web app
`server/Dockerfile`	Builds the API image used by Compose
`server/docker-entrypoint.sh`	Starts the server inside the container
`client/Dockerfile`	Builds the web app
`client/nginx.conf`	Serves the built web app with nginx
`Dockerfile`	Root image that CI and Cloud Run use later (Episodes 4 and 5)
`.dockerignore`	Keeps `node_modules`, `.env` and other files out of the images



Starter Kit Treefile

```
toktickit-lab5-starter/
├── .dockerignore                ← Ep 3
├── Dockerfile                   ← Ep 3
├── docker-compose.yml           ← Ep 3
├── README.md
├── .github/
│   ├── dependabot.yml
│   └── workflows/
│       ├── ci-docker.yml        ← Ep 4
│       └── deploy.yml           ← Ep 7
├── client/
│   ├── .env.example
│   ├── Dockerfile               ← Ep 3
│   └── nginx.conf               ← Ep 3
├── server/
│   ├── .env.example
│   ├── Dockerfile               ← Ep 3
│   └── docker-entrypoint.sh     ← Ep 3
├── deploy/
│   ├── deploy.sh                ← Ep 5
│   ├── release.sh               ← Ep 7
│   ├── smoke_test.py                  ← Ep 7
│   ├── teardown.sh              ← Teardown
│   └── tests/
│       ├── test_deploy_stub.sh
│       ├── test_release_stub.sh
│       └── test_smoke.py
└── docs/
    ├── auto-deploy.md
    ├── gcp-cicd-guide.md
    └── gcp-first-lab.md
```

4. Run `git status` and check that exactly these 7 files are new.

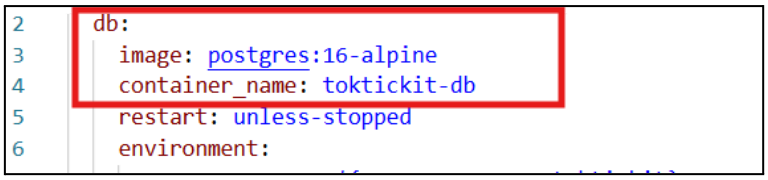
```
PS C:\Users\CPE KMUTT\Documents\GitHub\toktickit-webapp> git status
On branch build/3-docker
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .dockerignore
        Dockerfile
        client/Dockerfile
        client/nginx.conf
        docker-compose.yml
        server/Dockerfile
        server/docker-entrypoint.sh
```

3.3 Read the files before you run them

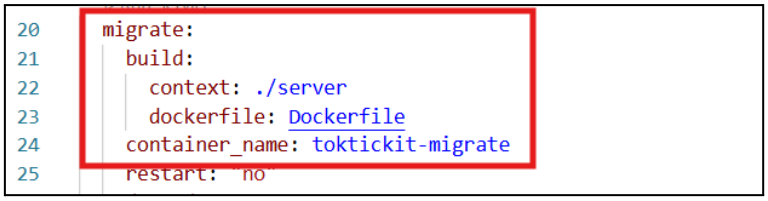
Before you begin: The 7 files are in your repository.

Steps

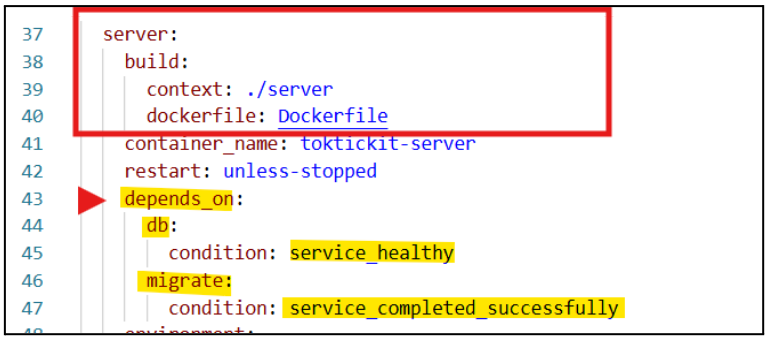
1. Open `docker-compose.yml`. Find the four services and say what each one does:
   - `db`: PostgreSQL with a volume, so data survives a restart.



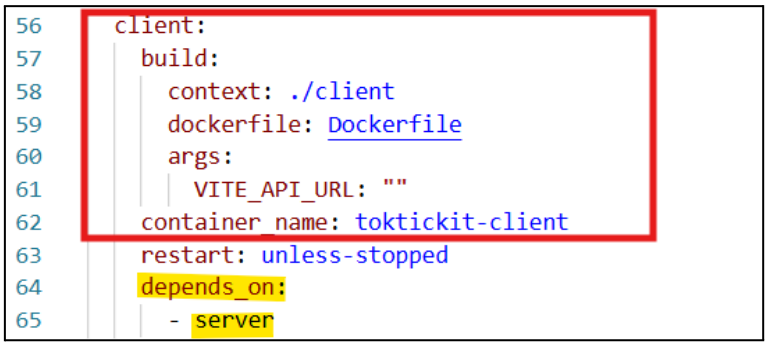
   - `migrate`: runs once, applies the database migrations and the seed, then stops.



   - `server`: the API on port 3000. It starts only after `migrate` has finished.



   - `client`: the web app served by nginx, on port 5173.



2. Check three things:
   - No real password or secret is written in any Dockerfile (`ARG` or `ENV`).

    - The passwords in `docker-compose.yml` are fallbacks for your own computer only. The cloud gets its values from Secret Manager in Episode 5.
    - The final image contains every file the server and the seed need at run time (for TokTickIT: `src/`, `prisma/`, `node_modules`).
3. **If your stack is different**, change only these lines:
    - the base image and the install and build commands in each Dockerfile;
    - the port and the start command (`CMD`);
    - the database image and the `DATABASE_URL` format in `docker-compose.yml`;
    - the command of the `migrate` service (your own migration and seed commands).

Reference – If your project uses a different stack

The Starter Kit Dockerfiles are written for Node.js. If your team uses another language or framework, write your own Dockerfile from the official guides instead of editing ours.

How to write a Dockerfile

- Writing a Dockerfile – the basic instructions (`FROM`, `COPY`, `RUN`, `CMD`)
- Dockerfile best practices – small images, build cache, multi-stage builds
- Dockerfile reference – every instruction in detail

Docker's guide for your language
(a recommended base image, Dockerfile and Compose file for each)

- Node.js · Python · Java · Go · PHP · .NET · Ruby · Rust

Google Cloud's guide for your language (what Cloud Run expects from the app)

- Containerize your code for Cloud Run
- Quickstarts: Node.js · Python · Java · Go · PHP · Ruby · C# · Other languages
- Frameworks: Next.js · Python Flask · Python FastAPI · Node.js Express

Whatever the stack, your image must: listen on the port in the `PORT` environment variable on `0.0.0.0`, read all settings from environment variables, and keep no data on the container's disk. See the Cloud Run container runtime contract.

3.4 Build and start everything (recommended)

Before you begin: Docker is running. Ports 3000 and 5173 are free. If you also run PostgreSQL on your computer, port 5432 must be free too.

Steps

1. Build the images and start the containers in the background:

```
docker compose up --build -d
```

* The first build takes a few minutes because Docker downloads the base images.

```
#33 [client] resolving provenance for metadata file
#33 DONE 0.0s
[+] up 8/8
 ✓ Image toktickit-webapp-server    Built                                3.5s
 ✓ Image toktickit-webapp-client    Built                                3.5s
 ✓ Image toktickit-webapp-migrate   Built                                3.5s
 ✓ Network toktickit-webapp_default Created                              0.2s
 ✓ Container toktickit-db           Healthy                              8.7s
 ✓ Container toktickit-migrate      Exited                              14.5s
 ✓ Container toktickit-server       Started                             14.7s
 ✓ Container toktickit-client       Started                             15.0s
```

2. Check the state:

```
docker compose ps -a
```

```
PS C:\Users\CPE KMUTT\Documents\GitHub\toktickit-webapp> docker compose ps -a
NAME               IMAGE                     COMMAND                  SERVICE   CREATED          STATUS
          PORTS
toktickit-client   toktickit-webapp-client   "/docker-entrypoint.…"   client    11 minutes ago   Up 13 minutes
          0.0.0.0:5173->80/tcp, [::]:5173->80/tcp
toktickit-db       postgres:16-alpine        "docker-entrypoint.s…"   db        11 minutes ago   Up 13 minutes (he
althy)    0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
toktickit-migrate  toktickit-webapp-migrate  "./docker-entrypoint…"   migrate   11 minutes ago   Exited (0) 13 min
utes ago
toktickit-server   toktickit-webapp-server   "./docker-entrypoint…"   server    11 minutes ago   Up 13 minutes
          0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
PS C:\Users\CPE KMUTT\Documents\GitHub\toktickit-webapp>
```

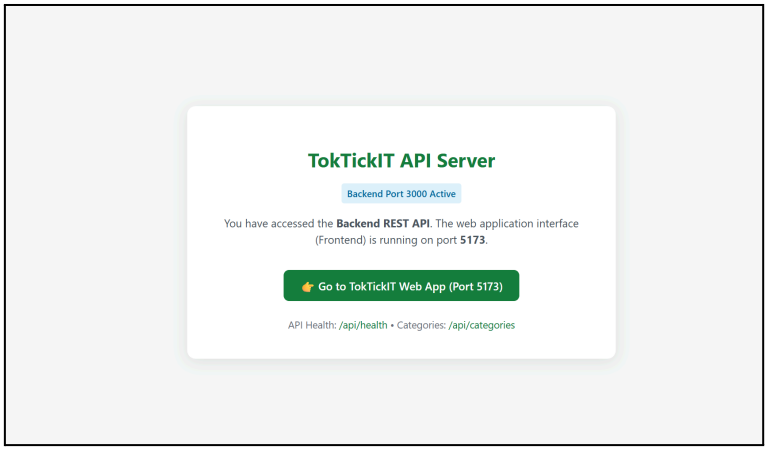
`db`, `server` and `client` must be running, and `migrate` must show **exited (0)**. A non-zero exit code means the migration or the seed failed (see the table at the end).

3. Check that the API is alive and connected to the database:

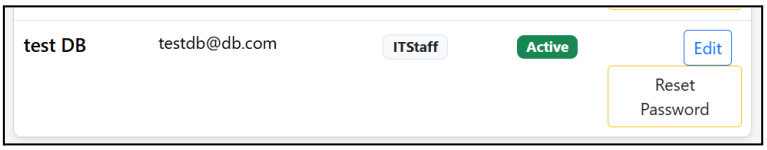
```
curl --fail http://localhost:3000/api/health
curl --fail http://localhost:3000/api/ready
```

*On Windows PowerShell, type `curl.exe` instead of `curl`.

```
PS C:\Users\CPE KMUTT\Documents\GitHub\toktickit-webapp> curl.exe --fail http://localhost:3000/api/health
{"status":"ok","service":"TokTickIT API"}
PS C:\Users\CPE KMUTT\Documents\GitHub\toktickit-webapp> curl.exe --fail http://localhost:3000/api/ready
{"status":"ready","db":"connected"}
```



4. Create one record in your app (for TokTickIT, a ticket) so that you can test persistence in the next step.



3.5 Check that the data survives a restart (recommended)

Before you begin: You created a record in step 3.4.

Steps

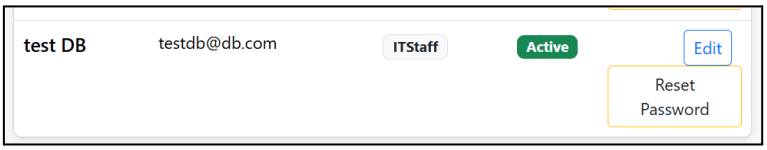
1. Stop the containers but keep the data.

```
docker compose down
```

2. Start them again:

```
docker compose up -d
```

3. Sign in again. The record you created is still there, because it lives in the database volume, not in a container.



4. If you want a clean database later (for example after a failed first run), delete the volume too:

```
docker compose down -v
```

3.6 Commit and open a Pull Request

Before you begin: `git status` shows only the 7 Docker files.

Steps

1. Commit and push
2. On GitHub, open a Pull Request from `build/3-docker` into **`staging`**.
3. In the description write `Part of #3`, then write what you checked: all containers start, `/api/health` and `/api/ready` answer, a record survives a restart. If you skipped steps 3.4 and 3.5, write that instead.
4. Move Issue #3 to *In review*. Merge when your team has reviewed it

Done when

- *The 7 Docker files are in your repository and contain no real secret.
- The Pull Request into `staging` is open with `Part of #3`.
- If you ran Docker: `docker compose up --build -d` starts everything without errors, `/api/health` and `/api/ready` answer, you can sign in, and your record is still there after `down` and `up`.

***Note**: What "done" looks like depends on your project and the stack your team chose. The list above uses TokTickIT as the example (Node.js, React, PostgreSQL, ports 3000 and 5173, /api/health and /api/ready). Adapt each item to your own app: your own files and ports, your own health endpoint (or another way to show the app is running), your own database, and your own way to ensure that data survives a restart.
What must stay true for every team: the app starts from Docker files in the repository, it reads its settings from environment variables, no real secret is committed, and data is kept in the database and not in the container.

If something fails

What you see	What to check
`migrate` exited with a non-zero code	`docker compose logs migrate`. Usually a wrong `DATABASE_URL`, a missing migration file, or a seed that needs an environment variable
Port already in use	Stop the other program, or change the left side of `"3000:3000"` or `"5173:80"`
The page opens but sign-in fails	`docker compose logs server`. Check that `migrate` finished and the seed ran
Old data or old passwords keep appearing	`docker compose down -v`, then start again

Official references

- Dockerfile best practices – for adapting the Dockerfiles to your stack
- Docker Compose overview – the `up` and `down` commands used here
- Control startup order with `depends_on` – why `server` waits for `migrate`
- PostgreSQL image on Docker Hub – environment variables, or a different database version
- Cloud Run container runtime contract – what the image must do in Episode 5

Guide/Episode 4: CI on pull requests

Videos: LEB2 / Learn It / Student Guide: GCP Deployment and CI/CD (Video) / Episode 4

Prerequisite: Episode 3

Issue #4 Branch: ci/4-pr-checks (example name; keep the Issue number)

Criterion: CI and quality gate

Goal: Make GitHub run your tests and a Docker build on every Pull Request, so broken code is caught before it is merged. This is the first part of the "CI and quality gate" criterion. In Episode 6 you will turn the result into a rule that blocks merging. No Google Cloud is used in this episode and nothing costs money.

> **Adapt to your project.** The workflow file is written for TokTickIT (Node.js, PostgreSQL, Prisma). If your team uses another stack, keep the same structure and change only the install, test and build commands. Step 4.2 lists what to change.

Before you begin

- The Pull Request from Episode 3 is merged into `staging`.
- Your repository is on GitHub, and you can open the **Actions** tab.
- You have unzipped the Starter Kit outside your repository.
- Your terminal is open in the root folder of your repository, and `git status` is clean.

4.1 Create a branch and copy the CI files

Before you begin: Episode 3 is merged.

Steps

1. Get the latest **staging**
2. Create the branch: ci/4-pr-checks
3. Copy two things from the Starter Kit into your repository:
   - `.github/workflows/ci-docker.yml` into the same place (create the folders if they do not exist)
   - the whole `deploy/` folder
4. Run `git status` and check that the new files are .github/workflows/ci-docker.yml and the deploy/ folder.

> **Why deploy/ now?** **The workflow checks the deploy scripts and runs their offline tests, so it fails without the folder. The scripts do nothing until you run them yourself in Episode 5.**

```
PS C:\Users\CPE KMUTT\Documents\GitHub\toktickit-webapp> git status

On branch ci/4-pr-checks
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .github/
        deploy/
```

4.2 Read the workflow and edit the .github/workflows/ci-docker.yml

Before you begin: The files are copied.

Why do we edit this?

The kit's workflow runs only for Pull Requests into **main**. In this guide every change goes into **staging first**, and main receives only release Pull Requests. If you leave the file as it is, your Pull Requests into staging get no check, and the first time CI runs is the release. Adding staging makes CI check each change before it is merged.

This edit may not be needed for your team. Change the list to match your own branching:

- If your team opens Pull Requests straight into `main`, keep `[main]`.
- If your team uses a different integration branch (for example `develop`), put that name in the list.
- If you use more than one, list them all.

The rule is simple: the `branches` list must contain every branch that your Pull Requests are opened into.

What the other two lines in `on:` are for

- `workflow_call` lets another workflow reuse this one. In Episode 7 the deploy workflow calls it, so the same tests run again before every release. Leave it as it is.
- `workflow_dispatch` adds a **Run workflow** button in the Actions tab, so you can start the checks by hand without opening a Pull Request. It is useful for testing the workflow itself. You can remove it if you do not want it.

Only the `pull_request` lines decide which Pull Requests get checked.

Two more settings near the top of the file

- `concurrency` makes sure only one run of the checks is active per branch. With `cancel-in-progress: true`, when you push a new commit while a run is still going, GitHub cancels the old run and starts a new one. This saves time and runner minutes. A cancelled run is not a failure.
- `permissions: contents: read` gives the workflow read-only access to your repository. It cannot push code, change settings or create

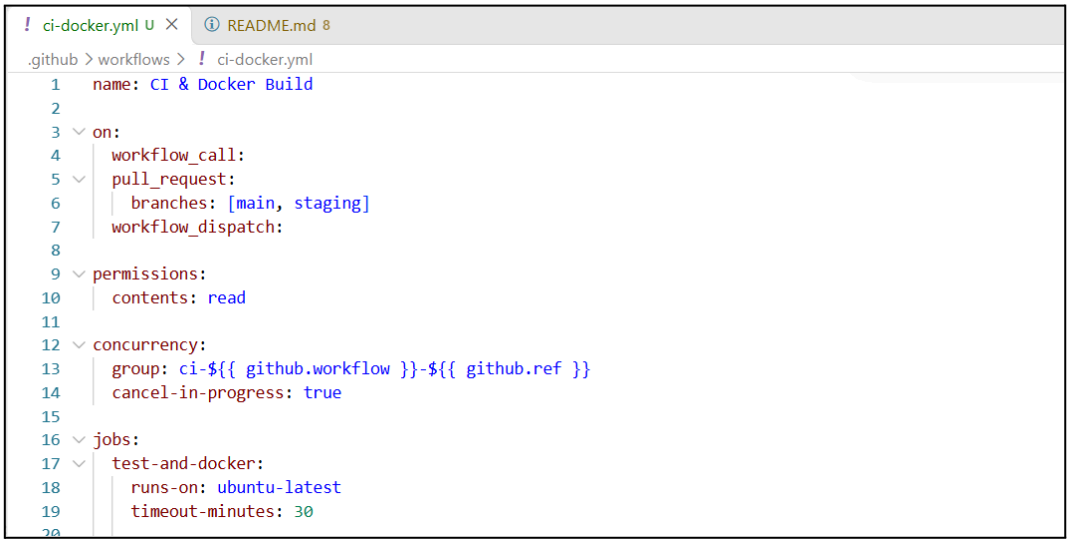
Steps

1. Open `.github/workflows/ci-docker.yml`. Find the `on:` section at the top and change the branches so that it runs for Pull Requests into both `staging` and `main`:

```
on:
  workflow_call:
  pull_request:
    branches: [main, staging]
  workflow_dispatch:
```

2. Read the job `test-and-docker` and say what each part does:
   - **services: postgres:** starts a temporary PostgreSQL database for the tests.
   - **Check the deploy scripts:** runs offline tests that never touch Google Cloud.
   - **Install and test the backend:** applies the migrations, runs the seed, runs the tests.
   - Install, test and build the frontend.
   - **Verify the Docker build:** runs `docker build` on the root `Dockerfile` and does not push it anywhere.
3. Check three things:
   - **permissions is contents: read** only. The workflow cannot change your repository.
   - The workflow does not sign in to Google Cloud. That comes in Episode 7.
   - The job is named `test-and-docker`. Keep this name, because Episode 6 uses it as the required check.
4. **If your stack is different**, change only these steps:
   - the database service (image, user, password) to match your database;
   - the setup step (for example `pip install`, `mvn`, `dotnet restore`) and the **commands** that run your migrations, seed and tests;
   - the frontend step, if your frontend has a different build **command**;
   - the `docker build` path, if your Dockerfile is not in the root.

   Keep the job name, the `on:` section and the permissions.

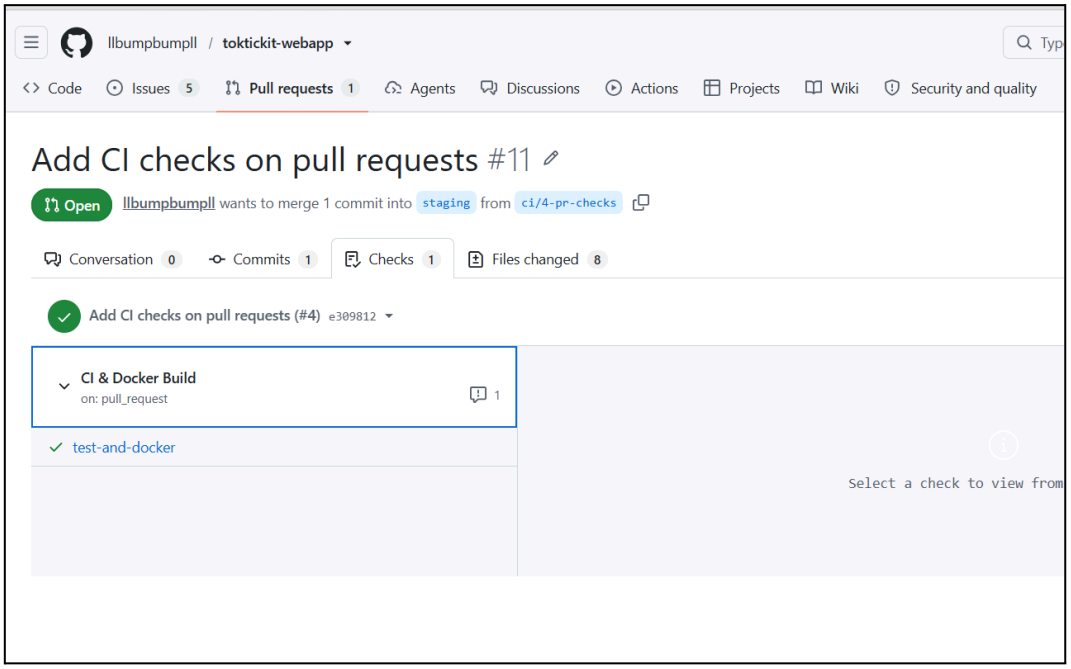


4.3 Commit and open a Pull Request into staging

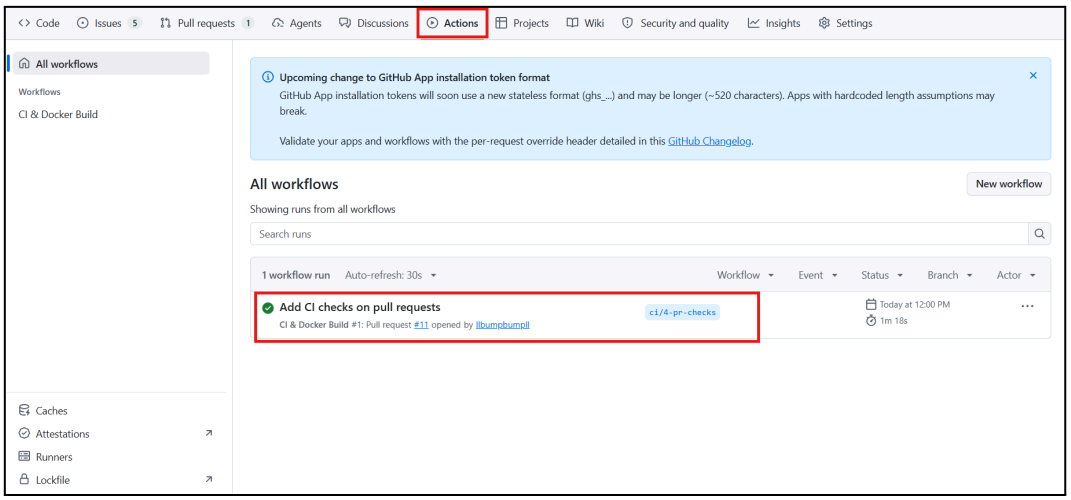
Before you begin: The `on:` section is edited.

Steps

1. Commit and push: ci/4-pr-checks
2. On GitHub, open a Pull Request from `ci/4-pr-checks` into **`staging`**. In the description write `Part of #4`.
3. Scroll to the checks. A check named **CI & Docker Build / test-and-docker** starts by itself. It takes a few minutes.



4. Wait until it is green. Open **Details** (or the **Actions** tab) and expand the steps to see each one pass.



4.4 If the check fails

Before you begin: The check shows a red cross.

Steps

1. Open **Details**, find the first step with a red cross, and expand it. Read the last lines of its log.
2. Fix the problem in your branch, commit and push again. The check runs again by itself.
3. Common causes are in the table at the end of this episode.

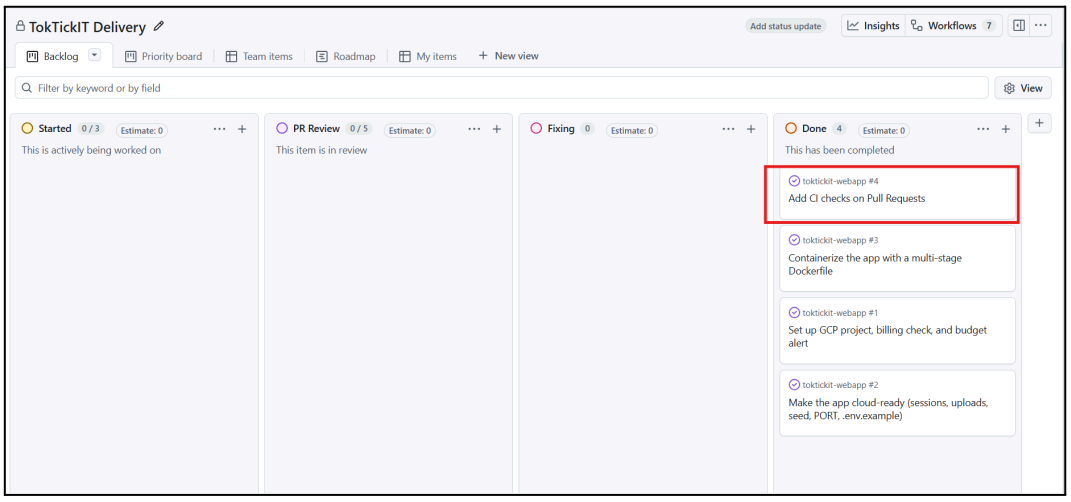
4.5 Merge, then make the first release to main

Before you begin: The check on the `staging` Pull Request is green.

Steps

1. Get an approval from a teammate if your team requires one, then merge the Pull Request into `staging`.
2. Open a **release Pull Request** from `staging` into `main` (title for example “Release: CI and Docker files”). The same check runs again. When it is green and approved, merge it.
3. This puts the workflow and the `deploy/` folder on `main`. You need them there in Episodes 5, 6 and 7.
4. Move Issue #4 to *Done*.





Done when

- A Pull Request into `staging` starts the `test-and-docker` check by itself and it passes.
- A Pull Request from `staging` into `main` starts the same check and it passes.
- `.github/workflows/ci-docker.yml` and `deploy/` are on `main`.

Note: What passes depends on your project and your stack. The checks above use TokTickIT as the example. For your own app, "passes" means your own install, migration, test and Docker build steps all succeed, and the job keeps the name `test-and-docker`.

If something fails

What you see	What to check
No check appears on the Pull Request	The workflow file must be in `.github/workflows/` on the branch, and the `on:` branches must include the branch the Pull Request targets
`deploy/` step fails with "No such file"	The `deploy/` folder was not copied or not committed. Run `git status`
Backend step fails on `prisma migrate deploy`	`DATABASE_URL` in the workflow must match the `services: postgres` values
Tests pass on your computer but fail in CI	CI starts with an empty database and a clean install. Check that the tests do not depend on data or files only you have
Docker build step fails	Run `docker build .` on your computer and read the error. It is the same build
Workflow does not start for a first-time contributor	In the repository settings, Actions must be enabled

Official references

- Building and testing Node.js – the pattern this workflow follows. Other stacks: Python · Java with Maven · Java with Gradle · .NET · Go · Ruby
- Creating PostgreSQL service containers – the temporary database used by the tests
- Workflow syntax: `on.pull_request` – which branches start a workflow

Guide/Episode 5: First deployment to GCP

Videos: LEB2 / Learn It / Student Guide: GCP Deployment and CI/CD (Video) / Episode 5

Prerequisite: Episode 1, 4

Issue #5 (no branch: you run a script; you do not change code)

Criterion: GCP deployment and persistence

Goal: Run the deploy script once, by hand, from Cloud Shell, so your app runs on Cloud Run with its data in a Cloud SQL database. From Episode 7 a pipeline does this for you; this first run lets you see and understand every resource.

> **Costs start in this episode.** Cloud SQL is billed for every hour it exists, even when nobody uses the app. Do Episodes 5 to 8 close together (ideally on the same day) and run the teardown in the last episode when you finish. Check that your budget alert from Episode 1 is in place before you start.

> **Adapt to your project.** `deploy/deploy.sh` is written for TokTickIT (PostgreSQL, one container on port 3000, Prisma migrations). If your stack differs, change the script before running it. The lines to change are listed in the Starter Kit `README.md` under “What to change for your project”, and in Episode 0, section 0.4.

Before you begin

- Your Free Trial is **active,** and billing is enabled (Episode 1). If the trial is still *Pending*, do only 5.1 and 5.2 (they create nothing) and stop before 5.3.
- Your budget alert exists.
- The release Pull Request from Episode 4 is merged, so `main` contains your app, the Docker files, `.github/workflows/ci-docker.yml` and the `deploy/` folder.
- You know your **Project ID** (not the project name) and the region you will use, for example `asia-southeast1`.
- You have a quiet place to work: this episode shows a generated password (5.6), which you must never show on screen, in a screenshot or in a chat.

5.0 Adapt deploy/deploy.sh script

`deploy/deploy.sh` is written for TokTickIT (Node.js, PostgreSQL, Prisma, one container on port 3000). Do not run it on your app until you have read it and changed what does not fit. Always run the dry run (5.2) after every change.

<table>
<tr><th>What to check</th><th>Where in <code>deploy/deploy.sh</code></th><th>What to do</th></tr>
<tr><td rowspan="2">Names</td><td>The variables at the top: <code>REPO_NAME</code>, <code>SQL_INSTANCE</code>, <code>DB_NAME</code>, <code>DB_USER</code>, <code>SERVICE_NAME</code>, <code>MIGRATE_JOB_NAME</code>, <code>SA_NAME</code>, and the secret names</td><td>Rename them to your app. Use the same names in <code>deploy/release.sh</code> and <code>.github/workflows/deploy.yml</code>, or Episodes 6 to 8 will not find them.</td></tr>
<tr><td colspan="2">

```
REPO_NAME="toktickit"
SQL_INSTANCE="toktickit-pg"
DB_NAME="toktickit"
DB_USER="toktickit"
SERVICE_NAME="toktickit"
MIGRATE_JOB_NAME="toktickit-migrate"
SA_NAME="toktickit-runner"
SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"
SQL_CONN_NAME="${PROJECT_ID}:${REGION}:${SQL_INSTANCE}"
IMAGE_URI="${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO_NAME}/${SERVICE_NAME}:latest"

SECRET_DB_URL="toktickit-database-url"
SECRET_ADMIN_PASS="toktickit-admin-password"
SECRET_DEMO_PASS="toktickit-demo-password"

```

</td></tr>
<tr><td rowspan="2">Database engine</td><td>Step 3, <code>gcloud sql instances create ... --database-version=POSTGRES_16</code></td><td>PostgreSQL: keep. MySQL: use a <code>MYSQL_</code> version. SQL Server: a <code>SQLSERVER_</code> version (check the cost, it is higher). Another database (MongoDB and others): Cloud SQL does not offer it, so choose a managed option with your lecturer first</td></tr>
<tr><td colspan="2">

```
# 3. Create Cloud SQL for PostgreSQL instance (idempotent, db-f1-micro requires --edition=enterprise on POSTGRES_16)
if [ "$DRY_RUN" = "1" ]; then
  run_cmd gcloud sql instances describe "$SQL_INSTANCE" --project "$PROJECT_ID"
  run_cmd gcloud sql instances create "$SQL_INSTANCE" --database-version=POSTGRES_16 --edition=enterprise --tier=db-f1-micro --r
else
  if ! gcloud sql instances describe "$SQL_INSTANCE" --project "$PROJECT_ID" >/dev/null 2>&1; then
    run_cmd gcloud sql instances create "$SQL_INSTANCE" \
      --database-version=POSTGRES_16 \
      --edition=enterprise \
      --tier=db-f1-micro \
      --region="$REGION" \
      --storage-size=10GB \
      --storage-type=SSD \
      --backup-start-time=03:00 \
      --project "$PROJECT_ID"
  fi
fi
```

</td></tr>
</table>

What to check	Where in `deploy/deploy.sh`	What to do
Database URL	Step 6, `DB_URL_VALUE=`	The format belongs to your driver. Change it to the connection string your app expects. Cloud Run connects to Cloud SQL through a Unix socket at `/cloudsql/PROJECT:REGION:INSTANCE`. Check your language's guide in "Connect Cloud Run to Cloud SQL"
		
Secrets your app reads	Steps 6 to 8 and `JOB_SECRETS`	Keep one secret for the database URL. Replace the admin and demo password secrets with whatever your app needs at first start. Every secret must be generated, never typed into the script
		
Migration and seed command	Step 10, `--args "-c,npx prisma migrate deploy && npm run prisma:seed"`	Replace with your own migration and seed commands (for example `alembic upgrade head`, `php artisan migrate --force`, `dotnet ef database update`). The command must be safe to run twice

<table>
<tr><th>What to check</th><th>Where in <code>deploy/deploy.sh</code></th><th>What to do</th></tr>
<tr><td></td><td colspan="2">

```
222  # 10. Create/Update & Execute Cloud Run Job for Migration + Non-destructive Seed
223  run_cmd gcloud run jobs deploy "$MIGRATE_JOB_NAME" \
224    --image "$IMAGE_URI" \
225    --region "$REGION" \
226    --service-account "$SA_EMAIL" \
227    --set-cloudsql-instances "$SQL_CONN_NAME" \
228    --set-secrets "$JOB_SECRETS" \
229    --set-env-vars "NODE_ENV=production,TZ=Asia/Bangkok,SEED_DEMO_DATA=${SEED_DEMO_D
230    --command "sh" \
231    --args "-c,npx prisma migrate deploy && npm run prisma:seed" \
232    --max-retries 1 \
233    --task-timeout 300s \
234    --project "$PROJECT_ID"
```

</td></tr>
<tr><td rowspan="2">Port and settings</td><td>Step 11, <code>--port 3000</code>, <code>--set-env-vars</code>, <code>--memory</code>, <code>--min-instances</code>, <code>--max-instances</code></td><td>Set the port your container listens on. Keep environment variables that are not secret here and secrets in <code>--set-secrets</code>. Keep minimum instances at 0 to save money</td></tr>
<tr><td colspan="2">

```
246  # 11. Deploy Cloud Run Service
247  run_cmd gcloud run deploy "$SERVICE_NAME" \
248    --image "$IMAGE_URI" \
249    --region "$REGION" \
250    --platform managed \
251    --allow-unauthenticated \
252    --service-account "$SA_EMAIL" \
253    --add-cloudsql-instances "$SQL_CONN_NAME" \
254    --set-secrets "DATABASE_URL=${SECRET_DB_URL}:latest" \
255    --set-env-vars "NODE_ENV=production,TZ=Asia/Bangkok" \
256    --min-instances 0 \
257    --max-instances 3 \
258    --memory 512Mi \
259    --cpu 1 \
260    --port 3000 \
261    --project "$PROJECT_ID"
```

</td></tr>
<tr><td rowspan="2">Public access</td><td>Step 11, <code>--allow-unauthenticated</code></td><td>Keep it only if your app must be reachable by anyone (needed for the demo). If your organization blocks it, see the table at the end</td></tr>
<tr><td colspan="2">

```
251    --allow-unauthenticated \
```

</td></tr>
<tr><td>What “healthy” means</td><td><code>deploy/smoke_test.py</code> and the commands in 5.5</td><td>Change the paths and the expected answers: your own health endpoint, and a page or API that shows the database works. Keep it read-only</td></tr>
</table>

What to check	Where in `deploy/deploy.sh`	What to do
Instance name in later steps	5.4 and 5.7 in this guide	`toktickit-pg` and `toktickit` appear in the commands; use your own names

Keep as they are, unless you know why: the dedicated runtime service account with only two roles (Cloud SQL Client on the project and Secret Manager Secret Accessor on each secret), generating passwords instead of typing them, the migration as a separate Cloud Run job that runs before the service is updated, storage auto-increase off, and the “skip if it already exists” checks that make the script safe to run again.

If you are unsure, ask before you run it: the first real run creates a Cloud SQL instance, which costs money from the first hour.

5.1 Prepare Cloud Shell

Before you begin: You are signed in to the Google account that owns the project.

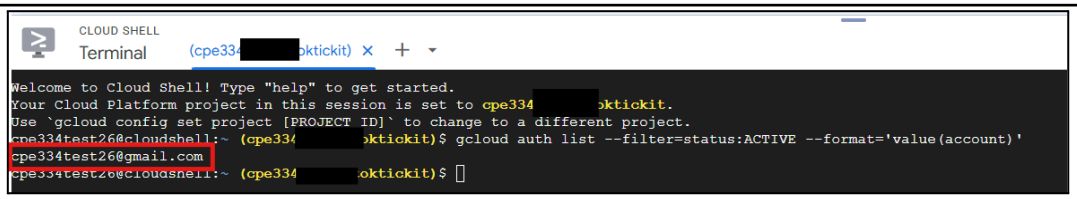
Steps

1. Open the Google Cloud Console, select your project, and open **Cloud Shell** with the terminal button in the top bar.



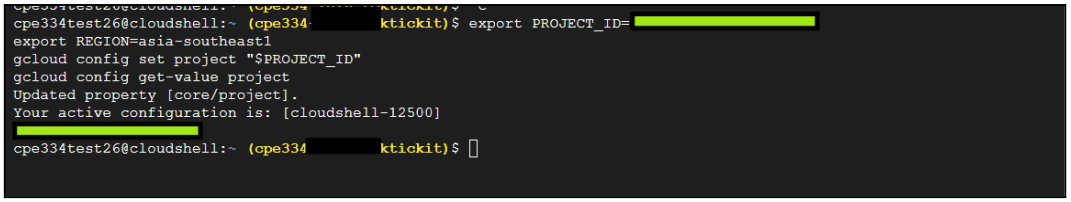
2. Check that the CLI uses the same account as the Console:

```
gcloud auth list --filter=status:ACTIVE --format='value(account)'
```



3. Set your project and region (replace the values with your own):

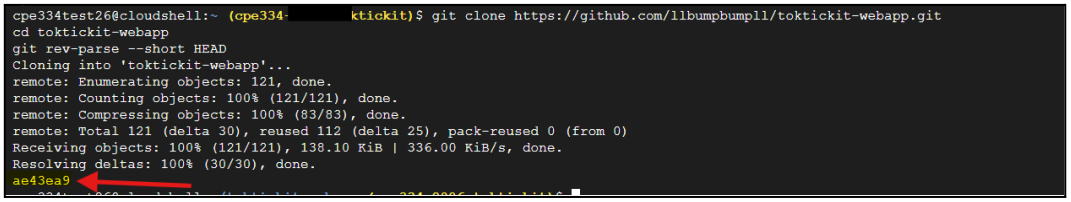
```
export PROJECT_ID=YOUR_PROJECT_ID
export REGION=asia-southeast1
gcloud config set project "$PROJECT_ID"
gcloud config get-value project
```



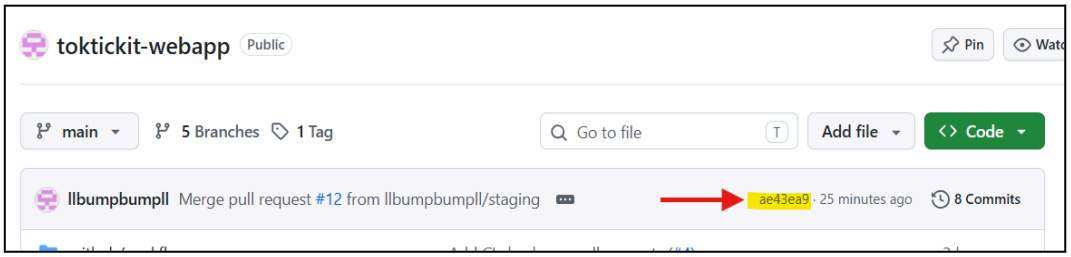
4. Download your code and check the commit:

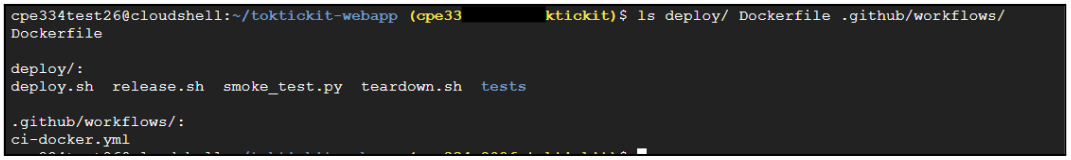
```
git clone https://github.com/YOUR_ORG/YOUR_REPO.git
cd YOUR_REPO
git rev-parse --short HEAD
```

This copies your code from GitHub into Cloud Shell, which is a separate computer in Google Cloud. Your computer is not changed.



**The commit must match the latest commit on `main` in GitHub. If you cloned before, use the existing folder and run `git pull` instead of cloning again.





5.2 Read the plan before creating anything

Before you begin: You are in the repository folder in Cloud Shell.

Steps

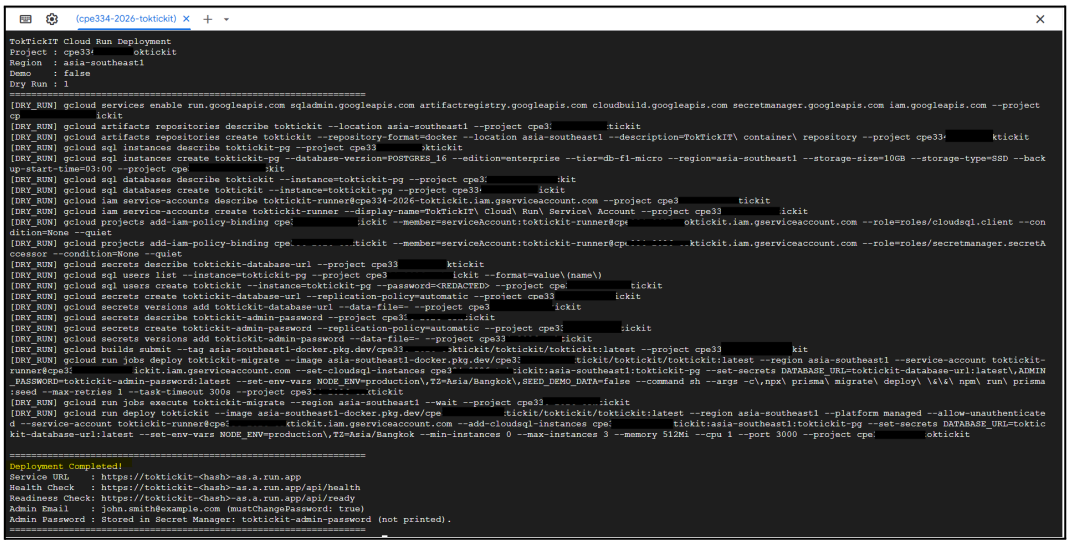
1. Run the script in dry-run mode. It prints every command and creates nothing:

```
DRY_RUN=1 PROJECT_ID="$PROJECT_ID" REGION="$REGION" bash deploy/deploy.sh
```

2. **Read** the output and find these steps in order:

1. Enable the Google APIs;
2. Create the Artifact Registry repository (where images are stored);
3. Create the Cloud SQL instance and the database;
4. Create the runtime service account and grant it `Cloud SQL Client`;
5. Create the database user, store the secrets in Secret Manager, and grant toktickit-runner Secret Manager Secret Accessor on each secret;
6. Build the image with Cloud Build;
7. Create and run the migration job on Cloud Run;
8. Deploy the Cloud Run service.

3. The line “Deployment Completed!” at the end of a dry run is only a simulation.



5.3 Deploy for real

Before you begin: The dry run looked right, and billing is active.

Steps

1. Run the script:

```
PROJECT_ID="$PROJECT_ID" REGION="$REGION" bash deploy/deploy.sh
```

2. Wait. It takes about 10 to 20 minutes, mostly for Cloud SQL and the image build. **Do not cancel it.**



3. If it fails, read the last lines of the log before you do anything. If it stops at the migration job, do not run the script again straight away; read the job log first (the script prints the command to see it).
4. At the end the script prints the service URL. Copy it.

The script is safe to run again after a failure: it skips resources that already exist. Passwords are generated for you and **stored in Secret Manager**; they are not printed.

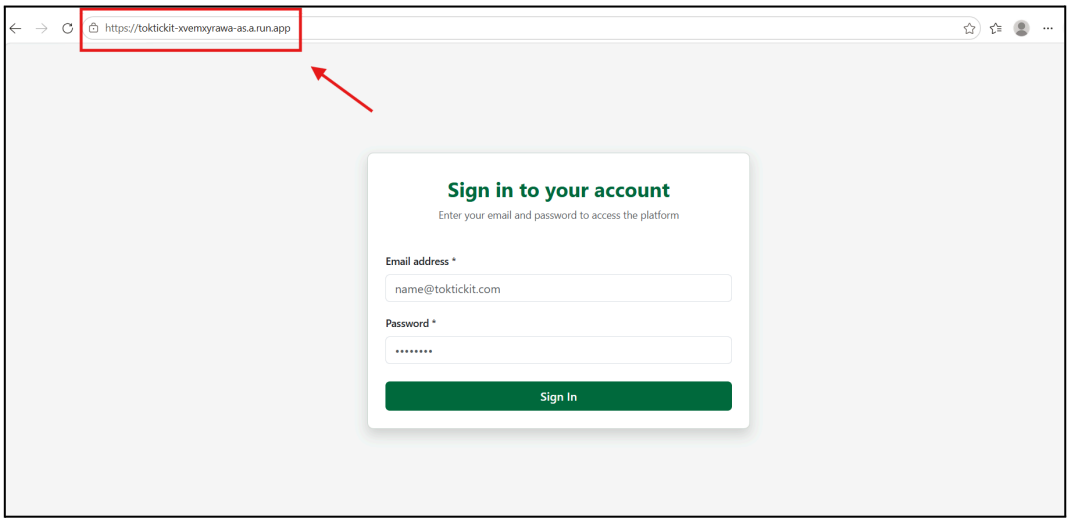
Demo accounts: the script can also create sample users with the option `--demo`. Demo accounts are allowed in your project (you will need them for UAT in Lab 6), but their passwords must come from Secret Manager, never from code. Use `--demo` only if your team wants them now.

```
Execution [toktickit-migrate-zsrrm] has successfully completed.

View details about this execution by running:
gcloud run jobs executions describe toktickit-migrate-zsrrm

Or visit https://console.cloud.google.com/run/jobs/executions/details/asia-southeast1/toktickit-migrate-z
+ gcloud run deploy toktickit --image asia-southeast1-docker.pg.dev/cpe33          ickit/toktickit/tokt
ice-account toktickit-runner@cpe          ickit.iam.gserviceaccount.com --add-cloudsql-instances cpe33
base-url:latest --set-env-vars NODE_ENV=production\,TZ=Asia/Bangkok --min-instances 0 --max-instances 3 --
Deploying container to Cloud Run service [toktickit] in project [cpe334          tickit] region [asia-soutl
OK Deploying new service... Done.
  OK Creating Revision...
  OK Routing traffic...
  OK Setting IAM Policy...
Done.
Service [toktickit] revision [toktickit-00001-f27] has been deployed and is serving 100 percent of traffi
Service URL: https://toktickit-906251530807.asia-southeast1.run.app

================================================================
Deployment Completed!
Service URL     : https://toktickit-xvemxyrawa-as.a.run.app
Health Check    : https://toktickit-xvemxyrawa-as.a.run.app/api/health
Readiness Check: https://toktickit-xvemxyrawa-as.a.run.app/api/ready
Admin Email     : john.smith@example.com (mustChangePassword: true)
Admin Password : Stored in Secret Manager: toktickit-admin-password (not printed).
================================================================
cpe334test26@cloudshell:~/toktickit-webapp (cpe33          ckit)$
```



5.4 Turn off storage auto-increase right away

Before you begin: Step 5.3 finished.

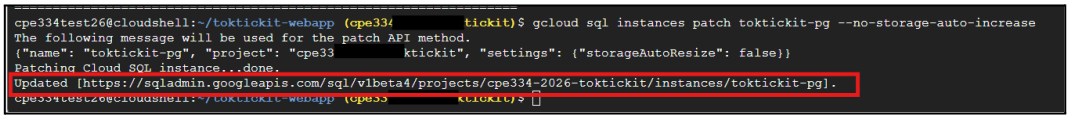
Steps

1. Run:

```
gcloud sql instances patch toktickit-pg --no-storage-auto-increase
```

** Answer y if it asks you to confirm.

Why: auto-increase is on by default, and a disk that has grown cannot shrink back. A fixed 10 GB disk keeps the cost predictable.



5.5 Test the running app.

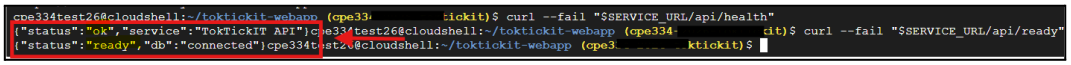
Before you begin: You have the service URL.

Steps

1. Store the URL and call the health endpoints:

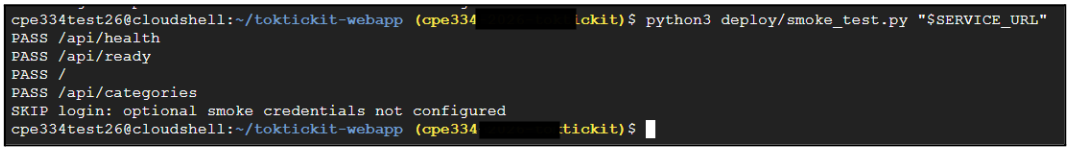
```
export SERVICE_URL=$(gcloud run services describe toktickit --region "$REGION"
--format='value(status.url)')
echo "$SERVICE_URL"
curl --fail "$SERVICE_URL/api/health"
curl --fail "$SERVICE_URL/api/ready"
```

`/api/ready` answers only when the app can reach the database.



2. Run the smoke test from the kit:

```
python3 deploy/smoke_test.py "$SERVICE_URL"
```



3. Open the URL in your browser. The login page must load from the public internet, without localhost and without a Google sign-in.

5.6 Get the first administrator password (privately)

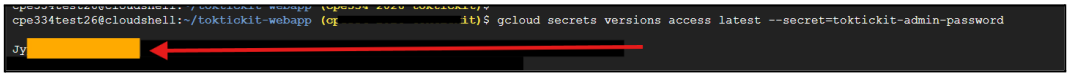
Before you begin: do not take a screenshot for this step.

Steps

1. Read the generated password:

```
gcloud secrets versions access latest --secret=toktickit-admin-password
```

2. Sign in to the app as the administrator (the script prints the email at the end). The app makes you choose a new password at the first sign-in. Do it immediately.
3. Never paste the password into a chat, a document, or an Issue.



5.7 Check in the Console what the script created.

Before you begin: The app works. Keep the Console open next to Cloud Shell.

Steps

1. Open each menu and compare with the table. In the Secret Manager page, **never click "View secret value."**

What to check	Menu	You should see
Services	APIs & Services → Enabled APIs & services	Cloud Run, Cloud SQL Admin, Artifact Registry, Cloud Build, Secret Manager, IAM
Permissions	IAM & Admin → IAM	`toktickit-runner` has only **Cloud SQL Client** (project-level)
Database	Cloud SQL → `toktickit-pg`	PostgreSQL 16, database `toktickit`, storage auto-increase off
Secrets	Security → Secret Manager	The secret names only (and under each secret's **Permissions** tab / Info Panel, `toktickit-runner` has **Secret Manager Secret Accessor**)
App settings	Cloud Run → `toktickit` → Revisions → Containers	`NODE_ENV` and `TZ` as plain values; `DATABASE_URL` from Secret Manager

What to check	Menu	You should see
Scaling	Cloud Run → `toktickit`	Minimum 0 instances, maximum 3
Image	Artifact Registry → `toktickit`	The app image

5.8 Close the Issue

Steps

1. In GitHub, comment on Issue #5 with the service URL and the screenshots from 5.5 and 5.7 (no secret values, no passwords).
2. Move Issue #5 to *Done*.

Done when

- The Cloud Run URL loads from the internet, and you can sign in.
- `/api/ready` answers, which shows the app reaches the database.
- The Cloud SQL instance, the database, the secrets, the runtime service account and the image all exist, and storage auto-increase is off.
- No password or secret is in your screenshots, your Issue or your chat.

Note: What "done" looks like depends on your project and the stack your team chose. The list above uses TokTickIT as the example. For your own app, "done" means your own service is reachable on a public URL, it reads its settings and secrets from the cloud and not from code, and its data is stored in a managed database and not in the container.

If something fails

What you see	What to check
Billing or "free trial" error when creating Cloud SQL	The trial must be active and billing enabled on this project (Console → Billing)
"Permission denied" or "API not enabled" right after enabling APIs	Wait one or two minutes and run the script again; permissions can take a moment to apply
`gcloud builds submit` fails	Read the build log link it prints; the same build already passed in CI, so check the project and region first
Script stops at the migration job	Read the job log with the command the script prints. Fix the cause, then run the script again

What you see	What to check
The public URL answers “Forbidden”	Your organization may block public access to Cloud Run. Use the project of your own personal account, not a school or company account
`/api/ready` fails but `/api/health` works	The app cannot reach the database. Check the `DATABASE_URL` secret and the Cloud SQL connection on the service
The first request after some idle time is slow	Minimum instances is 0, so Cloud Run starts a container on demand. This is expected

Official references

- Deploy a container to Cloud Run – what the last script step does
- Connect Cloud Run to Cloud SQL – how the service reaches the database
- Use secrets in Cloud Run – why the database URL and passwords are not in code
- Cloud Run container runtime contract – `PORT`, environment variables, no data on the container’s disk

Helpful VS Code extensions

Install them on your own computer (**Extensions** panel, `Ctrl+Shift+X`, search by name, and check the publisher).

Extension	Publisher	Helps with
GitHub Actions	GitHub	Editing workflow files such as `deploy.yml`, and seeing workflow runs in VS Code
YAML	Red Hat	Syntax checking for `deploy.yml` and `docker-compose.yml`
Docker (Container Tools)	Microsoft	`Dockerfile` and Compose files, and your images and containers
Google Cloud Code	Google Cloud	Cloud Run and other Google Cloud tools inside the editor (optional)

references

- GitHub Actions for VS Code
- YAML Language Support by Red Hat
- Docker Extension Pack for Visual Studio Code
- Google Cloud Code

Guide/Episode 6: Connect GitHub to GCP and set the merge gate

Videos: LEB2 / Learn It / Student Guide: GCP Deployment and CI/CD (Video) / Episode 6

Prerequisite: Episodes 4 and 5

Issue #6: (no branch for 6.1 to 6.4; a throwaway branch for 6.5)

Criterion: CI and quality gate; also prepares the repeatable release pipeline

---

Goal:

1. Let GitHub Actions deploy to your Google Cloud project without any password or key file, using Workload Identity Federation, and only from your repository, your main branch and your deploy workflow.
2. Make a failing check block merging into main, so a broken change can never be released.

No keys. Never create or download a JSON key for a service account, and never put one in GitHub. The whole point of this episode is to avoid keys.

Adapt to your project. This episode is about your GitHub repository and your Google Cloud project, so most of it is the same for every stack. Replace the names with your own: the repository, the project, the service account name, the environment name `demo`, the workflow file `deploy.yml` and the check name `test-and-docker`. Every name must match across GitHub, Google Cloud and `deploy.yml`.

Before you begin

- The first deploy from Episode 5 worked, and `toktickit-runner`, the Artifact Registry repository and the Cloud Run service exist.
- `ci-docker.yml` and `deploy/` are on `main` (Episode 4), and `.github/workflows/deploy.yml` is on `main` or ready to be added in Episode 7.
- You have admin rights on the GitHub repository (Settings is visible).
- You use Cloud Shell, signed in to the account that owns the project, for 6.2 and 6.3.
- Keep your own numbers: **never copy another team's repository ID, owner ID or project number.**

6.1 Find your repository IDs

Before you begin: You know your GitHub user or organization name and your repository name, exactly as they appear in the address of your repository.

Google Cloud will accept deploys only from your repository. It identifies it by two numbers, not by name, because a name can be taken over by someone else after a repository is deleted. You need both numbers for 6.2 and 6.3. They are not secret, but they are different for every repository, so **never copy another team's numbers**.

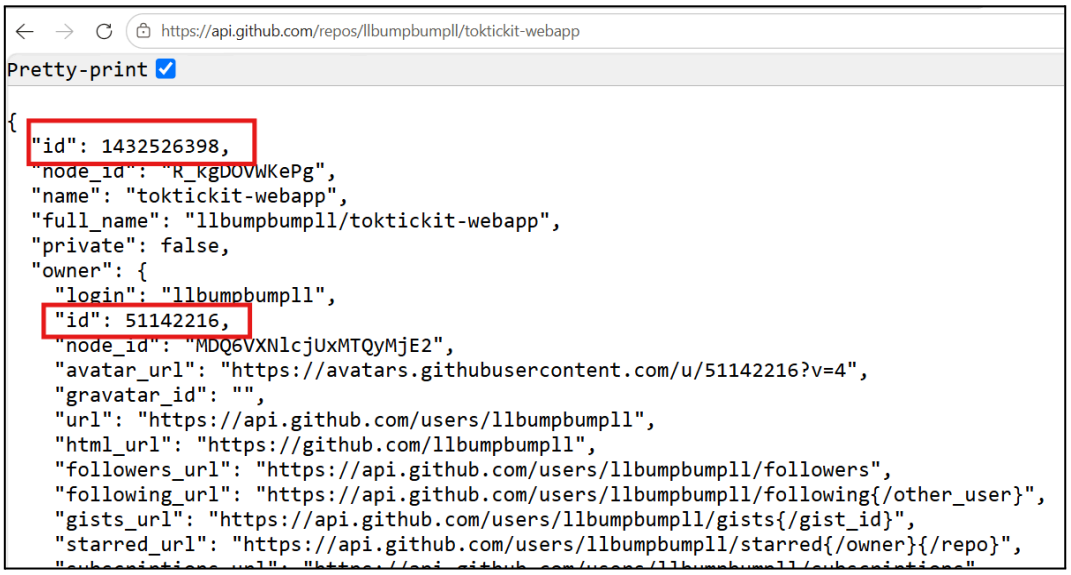
Steps

1. Open this address in your browser. Replace `YOUR_GITHUB` and `YOUR_REPO`:

```
https://api.github.com/repos/YOUR_GITHUB/YOUR_REPO
```

You see a page of text. Press **Ctrl+F** and find these two:

- the first `"id":` near the top of the page is your **`REPO_ID`**;
- the `"id":` inside the block that starts with `"owner": {` is your **`OWNER_ID`**.



Write both numbers down.

2. **If you see "Not Found"** your repository is private. Use one of these instead:
   - Open your repository page on GitHub (signed in), press **Ctrl+U** to view the page source, and press **Ctrl+F** and search for `octolytics-dimension`. Read `repository_id` (this is your `REPO_ID`) and `user_id` (this is your `OWNER_ID` when the repository belongs to your personal account). For an organization repository, open `https://api.github.com/orgs/YOUR_ORG` while signed in to read the organization `id`.

```
<meta name="octolytics-dimension-repository_id" content="1432526398"/><
```

```
<meta name="octolytics-dimension-user_id" content="51142216"/><
```

   - Or, if you have the GitHub CLI installed and signed in (`gh auth login`), run:

```
gh api repos/YOUR_GITHUB/YOUR_REPO --jq '.id, .owner.id'
```

The first number is `REPO_ID`, the second is `OWNER_ID`.

3. **Check the token format (optional).** GitHub gives each repository a token format, called the OIDC subject, either with these IDs in it (immutable) or without. If you have `gh`, run:

```
gh api repos/YOUR_GITHUB/YOUR_REPO/actions/oidc/customization/sub
```

Note whether `use_immutable_subject` is `true` or `false`. Without `gh`, skip this:

Use the format with IDs in 6.3, and only if the workflow in Episode 7 fails with “rejected by the attribute condition," switch to the format without IDs, as described in the table at the end of this episode.



6.2 Create the deployer service account (Console).

Before you begin: You have your Project ID, and you know the name of the runtime service account from Episode 5 (`toktickit-runner`).

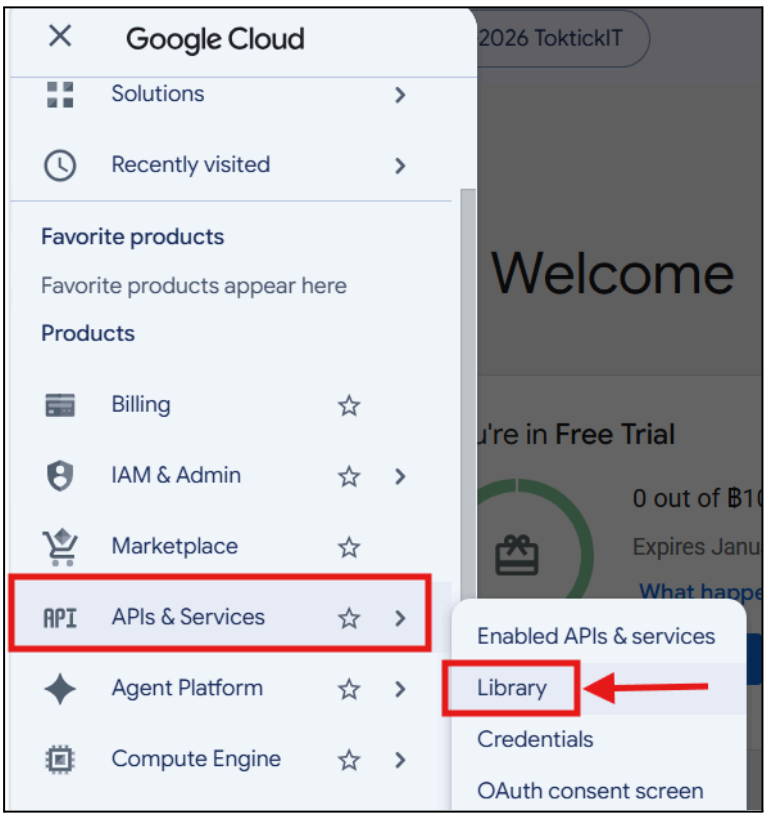
Names used in this step. These depend on your project. Find your own values at the top of your `deploy/deploy.sh`, and use the same names everywhere in Episodes 6 to 8.

What it is	Example (TokTickIT)	Your project
Deployer service account: the identity GitHub Actions uses to deploy. You create it in this step, so you choose the name	`toktickit-deployer`	`YOUR_APP-deployer`
Runtime service account: the identity your running app uses. It already exists from Episode 5 (`SA_NAME` in `deploy.sh`)	`toktickit-runner`	the value of `SA_NAME` in your `deploy.sh`
Artifact Registry repository: where your images are stored. It exists from Episode 5 (`REPO_NAME` in `deploy.sh`)	`toktickit`	the value of `REPO_NAME` in your `deploy.sh`
Role for deploying: what the deployer may change. It depends on where your app runs	Cloud Run Developer (the app and its migration job run on Cloud Run)	Cloud Run Developer if you also use Cloud Run. If you deploy somewhere else, ask your lecturer which role fits

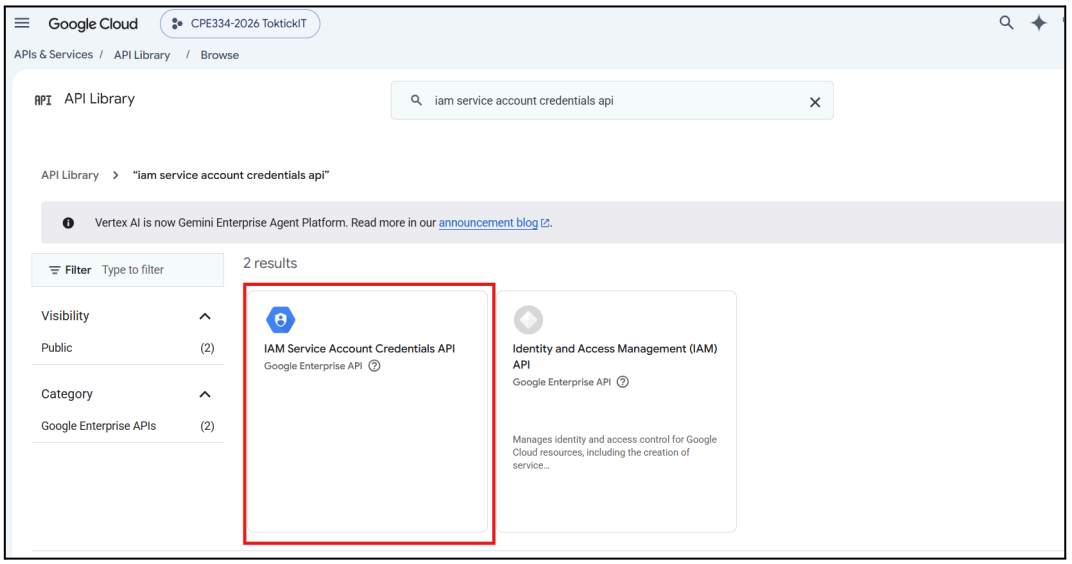
The deployer’s name must be the same in 6.3, in the GitHub variable `WIF_SERVICE_ACCOUNT` (6.4) and in `deploy.yml`.

Steps

1. Open the Google Cloud Console and select your project. Go to **APIs & Services** → **Library**,

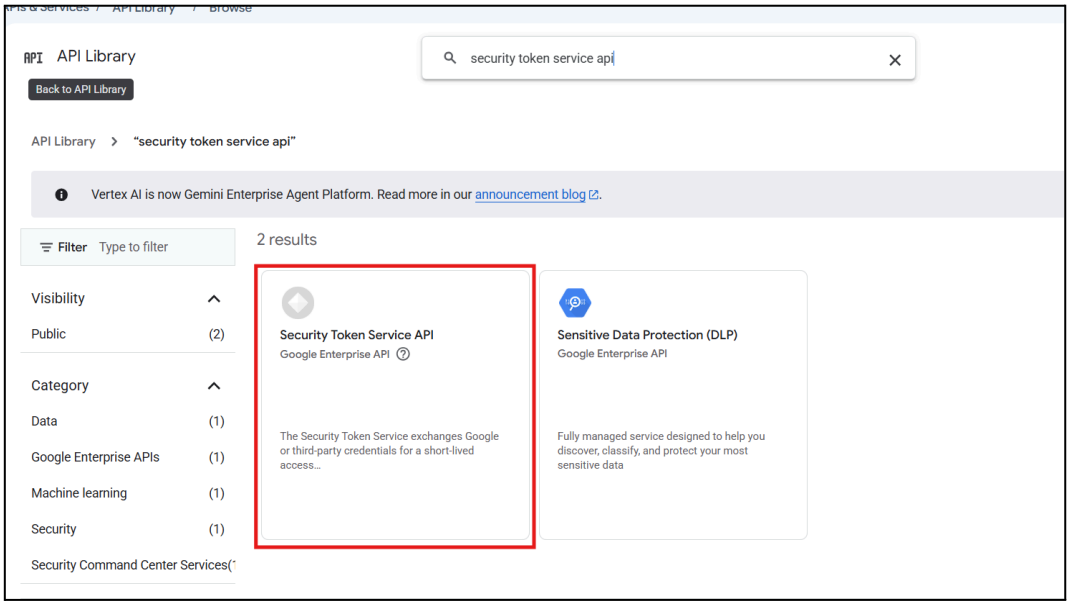


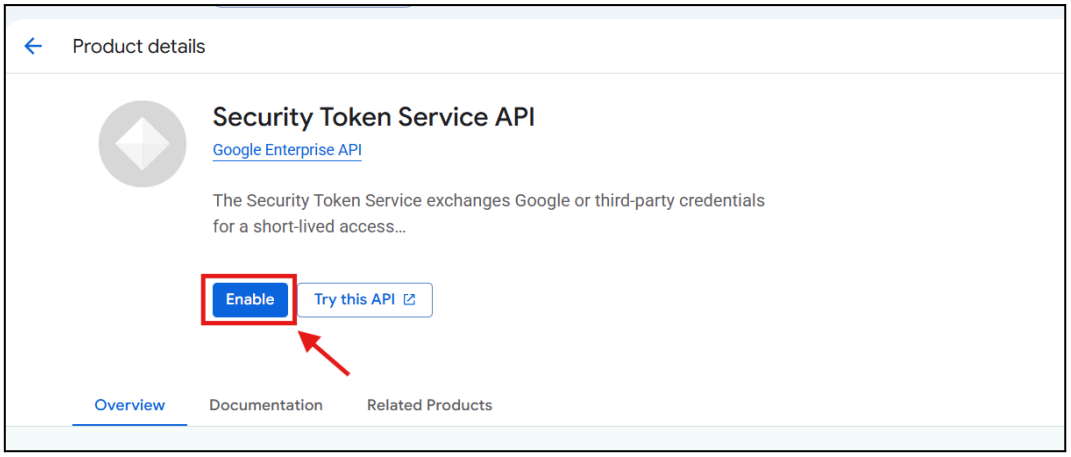
2. search for **IAM Service Account Credentials API** and click **Enable**.



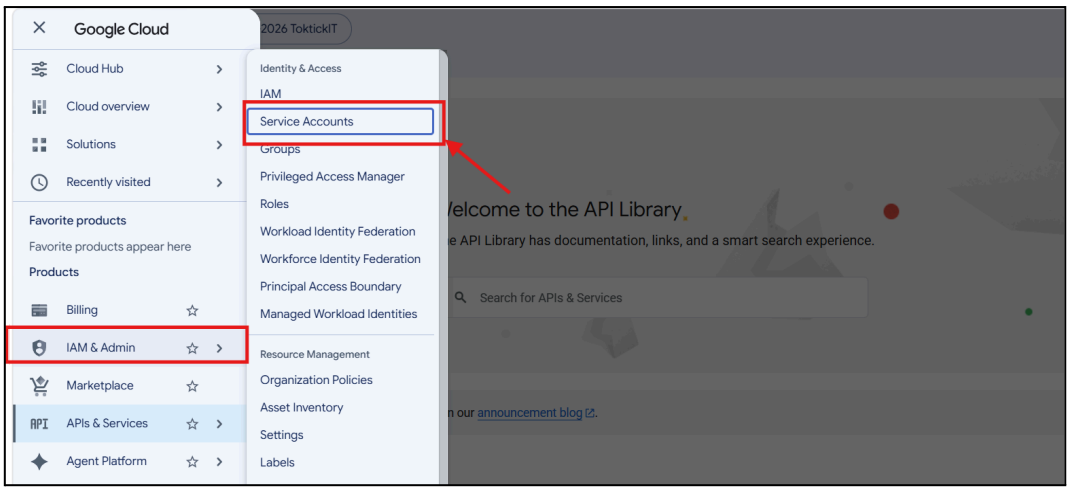
Note: If the page shows **Manage** (or **API Enabled**) instead of **Enable**, the API is already on for your project, and you do not need to do anything. Some APIs are turned on automatically when you use a service for the first time, and the deploy script in Episode 5 may have enabled them already. Move on to the next one.

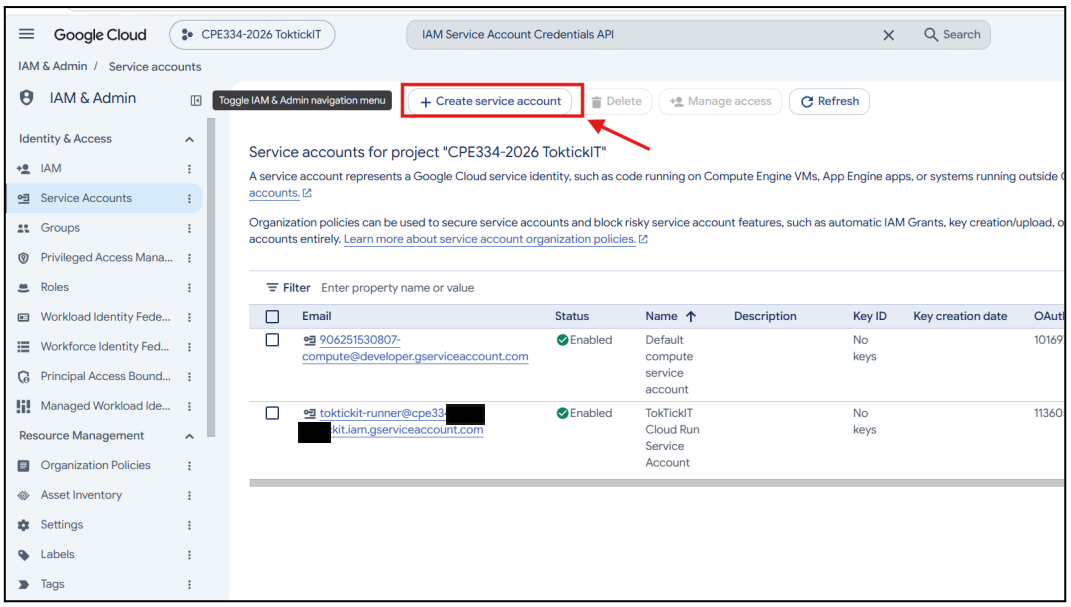
3. Do the same for **Security Token Service API**.



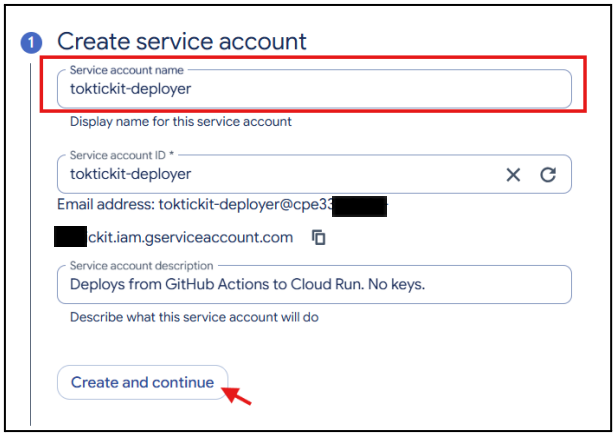


4. Go to **IAM & Admin** → **Service Accounts** → **Create service account**.

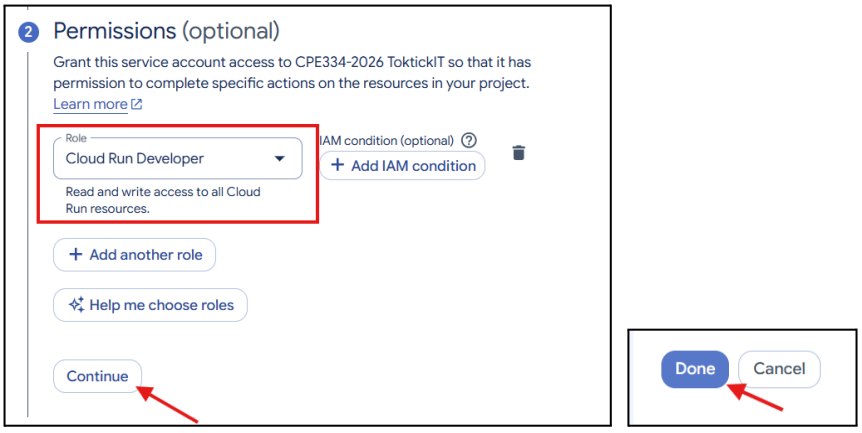


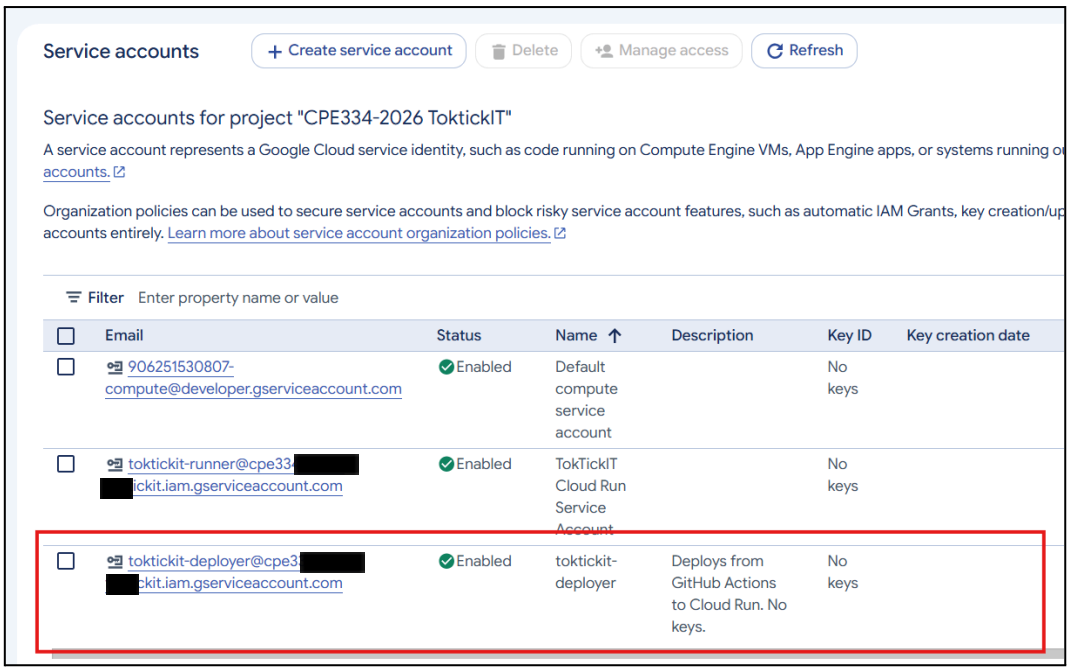


5. Enter the deployer name, for example `toktickit-deployer` (yours: `YOUR_APP-deployer`). Click **Create and continue**.

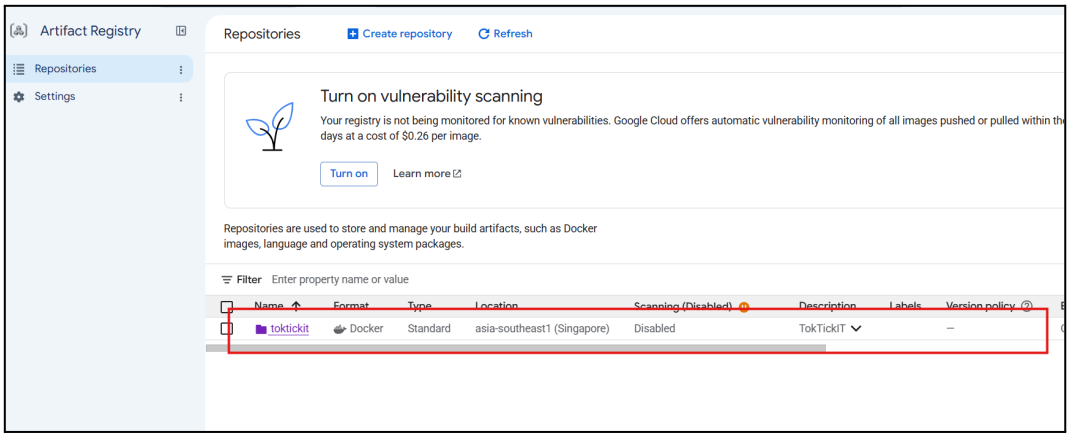


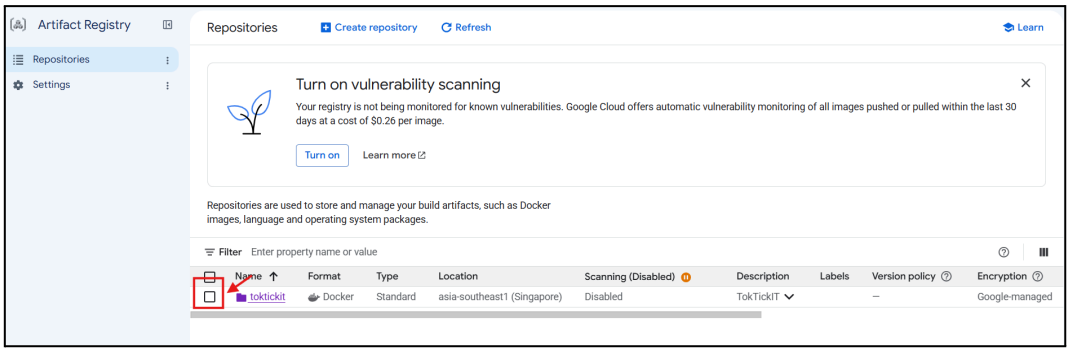
6. In step 2, **Permissions (optional)**, click **Select a role** and choose **Cloud Run Developer**. Click **Continue**. Leave step 3, **Principals with access (optional)**, empty and click **Done**.

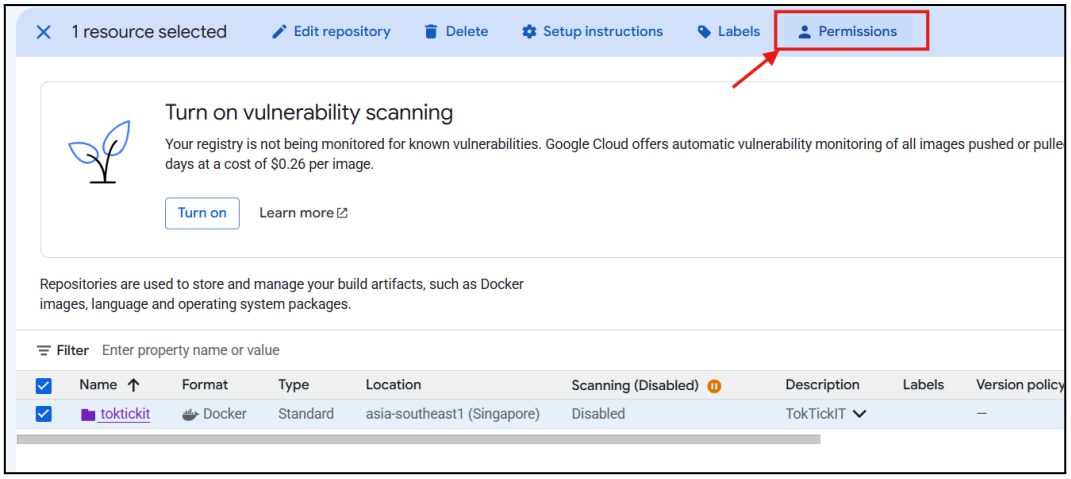


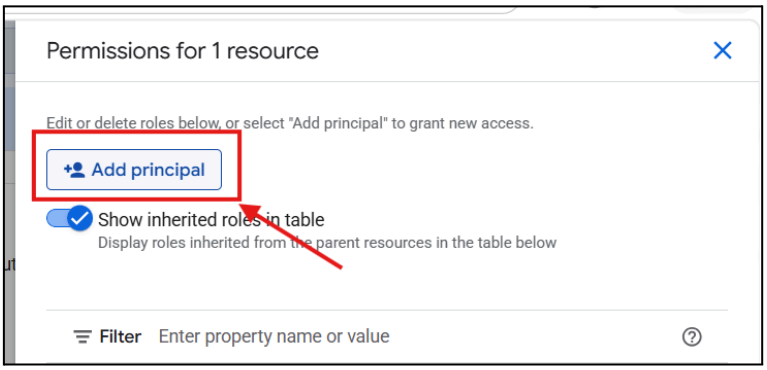


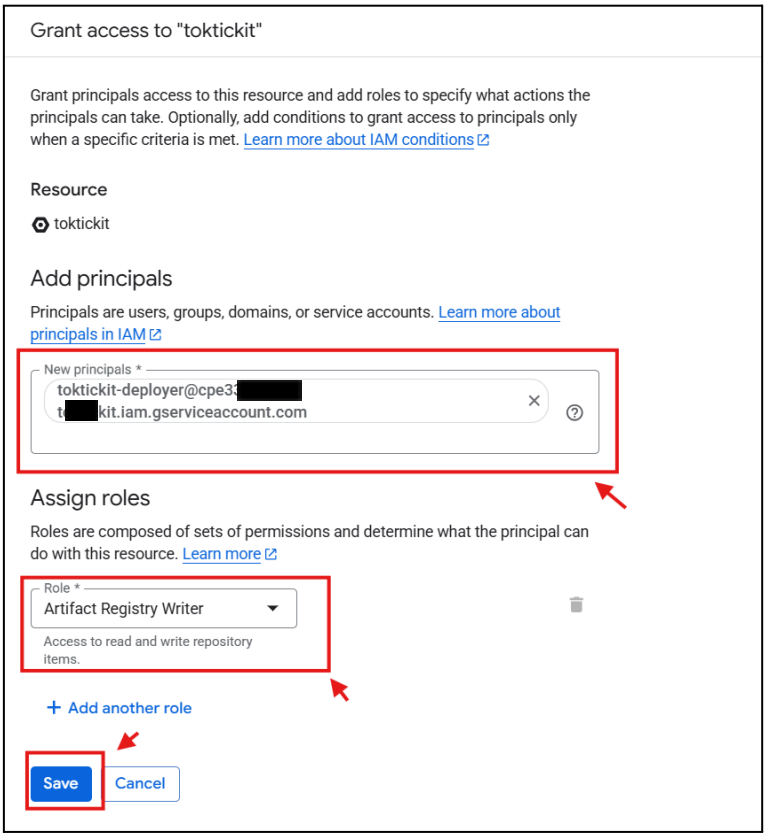
7. Give it permission to push images to **your repository only**: go to **Artifact Registry** → **Repositories**. Tick the row of your repository (for example `toktickit`). If the panel on the right is not shown, click **Show Info Panel** in the menu bar. On the **Permissions** tab, click **Add Principal**, enter the email of the deployer account, choose the role **Artifact Registry Writer**, and click **Save**. See Google's steps: [Access control with IAM – granting repository-specific rol]



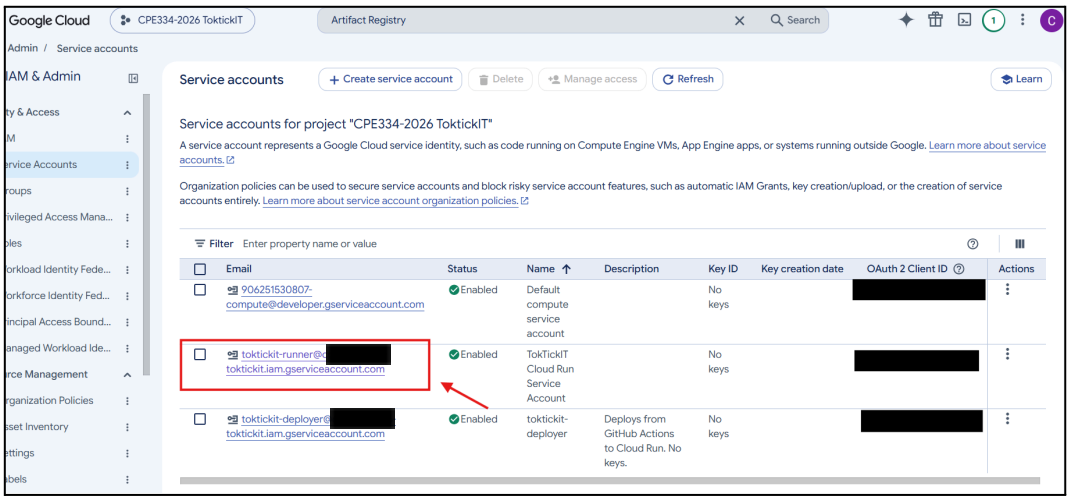


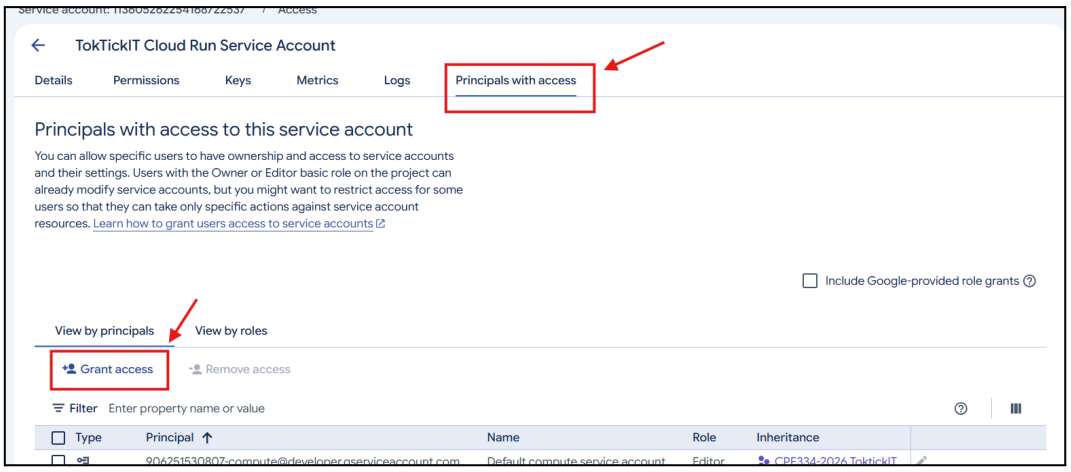

1 resource selected
Edit repository
Delete
Setup instructions
Labels
Permissions
Turn on vulnerability scanning
Your registry is not being monitored for known vulnerabilities. Google Cloud offers automatic vulnerability monitoring of all images pushed or pulle
days at a cost of $0.26 per image.
Turn on
Learn more
Repositories are used to store and manage your build artifacts, such as Docker
images, language and operating system packages.
Filter
Enter property name or value
Name
Format
Type
Location
Scanning (Disabled)
Description
Labels
Version policy
toktickit
Docker
Standard
asia-southeast1 (Singapore)
Disabled
TokTickIT


Permissions for 1 resource
Edit or delete roles below, or select "Add principal" to grant new access.
Add principal
Show inherited roles in table
Display roles inherited from the parent resources in the table below
Filter
Enter property name or value


Grant access to "toktickit"
Grant principals access to this resource and add roles to specify what actions the
principals can take. Optionally, add conditions to grant access to principals only
when a specific criteria is met. Learn more about IAM conditions
Resource
toktickit
Add principals
Principals are users, groups, domains, or service accounts. Learn more about
principals in IAM
New principals *
toktickit-deployer@cpe3
t kit.iam.gserviceaccount.com
Assign roles
Roles are composed of sets of permissions and determine what the principal can
do with this resource. Learn more
Role *
Artifact Registry Writer
Access to read and write repository
items.
Add another role
Save
Cancel

8. Let it run the app as the runtime account, on that account only: go to **IAM & Admin** → **Service Accounts**, click the runtime account (for example `toktickit-runner`), open the **Principals with access** tab, click **Grant access**, enter the email of the deployer account, choose the role **Service Account User**, and **Save**.





What each role allows:

- **Cloud Run Developer** updates the Cloud Run service and the migration job.
- **Artifact Registry Writer** on your repository only pushes the image.
- **Service Account User** on the runtime account lets the deployed app run as `toktickit-runner`. The deployer cannot read secrets, manage IAM or create databases.

6.3 Create Workload Identity Federation (Console)

Before you begin: You have `REPO_ID` and `OWNER_ID` from 6.1, and you know whether `use_immutable_subject` is true or false. Open a text editor (Notepad) for the next step.

Steps

1. **Prepare the condition in a text editor first.** Paste this text, then Replace `REPO_ID`, `OWNER_ID`, `YOUR_GITHUB`, `YOUR_REPO` and `ENV_NAME` with your own values. Use `demo` for `ENV_NAME` unless you have a reason to choose another name (see the note below). Do not leave spaces before or after the values, and keep every single quote.

```
assertion.repository_id == 'REPO_ID' && assertion.repository_owner_id == 'OWNER_ID' && assertion.ref == 'refs/heads/main' && assertion.workflow_ref == 'YOUR_GITHUB/YOUR_REPO/.github/workflows/deploy.yml@refs/heads/main' && assertion.sub == 'repo:YOUR_GITHUB@OWNER_ID/YOUR_REPO@REPO_ID:environment:ENV_NAME'
```

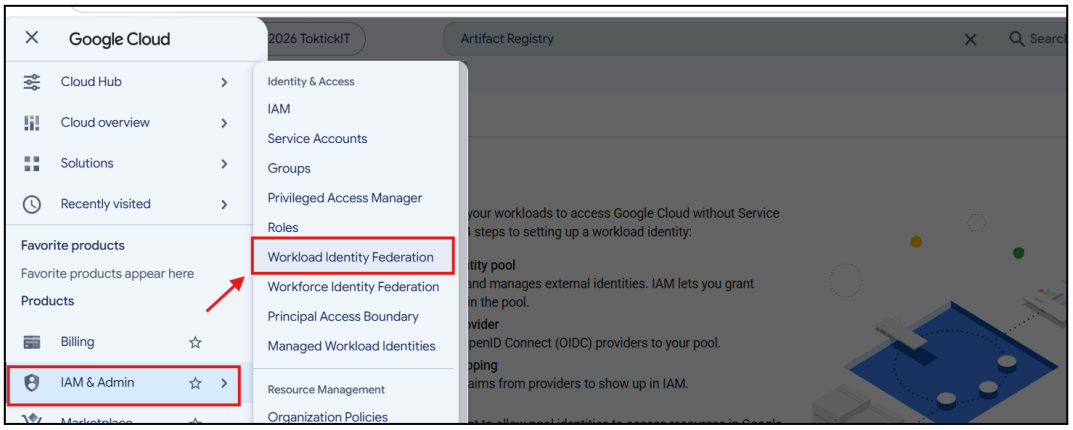
** If use_immutable_subject was **false**, the last part is assertion.sub == 'repo:YOUR_GITHUB/YOUR_REPO:environment:ENV_NAME'. Read it twice: it must show **your** values.

The environment name is part of this condition. The end of the last line, `:environment:ENV_NAME`, is the name of the GitHub environment you will create in step 6.4. The names must be exactly the same, including capital letters. If you choose another name in 6.4, change it here too, and change the `environment:` line in `.github/workflows/deploy.yml`. If one of the three differs, the deploy workflow cannot sign in to Google Cloud and fails with “rejected by the attribute condition”.

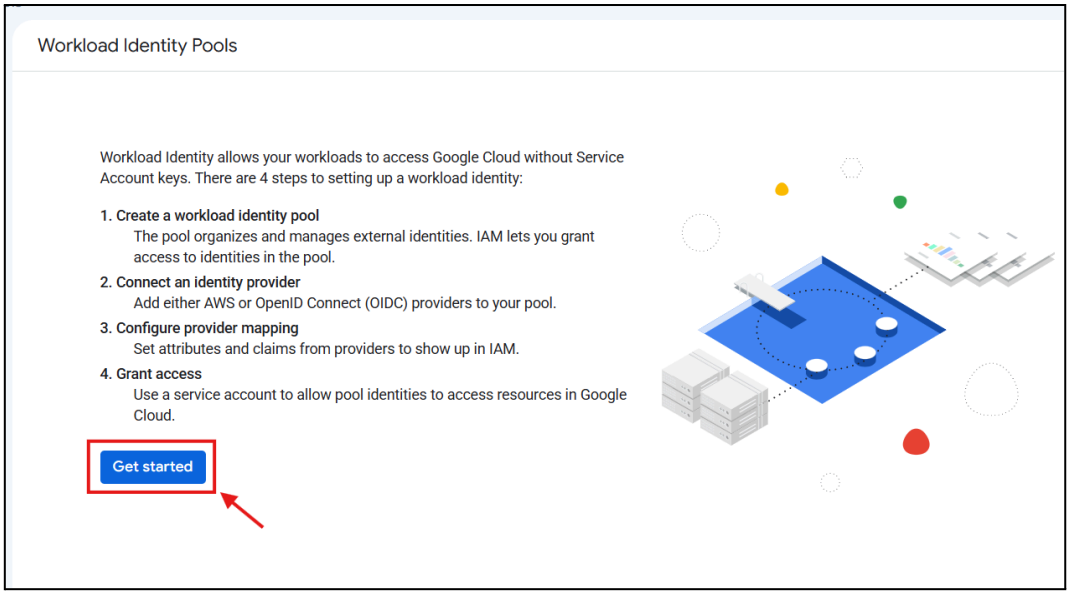
Example (TokTickIT): ENV_NAME is demo, so the line ends with environment:demo'.

```
assertion.repository_id == '1432526398' && assertion.repository_owner_id == '51142216' && assertion.ref == 'refs/heads/main' && assertion.workflow_ref == 'llbumpbumpll/toktickit-webapp/.github/workflows/deploy.yml@refs/heads/main' && assertion.sub == 'repo:llbumpbumpll@51142216/toktickit-webapp@1432526398:environment:demo'
```

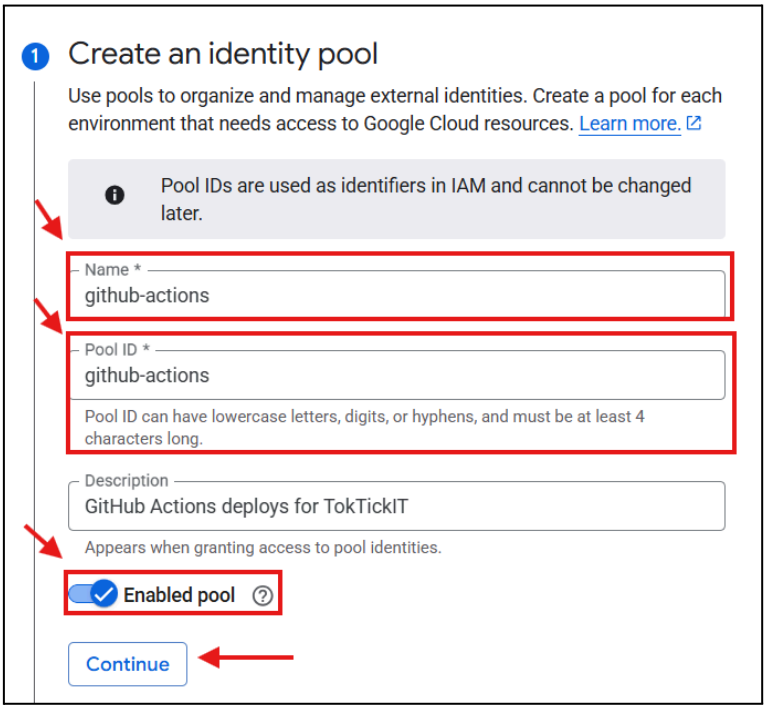
2. Go to **IAM & Admin → Workload Identity Federation**



- **If you have never created a pool in this project**, the page shows an introduction with four steps. Click **Get started**.
- **If your project already has a pool**, the page shows a list instead. Click **Create pool**.

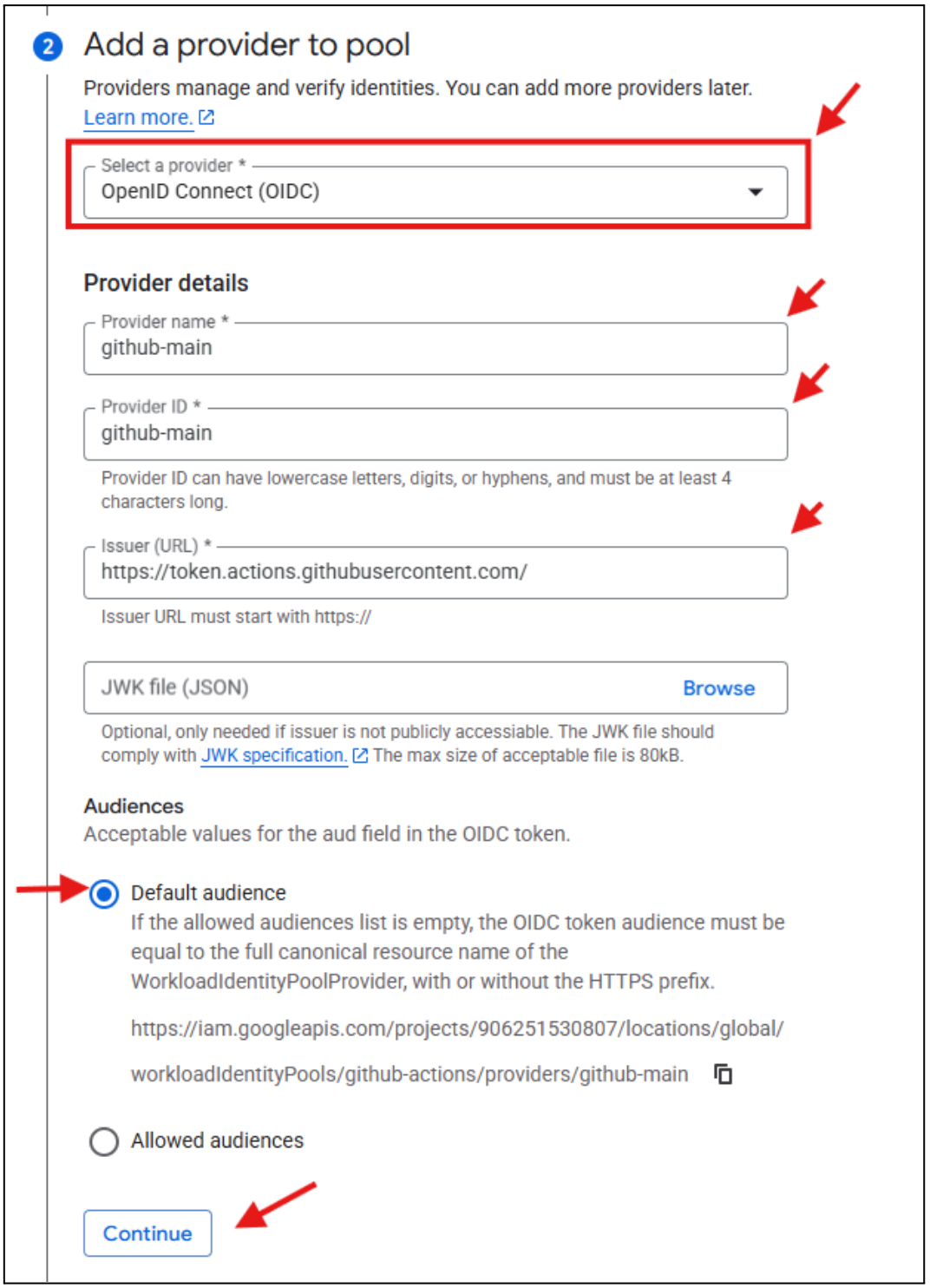


3. Under **Create an identity pool**, enter the name `github-actions`. Check that the **Pool ID** below it also says `github-actions` (click **Edit** to change it; the ID cannot be changed later). Leave **Enabled pool** on. Click **Continue**.

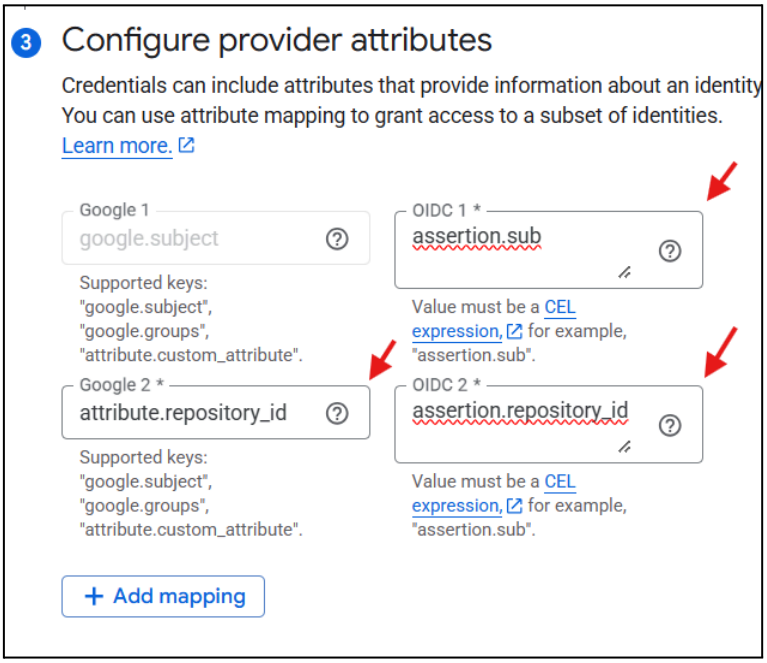


4. Under **Add a provider to pool**, check that **Select a provider** says **OpenID Connect (OIDC)**. Enter the **Provider name** `github-main`. Check that the **Provider ID** below it also says `github-main` (click **Edit** to change it; it cannot be changed later).

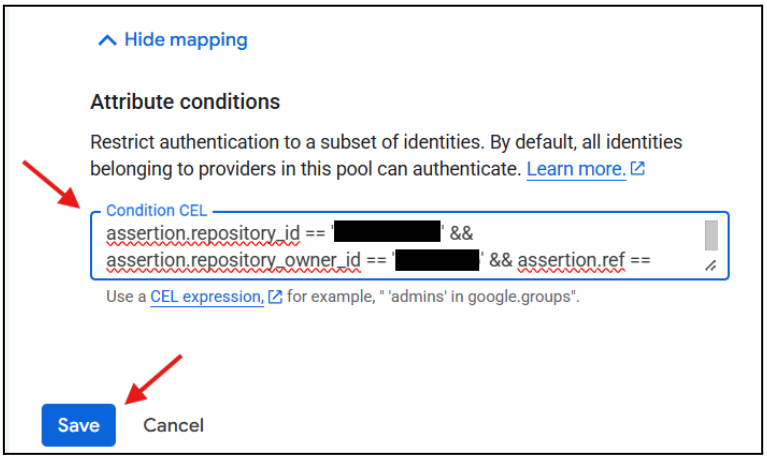
5. Set **Issuer (URL)** to `https://token.actions.githubusercontent.com/`. Leave **JWK file (JSON)** empty. Under **Audiences**, keep **Default audience**. Scroll down and click **Continue**.



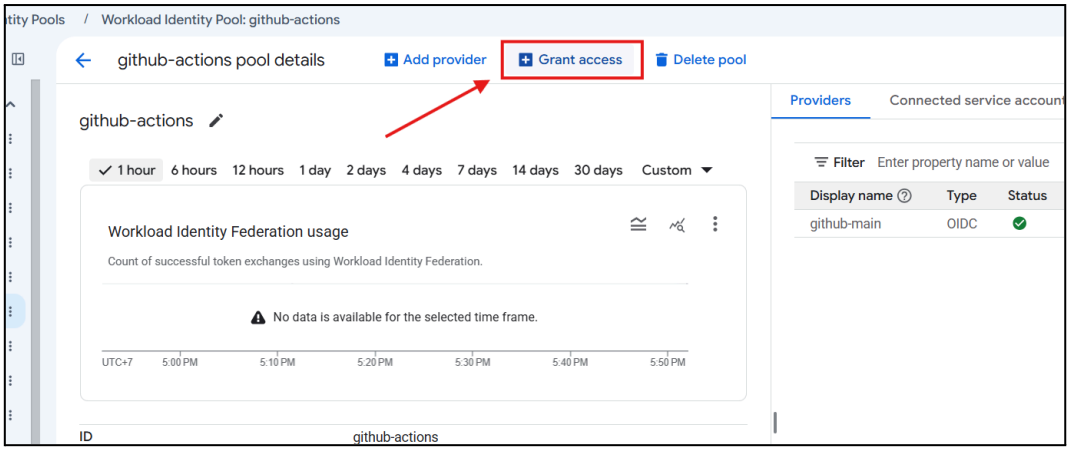
6. Under **Configure provider attributes**, the first row is `google.subject`. Enter `assertion.sub` in the box next to it. Click **+ Add mapping** and add a second row: `attribute.repository_id` on the left, `assertion.repository_id` on the right.

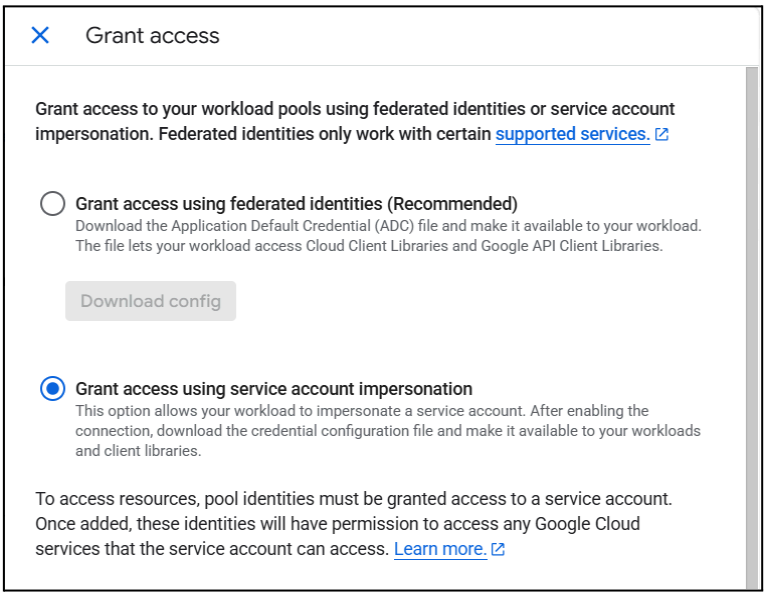


7. Under **Attribute conditions**, click **Add condition**, paste the condition you prepared in step 1, and click **Save**.

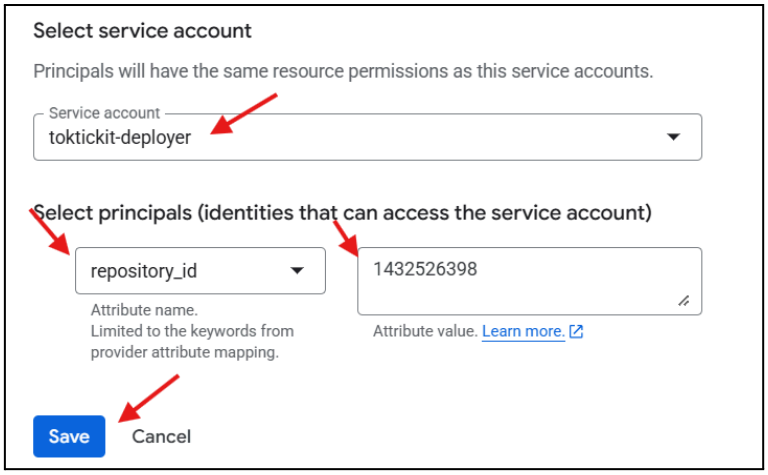


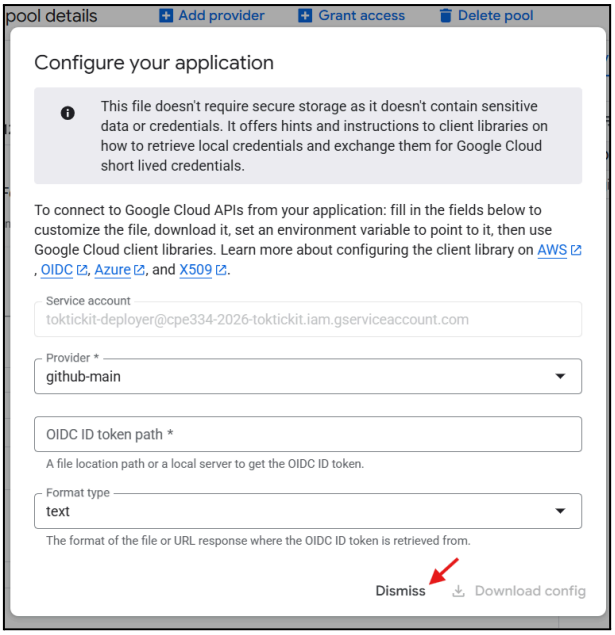
8. Give the deployer account access through the pool: on the **Workload Identity Pools** page click **Grant access**. Choose **Grant access using Service Account impersonation**.



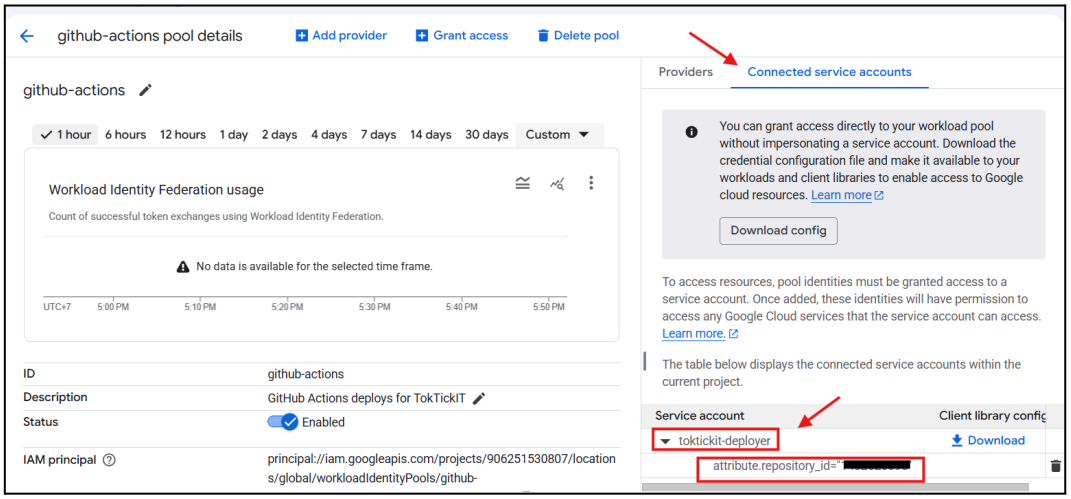


9. Select `toktickit-deployer` in the **Service accounts** list, choose **Only identities matching the filter**, set **Attribute name** to `repository_id` and **Attribute value** to your `REPO_ID`. Click **Save**, then **Dismiss**.

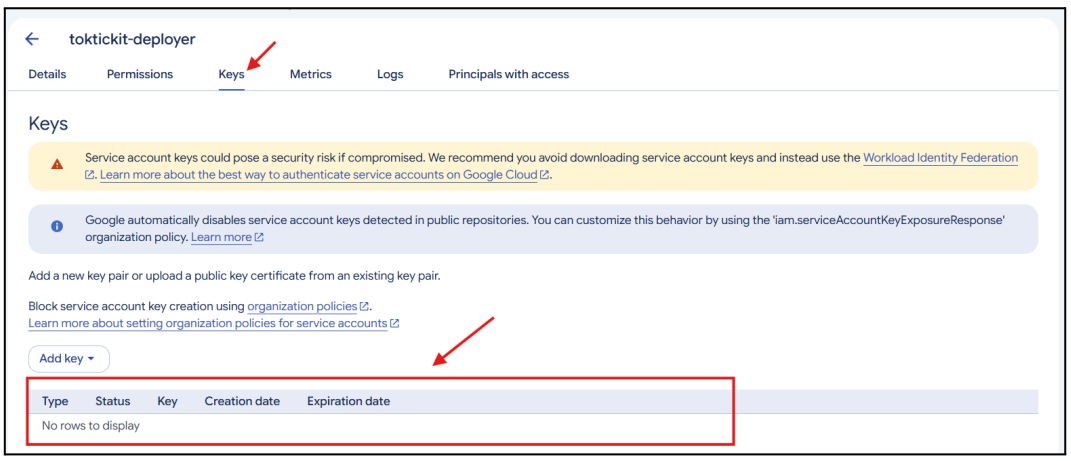




10. Check the result. Open the provider and read the attribute condition again: it must show your repository ID, owner ID, `main`, `deploy.yml` and `environment:demo`.



11. Then go to **IAM & Admin** → **Service Accounts** → `toktickit-deployer` → **Keys**: the list must be **empty**. Never click **Add key**.



6.4 Configure GitHub: environment and variables

Before you begin: 6.3 is done. You have the Project ID, the region from Episode 5, the **Project number**, and the names of your deployer service account, pool and provider.

Names used in this step. Most of these depend on your project and must match what you created earlier.

What it is	Example (TokTickIT)	Your project
Environment name. Enter `demo`. This name is used in three places and must be exactly the same in all of them: 1. the GitHub environment you create here; 2. the line `environment: demo` in `.github/workflows/deploy.yml`; 3. the end of the condition in step 6.3 (`...:environment:demo`).	`demo`	`demo` (keep it; if you change it, change all three places) We recommend you keep `demo`, because the Starter Kit uses it. If you choose another name, change it in all three places. If one of them differs, the deploy workflow cannot sign in to Google Cloud.
`GCP_PROJECT_ID`: your Google Cloud Project ID (not the project name)	`cpe334-team07`	your Project ID
`GCP_REGION`: the region you used in Episode 5	`asia-southeast1`	your region
`WIF_PROVIDER`: the full name of the provider from 6.3	`projects/123456789012/locations/global/workloadIdentityPools/github-actions/providers/github-main`	Replace `123456789012` with your **Project number**; keep the pool and provider names you chose in 6.3
`WIF_SERVICE_ACCOUNT`: the email of the deployer from 6.2	`toktickit-deployer@cpe334-team07.iam.gserviceaccount.com`	`YOUR_APP-deployer@YOUR_PROJECT_ID.iam.gserviceaccount.com`

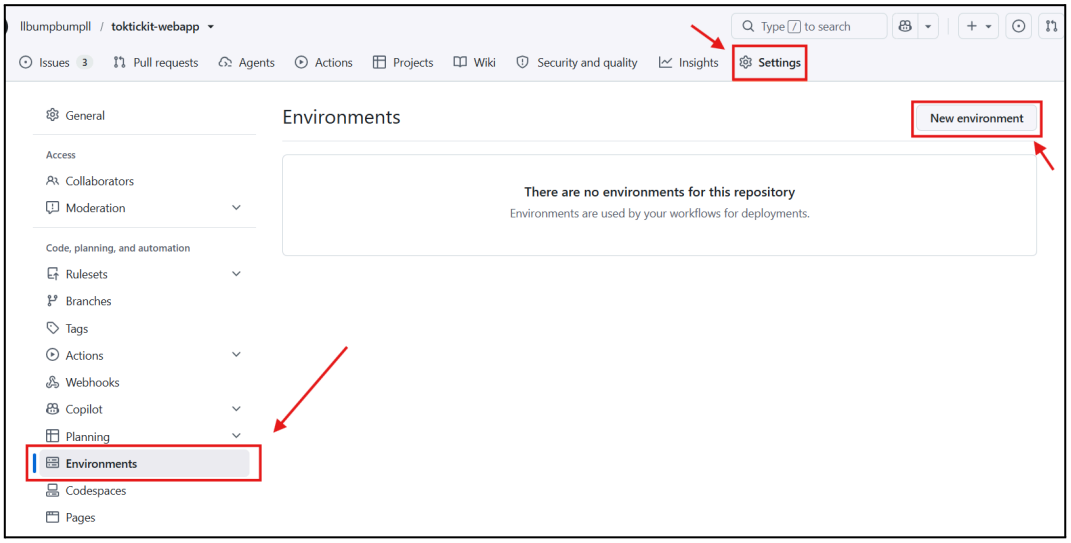
****Where to find your Project number:** in the Console, open the **Dashboard** and look at **Project info** → **Project number**. It is a number, different from the Project ID.

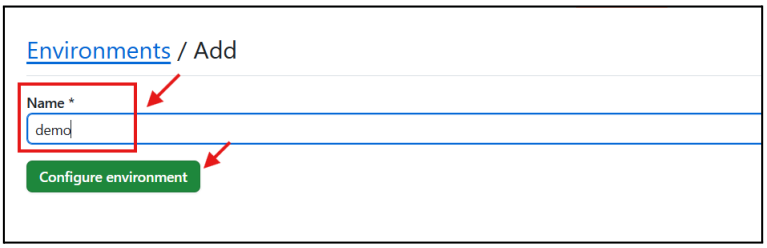
Do not rename the four variable names: `deploy.yml` reads exactly these names.

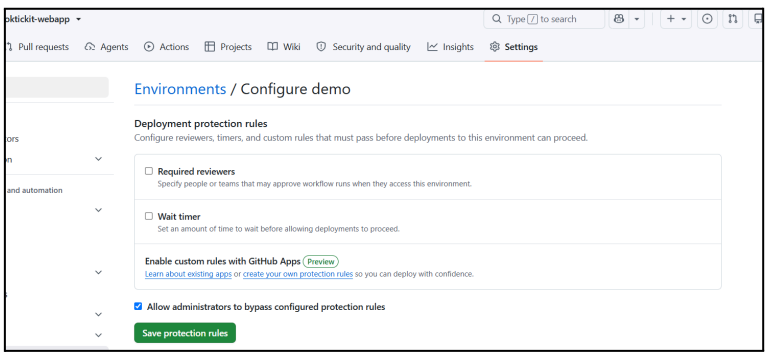
Steps

1. In GitHub, open your repository, then **Settings** → **Environments** → **New environment**. Enter `demo` and click **Configure environment**.

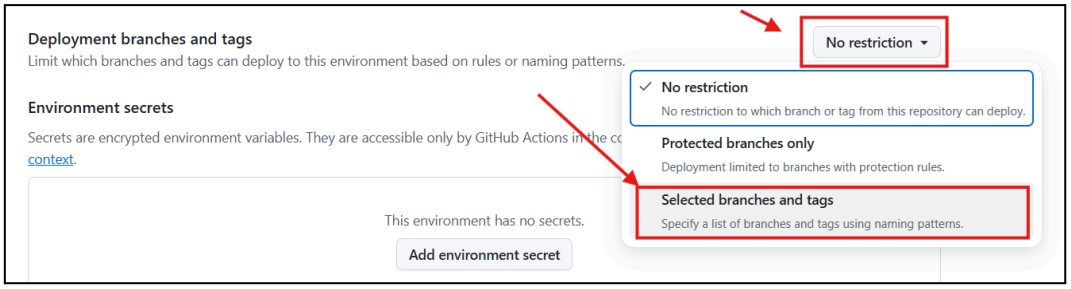
> **Environment name.** The name `demo` is the one used in the Starter Kit's `deploy.yml`, and we recommend you keep it. If you choose another name, it must be exactly the same in three places: the GitHub environment (here), the line `environment:` in `.github/workflows/deploy.yml`, and the end of the `assertion.sub` condition in step 6.3 (`...:environment:ENV_NAME`). If one of them differs, the deploy workflow cannot sign in to Google Cloud.

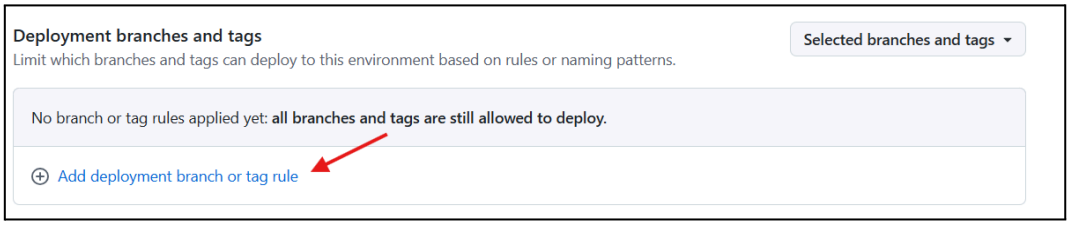


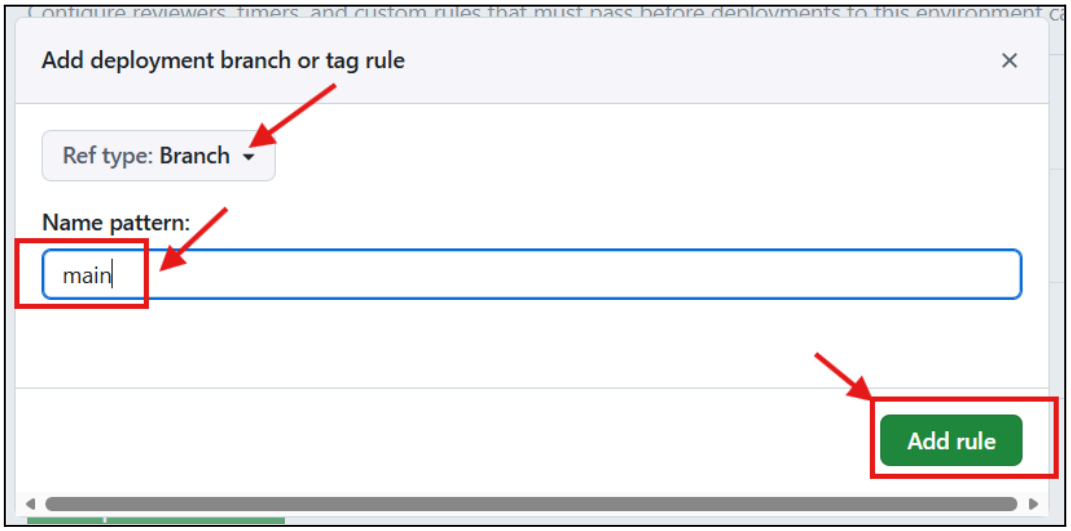




2. Under **Deployment branches and tags**, choose **Selected branches and tags**, click **Add deployment branch or tag rule**, enter `main` and save. Now only `main` can deploy to this environment.



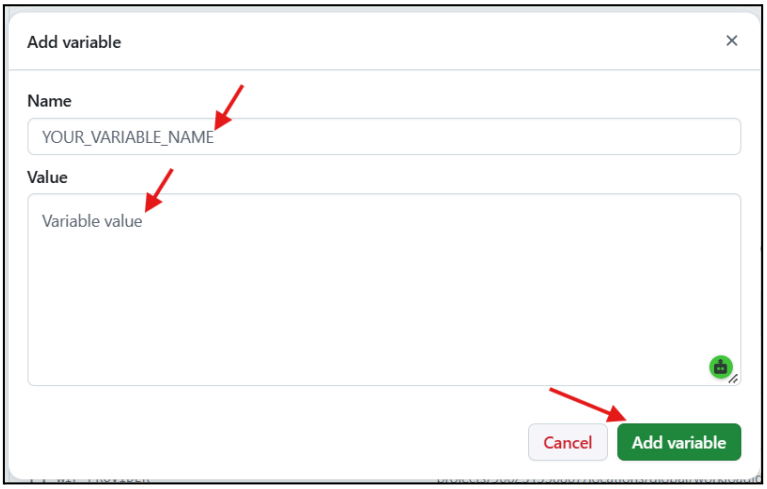




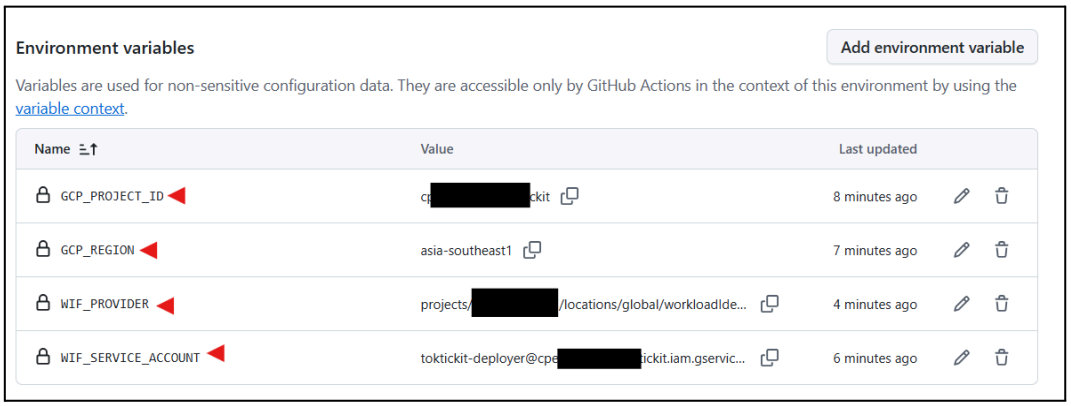


3. Under **Environment variables**, click **Add environment variable** and add the four variables from the table. They are **variables**, not secrets: they say where to deploy and are not passwords. Do not create any secret in this episode.





Name	Value
`GCP_PROJECT_ID`	your Project ID
`GCP_REGION`	your region, for example `asia-southeast1`
`WIF_PROVIDER`	`projects/PROJECT_NO/locations/global/workloadIdentityPools/github-actions/providers/github-main`
`WIF_SERVICE_ACCOUNT`	`YOUR_APP-deployer@YOUR_PROJECT_ID.iam.gserviceaccount.com`



4. Check the result: the environment lists the four names with their values, and the deployment rule shows `main`.

For more detail, see GitHub's guide to managing environments for deployment

6.5 Protect main with a ruleset (the merge gate)

Before you begin:

- The CI workflow from Episode 4 has run at least once in your repository (the check name only appears in the list after a run).
- Step 6.4 is finished: environment `demo` is restricted to `main` and has its four variables.
- You are an admin of the repository.

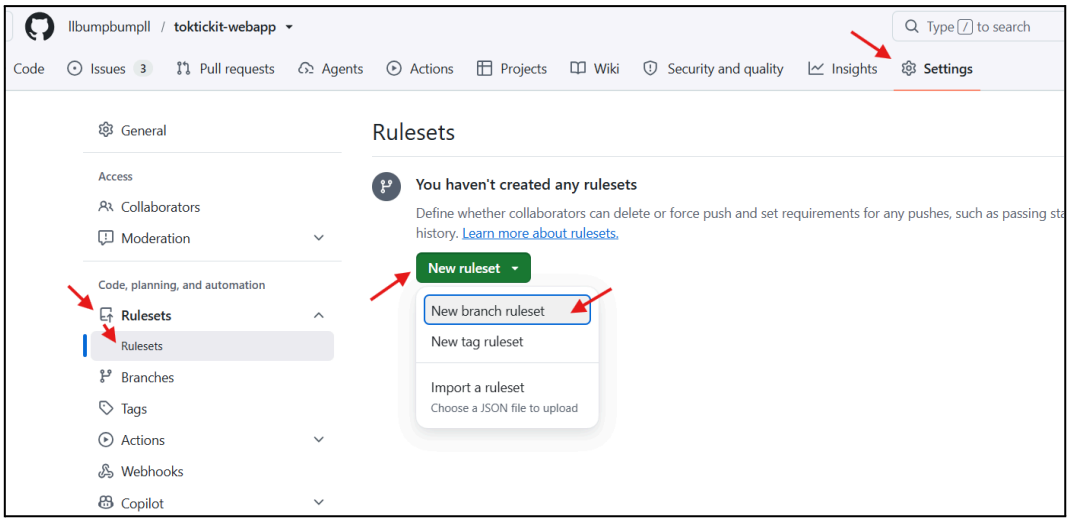
> **Why we do this:** the pipeline deploys whatever reaches `main`. A ruleset makes sure nothing reaches `main` unless a pull request passes the CI check. This is the "quality gate" part of the rubric.

What depends on your project?

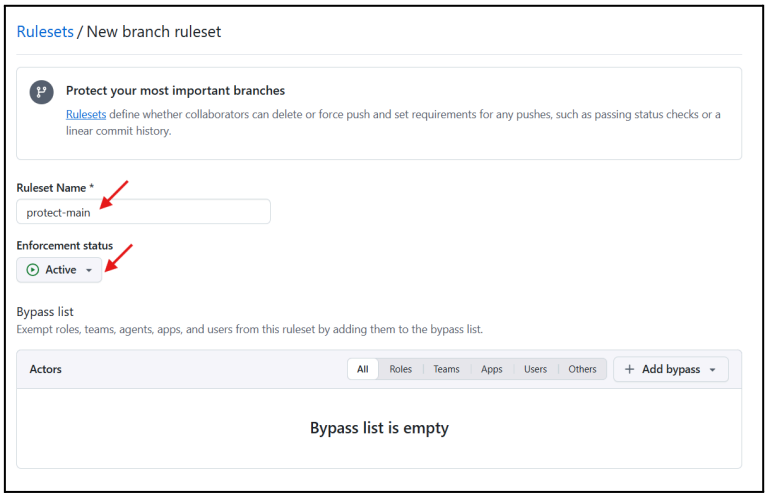
What it is	Example (TokTickIT)	Your project
Name of the CI check that must pass (the job name in your CI workflow)	`test-and-docker`	The job name in your own workflow file
Number of required approvals	`1` (team), `0` (working alone)	Your team size

Steps

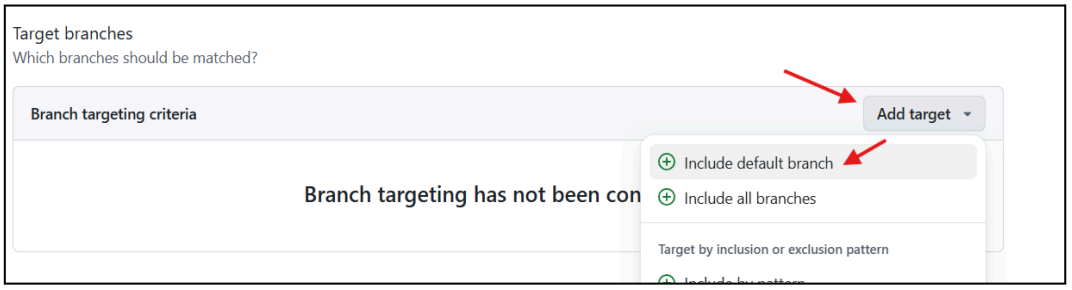
1. Open **Settings** → **Rulesets** → **New ruleset** → **New branch ruleset**.



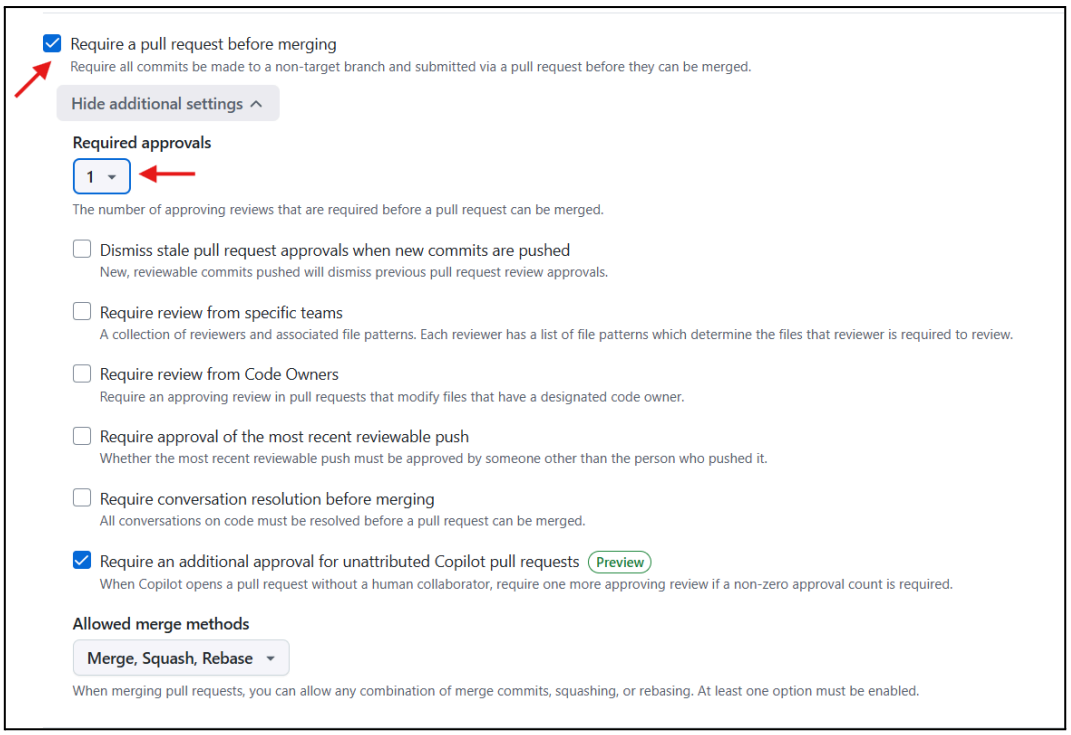
2. **Ruleset Name:** `protect-main`.
3. **Enforcement status:** change *Disabled* to **Active**. A ruleset left on Disabled does nothing.
4. **Bypass list:** leave it empty. Do not add yourself or any role. If someone can bypass, the gate proves nothing.

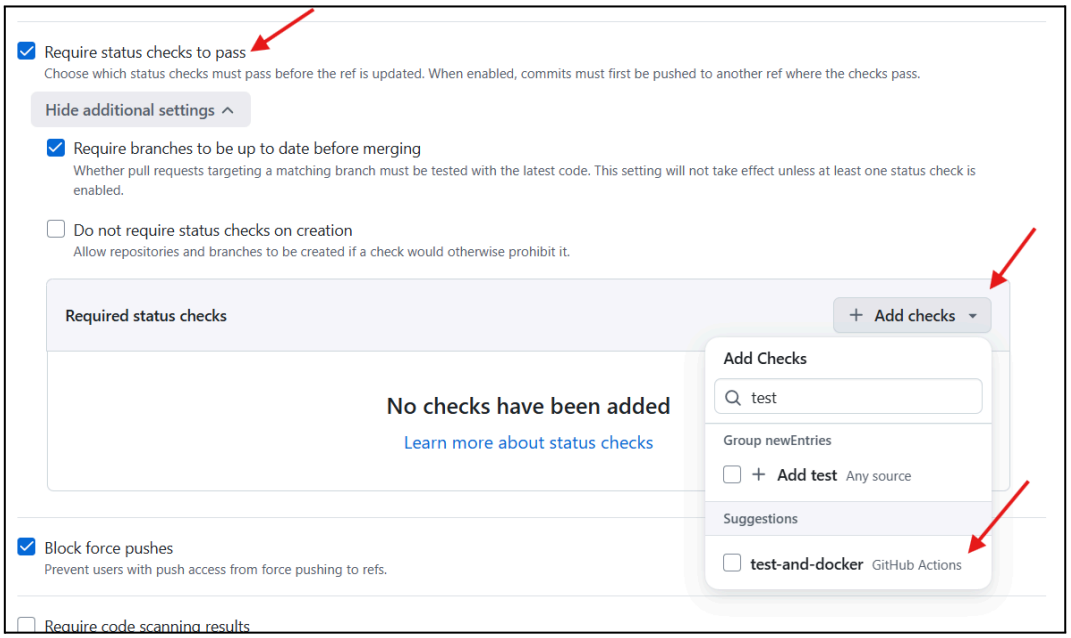


5. **Target branches:** click **Add target** → **Include default branch** (or *Include by pattern* with `main`).

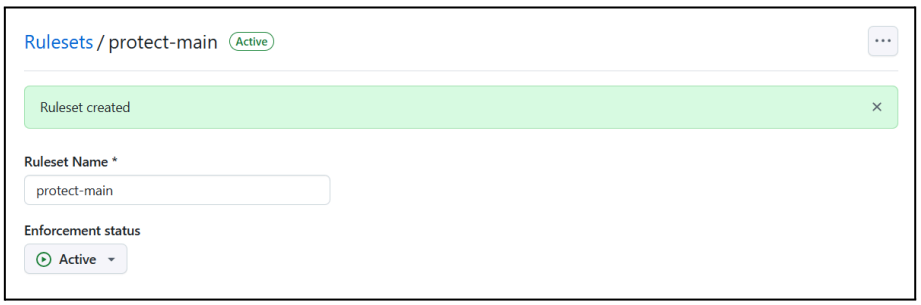


6. In **Rules**, tick:
   - **Restrict deletions** (on by default)
   - **Block force pushes** (on by default)
   - **Require a pull request before merging**, with **Required approvals** set to `1`. If you work alone, set it to `0`: you cannot approve your own pull request, and with an empty bypass list you would not be able to merge.
   - **Require status checks to pass**, then tick **Require branches to be up to date before merging**, then click **Add checks** and select your CI job (`test-and-docker` for TokTickIT). Typing the name is not enough; you must pick it from the list.
   - Leave the other boxes as they are.





7. Scroll to the bottom and click **Create**



6.6 Prove the gate works

Before you begin: The ruleset protect-main is Active (6.5). Your CI workflow also runs on pull requests into main (Episode 4). You can run your tests locally and know where they live.

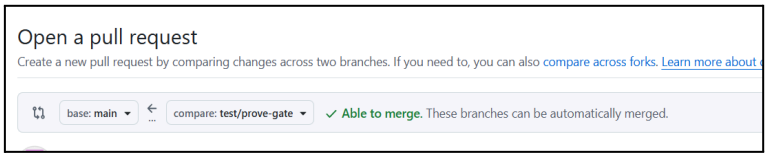
Why we do this: a rule you have never seen block anything is only a guess. Here you break a test on purpose, open a pull request into `main`, and watch GitHub refuse to merge it. The screenshot of this refusal is your evidence for the quality-gate criterion.

What depends on your project?

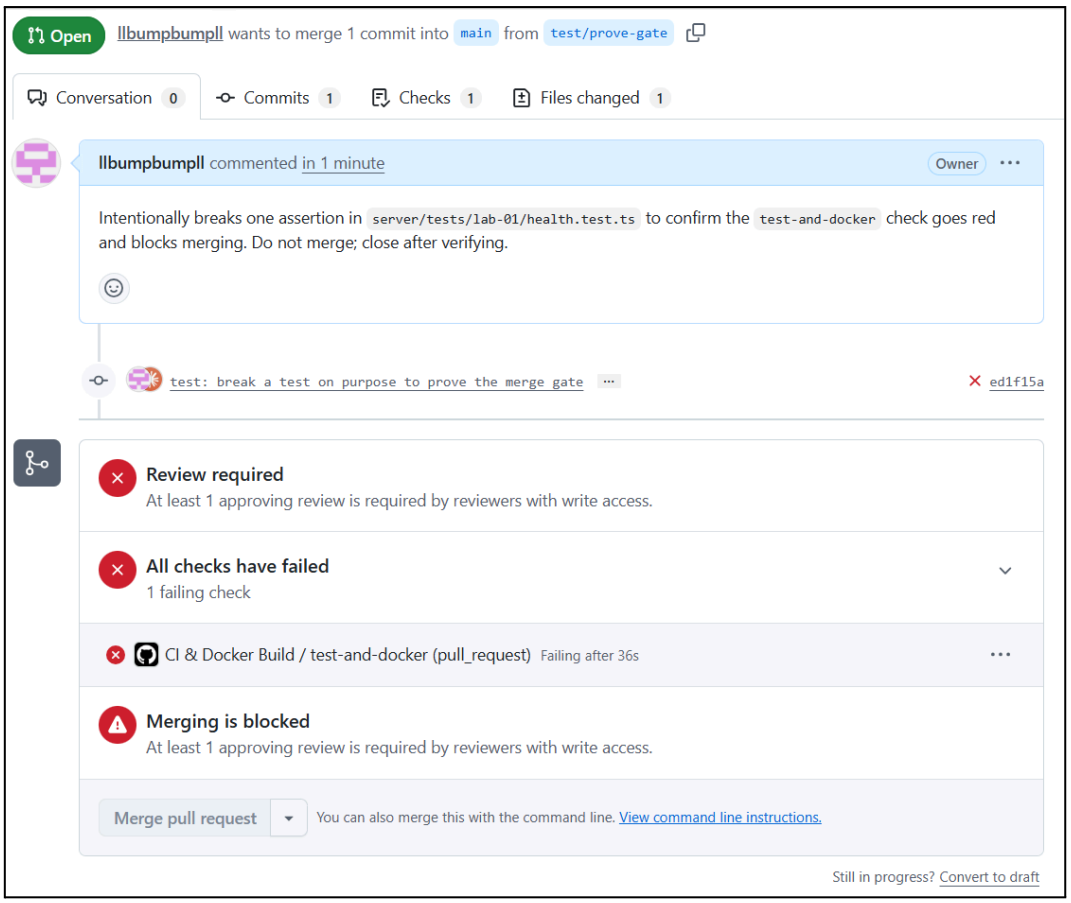
What it is	Example (TokTickIT)	Your project
A test you can break safely	One existing test in the server tests: change the expected value	Any existing test in your stack (Jest, Vitest, pytest, JUnit, and so on)
The command that runs your tests	The one CI uses (`npm test`)	Your own test command

Steps

1. Create a throwaway branch from main: test/prove-gate
2. Break one test on purpose, for example change an expected value so the assertion is wrong. Run your test command locally and confirm it fails.
3. Commit and push the branch
4. On GitHub, open a pull request with **base: `main`** and **compare: `test/prove-gate`**. Open it as a normal pull request, not a draft.



5. Wait for CI. The check `test-and-docker` turns red.
6. Scroll to the merge box at the bottom of the conversation. The merge button must be blocked, with a message saying required checks have not passed.



7. **Do not merge.** Click **Close pull request**, then delete the branch (**Delete branch**). Also remove the local copy
8. Optional second proof: on `main`, make any small change and try `git push origin main`. GitHub should reject it with a message about repository rule violations. Undo the local change afterwards.

Official references: Creating rulesets for a repository (GitHub Docs)

6.7 Close the Issue

1. Comment on Issue #6 with the screenshots from 6.4 to 6.6 (no secrets, no project number).
2. Move Issue #6 to *Done*.

Done when

- The `demo` environment exists, only `main` can deploy to it, and it has the four variables.
- The Workload Identity Federation provider exists and the deployer service account has **no keys**.
- A Pull Request with a failing test cannot be merged: **Merge** is disabled and `test-and-docker` is marked Required.
- No key file, password or secret is in GitHub, in your Issue or in your screenshots.

Note: The names and the exact rules depend on your project and your team. The settings above use TokTickIT as the example. For your own project, "done" means GitHub can deploy to your Google Cloud project without keys, only from your repository and `main`, and a failing required check always blocks the merge.

If something fails

What you see	What to check
"The given credential is rejected by the attribute condition" when the workflow signs in	Your condition does not match the token. Run: `echo "$WIF_CONDITION"` and compare with 6.3: repository ID, owner ID, the `main` branch, the `deploy.yml` file name, the `demo` environment name, and the immutable or older subject format
`test-and-docker` cannot be chosen in the ruleset	The check has not run yet, or its name is different. Open a Pull Request so it runs, then add it again
The Merge button is still enabled	The ruleset is not Active, the required check was ticked but not added, or your account is in the bypass list
You cannot merge your own Pull Request even when checks pass	Required approvals is 1 and you are the author. Ask a teammate to approve, or set approvals to 0 if you work alone
The ruleset page says it is not enforced	Some GitHub plans do not enforce rulesets on a private repository. Read the message on the page and ask your instructor
`gh: command not found` in Cloud Shell	Run 6.1 on your own computer, or read the IDs from `https://api.github.com/repos/YOUR_GITHUB/YOUR_REPO` for a public repository
`keys list` shows a key	Delete it with `gcloud iam service-accounts keys delete` and find out who created it. No key should exist

Official references

- google-github-actions/auth – Workload Identity Federation for GitHub Actions
- GitHub OIDC reference: immutable subject claims – why the token has the IDs in it
- Managing environments for deployment – the `demo` environment and its variables
- Creating rulesets for a repository – the merge gate

Guide/Episode 7: Auto deploy and smoke test

Videos: LEB2 / Learn It / Student Guide: GCP Deployment and CI/CD (Video) / Episode 7

Prerequisite: Episode 4, 5, 6

Issue #7: (branch `feature/auto-deploy` for 7.2 to 7.4: adapt names and add the deploy workflow; no branch for 7.5 and 7.6; a second branch `feature/deploy-proof` for 7.7)

Criterion: Repeatable release pipeline, supports GCP deployment

---

Goal: merging into main runs your tests, builds the image, runs the database migration, deploys to Cloud Run, and checks the public URL. Nobody types a deploy command.

> **Adapt to your project.** This episode is partly project-specific. The pipeline stages are the same for every project, but service names, image path, migration command and the endpoints checked by the smoke test depend on your project and tech stack. Each step marks what to change, with the TokTickIT value as an example.

Before you begin

- Episodes 4, 5 and 6 are finished.
- The Cloud Run service `toktickit`, the job `toktickit-migrate`, the Artifact Registry repository and the Cloud SQL database exist (Episode 5).
- The GitHub environment `demo` and the ruleset on `main` exist (Episode 6).
- `deploy.yml` is in your working copy from the starter kit and is not on `main` yet.
- CI also runs on pull requests into `main` (Episode 4).
- Environment `demo` has its four variables (6.4).

7.1 Understand the workflow

The workflow is provided in the starter kit, so you do not write it yourself. It is the file `.github/workflows/deploy.yml`. The scripts it calls, `deploy/release.sh` and `deploy/smoke_test.py`, are also in the kit. If you used the kit in Episode 5, the two scripts are already in your repository. In 7.4 you copy `deploy.yml` into it and adapt names if needed.

The workflow (name **Deploy demo**) does this, in order:

Stage	What happens	Why	Where it is defined
`test`	Reuses the CI workflow (`ci-docker.yml`)	Only tested code can be released	`deploy.yml` calls your Episode 4 file
Authenticate	Signs in to Google Cloud with Workload Identity Federation (no key)	Uses the setup from Episode 6	`deploy.yml`

Stage	What happens	Why	Where it is defined
Build and push	Builds the root `Dockerfile` and pushes it, tagged with the full commit SHA	Every release is traceable to one commit.	`deploy.yml`
`release.sh`	Resolves the image digest, runs the migration job, updates the Cloud Run service	Database and app change together	`deploy/release.sh`
Smoke test	`deploy/smoke_test.py` checks the public URL	Proves the release really works	`deploy/smoke_test.py`
Record	Writes the image tag, digest and URL to the run summary	Evidence for your report	`deploy.yml`

Steps

1. Open the starter kit and read `.github/workflows/deploy.yml` once. You do not need to change anything yet.
2. Find the stages from the table in the file (look for `test`, `Authenticate`, `Build and push image`, `Verify image, migrate and deploy`, `Smoke test public URL`).

7.2 Check the names against your project

The workflow uses names that must match the resources you created in Episode 5 and the GitHub environment from Episode 6. For TokTickIT they already match, so you only check. For another project, you note what to change and edit it in 7.4.

What it is	Where it is set	Example (TokTickIT)	Your project
Cloud Run service name	`SERVICE_NAME` in `deploy.yml`	`toktickit`	The name from Episode 5
Migration job name	`MIGRATE_JOB_NAME` in `deploy.yml`	`toktickit-migrate`	Your migration job
Artifact Registry repository and image	`.../toktickit/toktickit:TAG` in `deploy.yml` and `release.sh`	repository `toktickit`, image `toktickit`	Your repository and image
GitHub environment	`environment:` in `deploy.yml`	`demo`	The environment name from 6.4

What it is	Where it is set	Example (TokTickIT)	Your project
URLs checked by the smoke test	`smoke_test.py`	`/api/health`, `/api/ready`, `/`, `/api/categories`	Endpoints of your own app
Migration command	The Cloud Run migration job (Episode 5)	Prisma	Your own migration tool

Project, region and Google Cloud sign-in values are **not** in the files. They come from the environment variables you set in 6.4.

Steps

1. Open `deploy.yml` in the kit and find each setting in the table.
2. Open the Cloud Run console and Artifact Registry and compare the names with what you created in Episode 5.
3. Write down any name that differs. You will change it in 7.4, step 6.

Done when: every name in the table matches your Google Cloud resources and your GitHub environment, or you have a list of the ones to change. (This depends on your project and tech stack.)

7.3 Optional: smoke test login

The smoke test always checks health, database readiness, the home page and categories. It can also check that a user can sign in, read their profile and sign out. This is optional. Do it **before** the release in 7.4, otherwise the first deployment skips the login check.

What depends on your project?

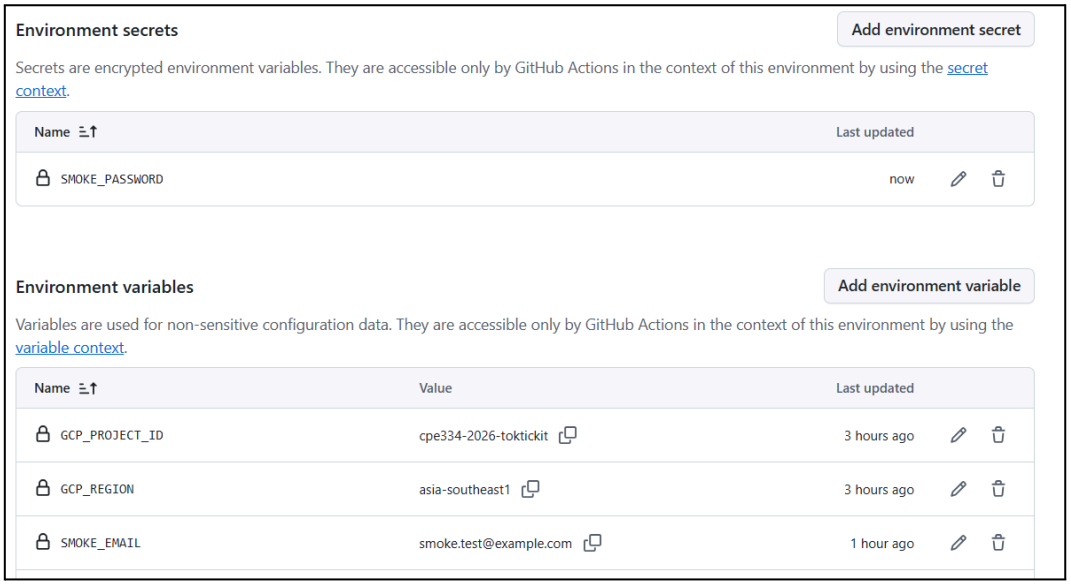
What it is	Example (TokTickIT)	Your project
Endpoints the login check calls (in `smoke_test.py`)	`/api/auth/login`, `/api/auth/me`, `/api/auth/logout`	Your own sign-in endpoints; edit the script if they differ
Test account	A Requester account you create in the app	Any low-privilege account of your app

Before you begin

- Your deployed app has a way to create a user (in TokTickIT the administrator can add users).
- The account must be a dedicated test account with the lowest role. Do not use the administrator, and do not reuse the password of a real person or a demo account.

Steps

1. Create the test account in your deployed app with a strong password. If your app forces a password change at first sign-in, sign in once yourself and change it, so the smoke test can sign in directly.
2. In GitHub open **Settings** → **Environments** → **demo**.
3. Add the **environment variable** `SMOKE_EMAIL` with the account's email.
4. Add the **environment secret** `SMOKE_PASSWORD` with the password (under *Environment secrets*, not variables).
5. Set both or neither. With neither, the script prints `SKIP login`.



Never put a password in the workflow file or in this guide.

Done when: Either both values are set (and the smoke step later prints PASS login and authenticated /api/auth/me), or you choose to skip login and the step prints SKIP login. (The endpoints and the account depend on your project and tech stack.)

7.4 Add the workflow files and release to main

Before you begin

- 7.2 is done (names checked), and 7.3 is done or skipped on purpose. Set the smoke test login first, because the first release already runs the smoke test.
- Your working copy is clean (`git status` shows nothing to commit).
- You know where the starter kit is unzipped (called `KIT` below).
- If a pull request from an earlier episode is still open (for example, the throwaway one from 6.6), close it without merging and delete its branch.

Steps

1. Update `staging` to match `main`.
    a. A release pull request merges `staging` into `main`
    b. This creates a merge commit on `main`, so `staging` falls behind.
    c. The ruleset requires up-to-date branches, so sync first.
2. If `git merge` says “Already up to date," nothing is needed.
3. Create the working branch: feature/auto-deploy (example)
4. Copy **only the files you are missing** from the kit. Usually that is one file.
5. Check with `git ls-files deploy`. If `release.sh` or `smoke_test.py` is missing, copy it from `/deploy/` as well.
    a. **Do not copy `ci-docker.yml` or `README.md`.** Yours are customized (CI also runs on `staging`), and the kit versions would overwrite them.
6. Apply the name changes you noted in 7.2, in `deploy.yml` and also in `deploy/release.sh` and `deploy/smoke_test.py` where the same names appear, so all three agree. For TokTickIT, there is nothing to change. Then check, commit, and push.
7. Open a pull request into `staging` with `Part of #7`. Wait for green checks, then merge.
8. Open the **release pull request**: base `main`, compare `staging`, with `Part of #7`. Wait until `test-and-docker` is green, then merge.
9. Merging pushes to `main`, which starts **Deploy demo** automatically. This step is also the first real test of your Episode 6 setup (sign-in to Google Cloud without a key).



Done when: `deploy.yml` is on `main` (together with the `deploy/` scripts) and a **Deploy demo** run appears in the Actions tab. (Whether the run is green is checked in 7.5.)

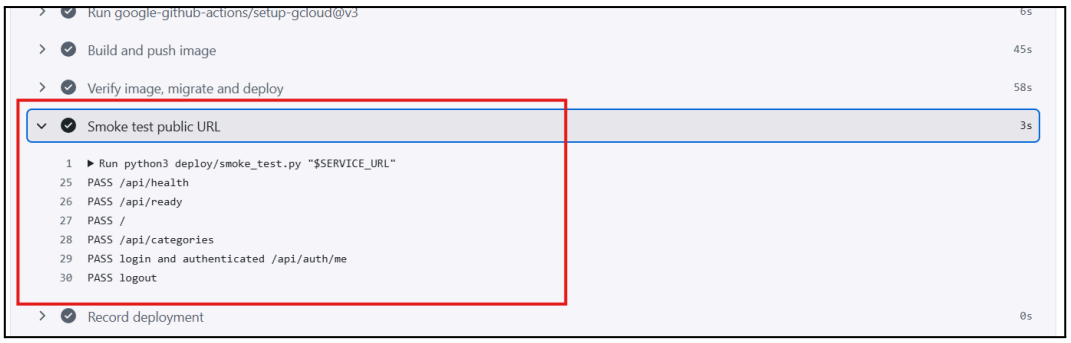
7.5 Watch the release.

Before you begin

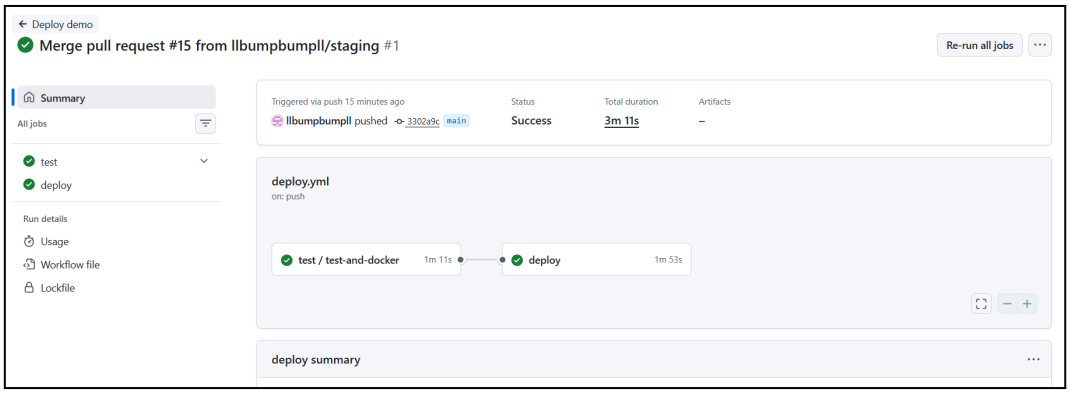
- The release pull request from 7.4 is merged into `main`.
- Your app allows public access (Episode 5), so the URL opens without sign-in.

Steps

1. Open **Actions** → **Deploy demo** → **the latest run**.
2. Wait for the jobs `test` and `deploy` to finish. The first run builds the whole image, so allow several minutes.
3. Open the **Smoke test public URL** step. You should see lines like `PASS /api/health`, `PASS /api/ready`, `PASS /`, `PASS /api/categories`. If you set the login check in 7.3 you also see `PASS login and authenticated /api/auth/me` and `PASS logout`; otherwise you see `SKIP login`.



4. Open the run **Summary** and read **TokTickIT demo deployment**: the image tag (a full commit SHA), the digest, and the URL.
5. Open the URL in a browser. The first request can take a few seconds because Cloud Run starts a new instance.

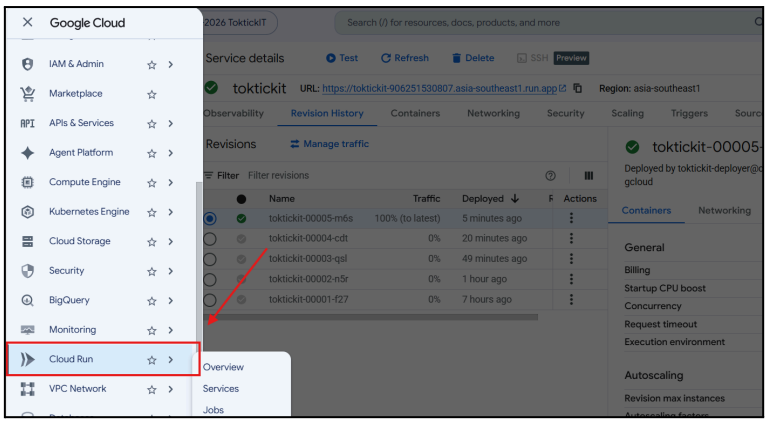


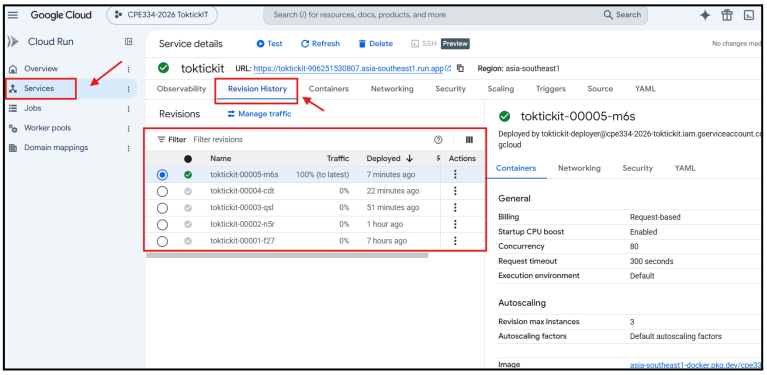


7.6 Check it in the Google Cloud Console

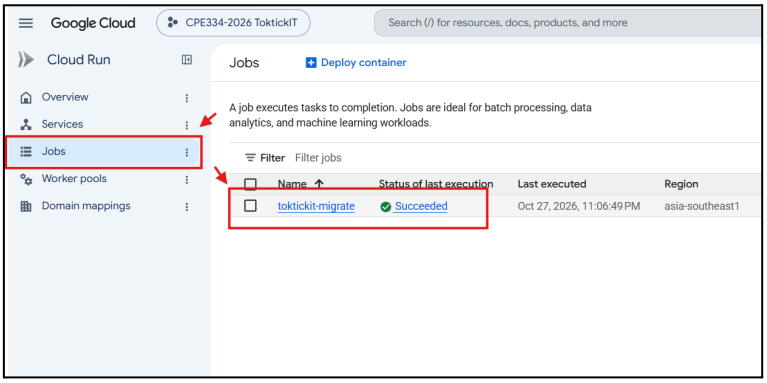
Steps

1. **Revision history of your service:** open **Cloud Run** (https://console.cloud.google.com/run/services), click your service, and open the **Revision history** tab. The newest revision must have 100% of the traffic. Select it and compare the container image digest in the details panel with the digest in the run summary.

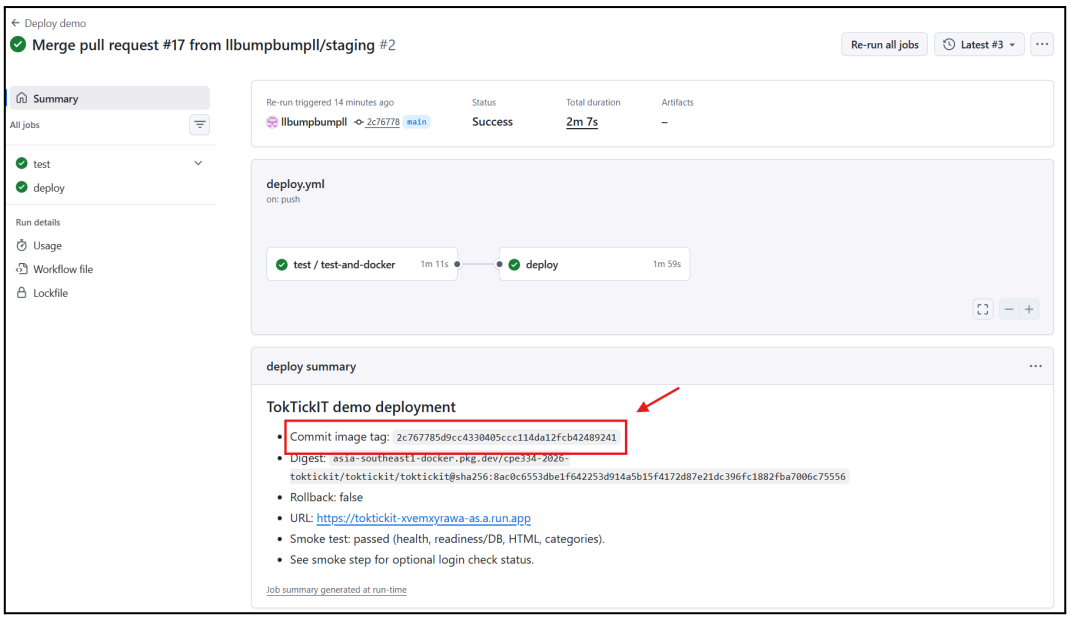


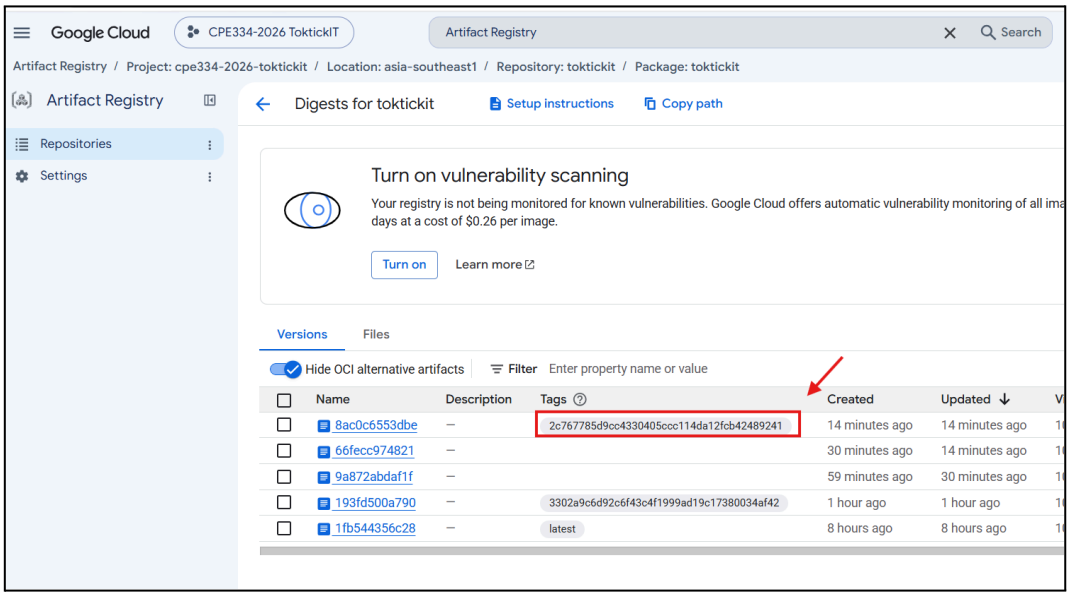


2. Open that revision and compare its container image digest with the digest in the run summary. They must be the same.
3. **Cloud Run → Jobs → your migration job → Executions:** the latest execution must show success.



4. **Artifact Registry → your repository → your image:** there is an image tagged with the same commit SHA as in the summary .
    a. There is an image tagged with the same commit SHA as in the run summary (Actions → Deploy demo → your run → Summary → “Commit image tag”).





Done when: the revision, the job execution, and the image all match the release you just made. (Console labels may differ slightly from this text; follow the meaning.)

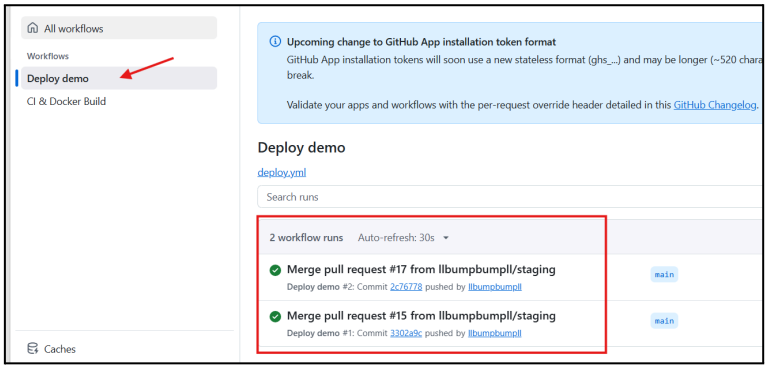
7.7 Prove it is repeatable

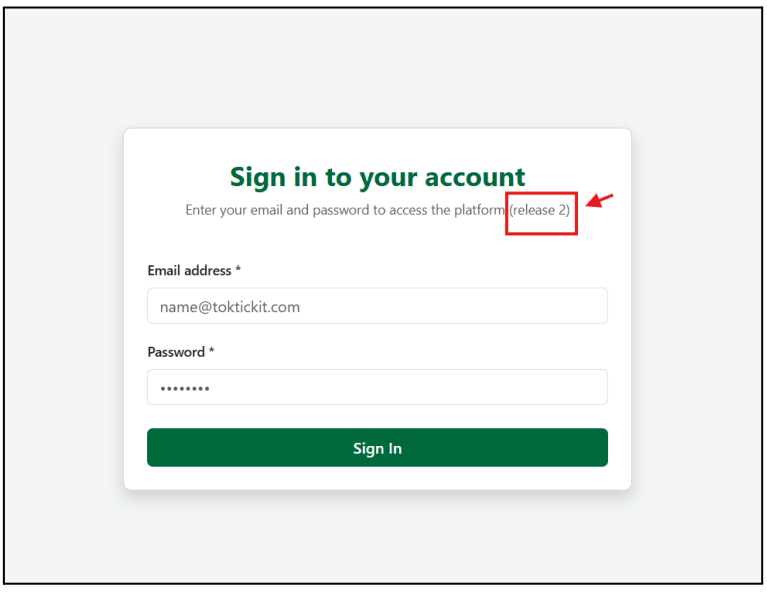
Before you begin

- 7.5 and 7.6 are done.
- Your first release has put a merge commit on `main`, so `staging` is behind again.

Steps

1. Sync `staging` with `main`, as in 7.4 step 1.
2. Create `feature/deploy-proof` from `staging`.
3. Make a small visible change that users can see in the browser (for example a sentence on the home page). Run your tests locally.
4. Commit, push, and open a pull request into `staging` with `Part of #7`. Wait for green checks, then merge.
5. Open the release pull request (base `main`, compare `staging`, `Part of #7`), wait for green checks, then merge.
6. Watch **Deploy demo** start by itself and finish green. You type no deploy command.
7. Reload the URL (hard refresh, `Ctrl+F5`) and look for your change. Also check that Cloud Run now has a second revision.





Done when

- `deploy.yml` is on `main`, and a merge into `main` starts **Deploy demo** with no manual command.
- The run is green, including the smoke test (`PASS` lines for health, readiness, home page, and categories; login is `PASS` or `SKIP`).
- The image tag in the run summary equals the commit you merged, and the Cloud Run revision uses the same image digest.
- The migration job execution succeeded, and the newest revision has 100% traffic.
- A second release, started only by merging, shows your change on the public URL.
- (The checks, names, and endpoints depend on your project and tech stack.)

If something fails

Open the failed run (Actions → Deploy demo), find the first red step, and use the matching row.

Symptom	Likely cause	Fix
Release pull request says the branch is out of date	`staging` is missing the last merge commit of `main`	Merge `origin/main` into `staging`, push (7.4 step 1)
CI no longer runs on `staging` after copying from the kit	`ci-docker.yml` was overwritten	Restore your Episode 4 version (`branches: [main, staging]`)
No Deploy demo run after the merge	`deploy.yml` is not in `.github/workflows/` on `main`	Check the path on `main`
Authentication: “rejected by the attribute condition”	The 6.3 condition does not match (environment name, IDs, filename)	Compare with the 6.3 template; the environment name must match everywhere
`Validate deployment settings` fails, or “Branch is not allowed to deploy”	A variable in environment `demo` is empty, or the run did not start from `main`	Check the four variables in 6.4; run from `main` only
Push or deploy: permission denied	A role is missing on the deployer account	Re-check the roles from 6.2
Migration job fails	Database not reachable, secret missing, or a migration error	Read the job execution logs in Cloud Run
Smoke test fails	The app did not start, cannot reach the database, or the login settings are wrong	Read the Cloud Run revision logs; set both `SMOKE_EMAIL` and `SMOKE_PASSWORD`, or neither

After fixing a setup problem (not the code): open the failed run and click Re-run all jobs. No new pull request is needed.

Official references

- Reuse workflows (GitHub Docs)
- Manage revisions (Cloud Run)
- Execute jobs (Cloud Run)
- Manage job executions (Cloud Run)
- Logging and viewing logs in Cloud Run

Guide/Episode 8: Persistence and rollback

Videos: LEB2 / Learn It / Student Guide: GCP Deployment and CI/CD (Video) / Episode 8

Prerequisite: Episode 7 is done, with two releases deployed by the pipeline: **A** (the first one) and **B** (the one with your visible change). `deploy.yml` is on `main`.

Issue #8: no-branch; uses the Actions tab, app, and the Google Cloud Console only

Criterion: GCP deployment and persistence; repeatable release pipeline

---

Goal: prove that your data survives a release and a rollback and that you can put an older version back by running a workflow. A rollback changes only the app image; it does not roll back the database.

> **Adapt to your project.** This episode is partly project-specific. The rollback steps are the same for every project. What data you create and check depends on your project and tech stack: only data stored in the database (or another cloud service) survives, and data written to the container's own disk is lost. TokTickIT keeps tickets, users, sessions and attachments in Cloud SQL.

Before you begin

- You can sign in to your app on the public Cloud Run URL.
- You have the full 40-character SHA of release **A**: open its **Deploy demo** run, **Summary**, "Commit image tag".

8.1 Roll back to release A

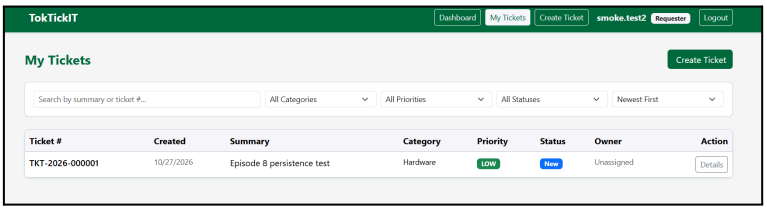
Steps

1. **Create test data in your own app.** Open your public URL, sign in, and create a record that uses a main feature of your project (TokTickIT example: a ticket). If your app lets users upload a file, add one and compute its checksum on your computer:

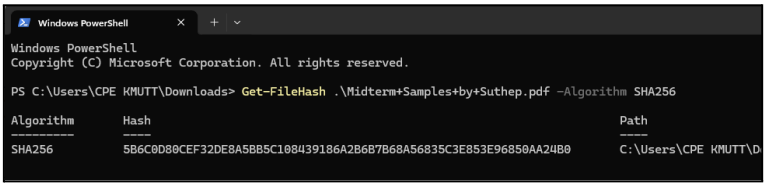
```
Get-FileHash .\your-file.pdf -Algorithm SHA256
```

macOS or Linux:

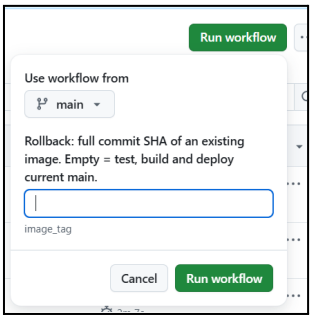
```
shasum -a 256 your-file.pdf
```



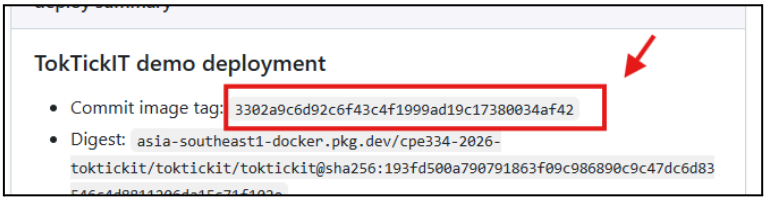


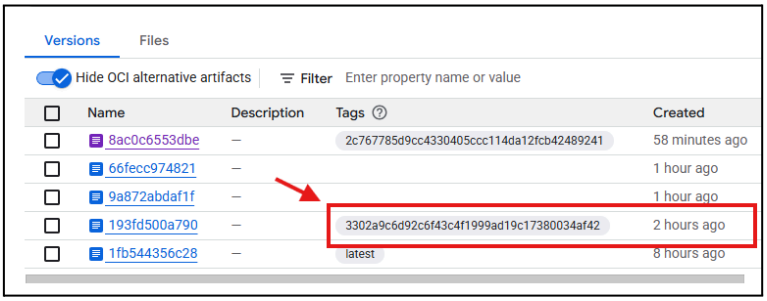


2. Write down the record's title or number and the checksum.
3. Open **Actions** → **Deploy demo** → **Run workflow**.



4. Select branch `main`, paste the full SHA of release **A** into `image_tag`, and run it.
5. Open the run. The test and build steps must be skipped; the deploy and the smoke test must pass.
6. In the Cloud Run console, the newest revision has 100% traffic and uses the image digest of release **A**.
7. Reload the app (`Ctrl+F5`). The visible change from 7.7 is gone.



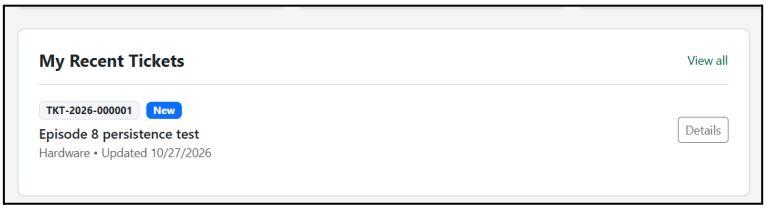


Done when: you have a record written down, the app shows the older version, and Cloud Run serves release A's image.

8.2 Check that your data survived

Steps

1. Reload the app. A signed-in user should still be signed in.
2. Open the record from 8.1. It must still be there.
3. If you uploaded a file, download it and compute its checksum again. It must equal the one you wrote down.




```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\CPE KMUTT\Downloads> Get-FileHash .\Midterm+Samples+by+Suthep.pdf -Algorithm SHA256

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          5B6C0D80CEF32DE8A5BB5C108439186A2B6B7B68A56835C3E853E96850AA24B0       C:\Users\CPE KMUTT\Downloads\...


PS C:\Users\CPE KMUTT\Downloads> Get-FileHash .\Midterm+Samples+by+Suthep+download.pdf -Algorithm SHA256

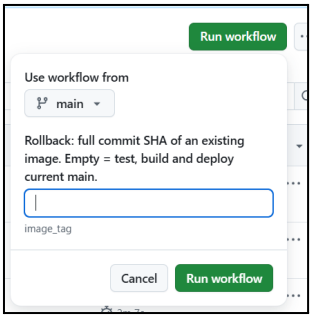
Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          5B6C0D80CEF32DE8A5BB5C108439186A2B6B7B68A56835C3E853E96850AA24B0       C:\Users\CPE KMUTT\Downloads\...
```

Done when: the record (and the file, with the same checksum) and your sign-in are still there. (What you check depends on your project and tech stack.)

8.3 Roll forward

Steps

1. Open **Actions** → **Deploy demo** → **Run workflow**, select `main`, and paste the **full 40-character SHA of release B** into image_tag (or leave image_tag empty to test, rebuild, and deploy the current `main`). Using release **B's SHA** reuses the exact image you already tested and skips the test and build steps.



2. When the run is green, reload the app. The visible change is back.



3. Repeat the data check from 8.2.

Done when: your change is back and the same data is still there.

Done when

- The record and attachment created before the rollback are still there, with the same checksum, after the rollback and after the roll-forward.
- The rollback ran with tests and build skipped, and Cloud Run served the older image.
- The roll-forward restored the current version through the pipeline.
- (The data you check depends on your project and tech stack.)

If something fails

Symptom	Likely cause	Fix
"Image tag must be a full 40-character commit SHA"	A short SHA was used	Copy the full SHA from the run summary
`Image digest not found`	That commit never built an image	Use the "Commit image tag" from a successful **Deploy demo** run
No **Run workflow** button, or "Branch is not allowed to deploy"	The workflow is not on `main`, or another branch is selected	Check `deploy.yml` on `main` and select `main`
Data is missing after the rollback	The data was not stored in the database (for example saved on the container disk)	Move it into the database or a storage bucket, then redeploy
The page does not change after the rollback	Browser cache, or traffic is not on the new revision	Hard refresh; check the Cloud Run revision traffic

Official references

- Rollbacks, gradual rollouts, and traffic migration (Cloud Run)

Guide/Episode 9: Pause when you are not using it

Videos: none (self-check)

Prerequisite: Episodes 5 to 8 are done and your app is running on Cloud Run with Cloud SQL.

Issue no-branch; this episode uses Cloud Shell only

Criterion: It keeps your cost low between labs.

Goal: stop paying for what you do not use, without deleting anything. You pause the database and close public access, and later you bring both back. Your data stays.

Why this matters: Cloud SQL runs and is billed all day, even with no users (about USD 10.7 per month for the instance at the size used in this lab, plus about USD 2.4 for 10 GB of storage; prices from the starter kit guide). Cloud Run scales down to zero when nobody visits, so it costs almost nothing while idle. Stopping the database is the part that saves money. A stopped database still costs for storage and backups.

Do not pause while a pipeline run is in progress, during a demo, or before your work is graded

Adapt to your project. This episode is partly project-specific. The commands are the same for every project, but the names depend on what you created in Episode 5.

What it is	Example (TokTickIT)	Your project
Cloud Run service	`toktickit`	Your service name
Cloud SQL instance	`toktickit-pg`	Your instance name
Project ID and region	`cpe334-team07`, `asia-southeast1`	Your own

Before you begin

- Your app works on the public URL (open it and sign in).
- You can open Cloud Shell, and your account can manage Cloud Run and Cloud SQL.
- Set your values in Cloud Shell:
    - export PROJECT_ID=YOUR_PROJECT_ID
    - export REGION=YOUR_REGION

9.1 Pause

Steps (code)

1. Close public access, so nobody can open the app:

```
gcloud run services remove-iam-policy-binding toktickit \
    --member=allUsers --role=roles/run.invoker \
    --region $REGION --project $PROJECT_ID

# Also enforce IAM invoker check (in case "Allow public access" was toggled in the Console):
gcloud run services update toktickit \
     --invoker-iam-check \
     --region $REGION --project $PROJECT_ID
```

2. Stop the database:

```
gcloud sql instances patch toktickit-pg --activation-policy=NEVER --project $PROJECT_ID
```

3. Check it:

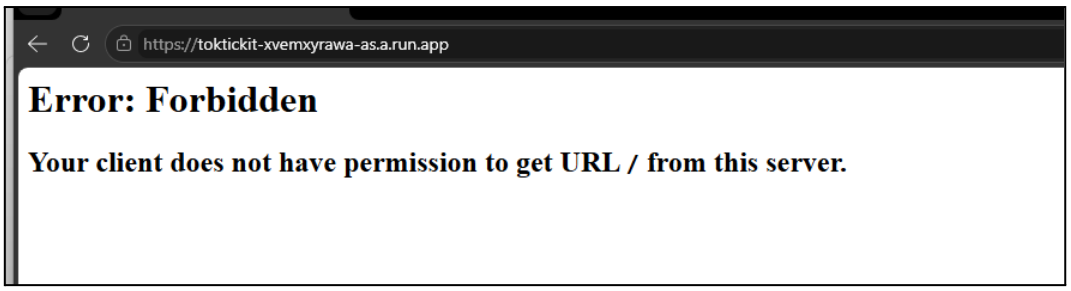
```
gcloud sql instances describe toktickit-pg --project $PROJECT_ID
--format="value(settings.activationPolicy)"
```

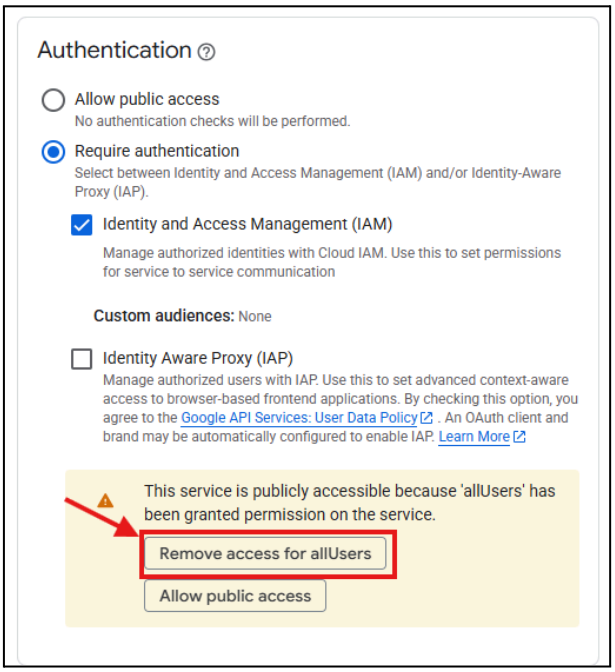
The answer must be NEVER. Opening the app URL in a browser now shows an error (403).

Done when: the database shows NEVER and the public URL no longer opens your app. Your data and your images are still there.

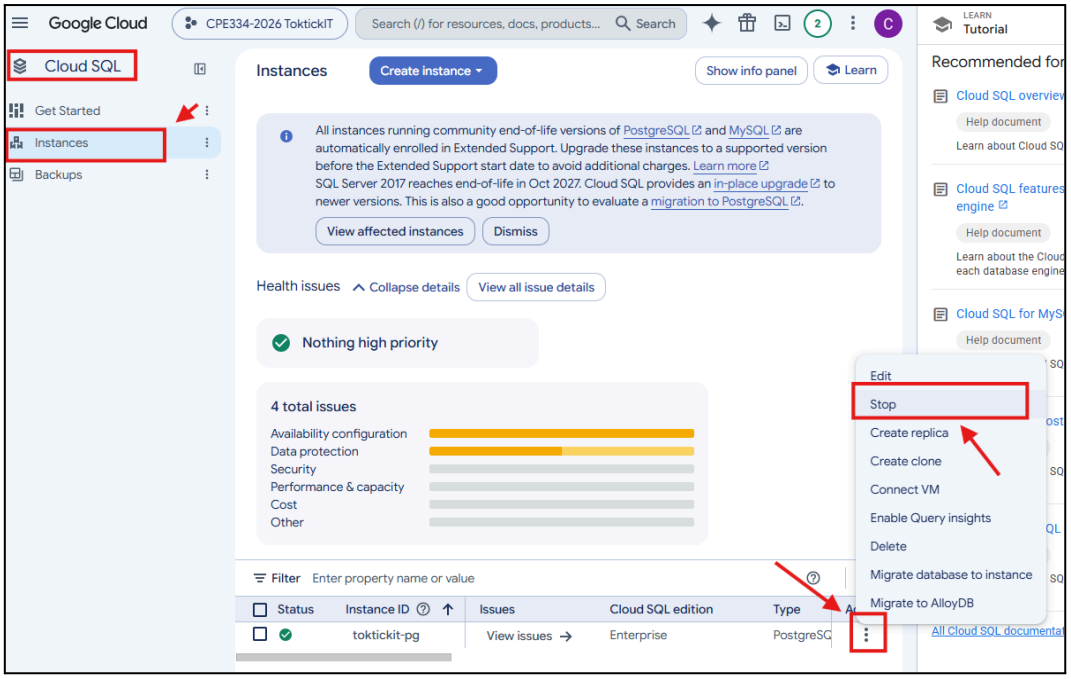
Steps (console)

1. **Close public access.** Open **Cloud Run** (https://console.cloud.google.com/run/services) and click your service. Open the **Security** tab,
    a. If the page says “This service is publicly accessible because ‘allUsers’ has been granted permission on the service”, click **Remove access for allUsers**.
    b. If the page shows **Allow public access** selected instead, select **Require authentication**, choose **Identity and Access Management (IAM)**, click **View diff & redeploy**, then **Deploy changes**.)
    c. Reload your app URL to check: it must show an error (403).





2. **Stop the database.** Open **Cloud SQL** → **Instances**, click your instance, and click **Stop**. In the dialog that says “Stop database instance?” click **Stop**.
   Link: https://console.cloud.google.com/sql/instances
3. Wait until the instance shows as stopped. Open your app URL: **it now shows an error (403).**


Google Cloud
CPE334-2026 TokticklT
Search (/) for resources, docs, products...
Search
Cloud SQL
Get Started
Instances
Backups
Instances
Create instance
Show info panel
Learn
All instances running community end-of-life versions of PostgreSQL and MySQL are automatically enrolled in Extended Support. Upgrade these instances to a supported version before the Extended Support start date to avoid additional charges. Learn more
SQL Server 2017 reaches end-of-life in Oct 2027. Cloud SQL provides an in-place upgrade to newer versions. This is also a good opportunity to evaluate a migration to PostgreSQL.
View affected instances
Dismiss
Health issues
Collapse details
View all issue details
Nothing high priority
4 total issues
Availability configuration
Data protection
Security
Performance & capacity
Cost
Other
Filter
Enter property name or value
Status
Instance ID
Issues
Cloud SQL edition
Type
toktickit-pg
View issues
Enterprise
PostgreSQ
Edit
Stop
Create replica
Create clone
Connect VM
Enable Query insights
Delete
Migrate database to instance
Migrate to AlloyDB
LEARN
Tutorial
Recommended for
Cloud SQL overview
Help document
Learn about Cloud SQ
Cloud SQL features
engine
Help document
Learn about the Cloud
each database engine
Cloud SQL for MyS
Help document
All Cloud SQL documentat

9.2 Resume

Steps (code)

1. Start the database:

```
gcloud sql instances patch toktickit-pg --activation-policy=ALWAYS --project $PROJECT_ID
```

2. Open public access again:

```
gcloud run services add-iam-policy-binding toktickit \
  --member=allUsers --role=roles/run.invoker \
  --region $REGION --project $PROJECT_ID
```

3. Wait one to three minutes for the database to start, then open `YOUR_URL/api/ready` in a browser. It must report that the database is connected. Then sign in to your app.

Steps (console)

1. **Start the database.** Open **Cloud SQL** → **Instances**, click your instance, and click **Start**. In the dialog "Start database instance?" click **Start**.
2. **Open public access again.** Open **Cloud Run** → **your service** → **Security**, select **Allow public access**, click **View diff & redeploy**, then **Deploy changes**.
    a. **Note**: if you open public access with this Console toggle, removing allUsers via CLI alone will not close it next time—either switch back to **Require authentication** here in the Console, or run:

    ```
    gcloud run services update toktickit --invoker-iam-check --region $REGION
    --project $PROJECT_ID
    ```

3. Wait one to three minutes for the database to start, then open `YOUR_URL/api/ready` in a browser. It must report that the database is connected. Then sign in to your app.

Done when: `/api/ready` reports the database as connected and you can sign in and see your data.

9.3 Rules while paused

- **Do not merge into `main` while paused. Deploy demo** would fail: the migration cannot reach the database and the smoke test cannot open the URL. Resume first, then merge.
- If a run failed because the app was paused, resume, then open the failed run and click **Re-run all jobs**.
- Pausing is not the same as the teardown in Episode 10: nothing is deleted here.

If something fails

Symptom	Likely cause	Fix
“Permission denied” on a command	Your account lacks rights on Cloud Run or Cloud SQL	Use the account that created the project, or ask the owner
URL still shows 403 after resume	The public access binding was not restored (or an organisation policy blocks public access)	Run the step 2 command of 9.2 again and read its error
`/api/ready` fails right after resume	The database is still starting	Wait a few minutes and try again
Deploy demo failed while paused	Expected: the database and URL were not available	Resume, then re-run the failed run

Guide/Episode 10: Teardown at the end of term

Videos: none (self-check)

Prerequisite: Your work is graded (or your instructor says you may shut down). Episodes 5 to 8 created the Google Cloud resources.

Issue no-branch; this episode uses Cloud Shell and the consoles only

---

Goal: delete everything you created in Google Cloud, check that nothing is left, and stop the pipeline from trying to deploy to resources that no longer exist.

Do not run the command before grading. The database is deleted with all its data, and the demo disappears. This action cannot be undone.

Adapt to your project. This episode is partly project-specific. `deploy/teardown.sh` has the TokTickIT names written in it. If your project uses other names, edit the list at the top of the script first.

What it is	Example (TokTickIT)	Your project
Cloud Run service and job	`toktickit`, `toktickit-migrate`	Your names from Episode 5
Cloud SQL instance	`toktickit-pg`	Your instance
Artifact Registry repository	`toktickit`	Your repository
Secrets	`toktickit-database-url`, `toktickit-admin-password`, `toktickit-demo-password`	Your secrets
Runtime service account	`toktickit-runner`	Your account

Before you begin

- You know your project ID and region (the ones you used in Episode 5).
- You can open Cloud Shell and your repository is there (Episode 5).
- You are sure the project holds nothing you still need.

10.1 Save your evidence

Steps

1. Keep what you need for your report: screenshots, links to the successful **Deploy demo** runs, and the Cloud Run URL you submitted.
2. Export any data you want to keep. The database is deleted with everything in it.

Done when: your report evidence is saved outside Google Cloud.

10.2 Preview what will be deleted

Steps

1. In Cloud Shell, open your repository folder and set your values:

```
export PROJECT_ID=YOUR_PROJECT_ID
export REGION=asia-southeast1
```

2. Run the script in preview mode. It only prints the commands:

```
DRY_RUN=1 bash deploy/teardown.sh
```

3. Read the list. It must show only your own resources in your own project.

Done when: the printed list matches the resources you expect to delete, and the project ID is yours.

10.3 Run the teardown

Steps

1. Run it for real:

```
bash deploy/teardown.sh
```

2. When it asks you to type the project ID, type it exactly to confirm. A different answer stops the script.
3. Wait for the end. The message “Teardown completed" does **not** prove everything was deleted, because each command is allowed to fail. Read the output for error lines.

Done when: the script finished and you have read its errors, if any.

10.4 Check what is left

Steps

1. List what still exists. Each list should be empty (or show only things you did not create):

```
gcloud run services list --region $REGION --project $PROJECT_ID

gcloud run jobs list --region $REGION --project $PROJECT_ID

gcloud sql instances list --project $PROJECT_ID

gcloud artifacts repositories list --location $REGION --project $PROJECT_ID

gcloud secrets list --project $PROJECT_ID

gcloud iam service-accounts list --project $PROJECT_ID

gcloud storage buckets list --project $PROJECT_ID
```

2. Delete anything of yours that is still there. The script does **not** remove Cloud Build storage buckets, so check the bucket list.
3. Open **Billing** in the Google Cloud console and check the costs again after it updates. A budget is an alert, not a spending cap.

Done when: none of the lists show your resources, and billing shows no running charges.

10.5 Remove the Episode 6 leftovers and stop the pipeline

The script does not delete what you created in Episode 6.

Steps

1. In Google Cloud: delete the service account `toktickit-deployer` (**IAM & Admin** → **Service Accounts**), and delete the pool `github-actions` (**IAM & Admin** → **Workload Identity Federation**).
2. In GitHub: open **Actions** → **Deploy demo** and **Disable workflow**. Otherwise, every merge to `main` starts a deployment that fails because the resources are gone.
3. Optionally delete the environment `demo` (**Settings** → **Environments**). Keep the ruleset on `main` if you still use the repository.

Done when: the deployer account and the pool are gone and **Deploy demo** is disabled.

Alternative: if the project was created only for this course, you can shut down the whole project in **IAM & Admin** → **Settings** → **Shut down**. This removes everything in it. Check the console message to see how long you can still recover a shut-down project before you rely on it.

Done when

- The preview matched your resources, and you confirmed the deletion with your project ID.
- Every list in 10.4 is empty (or shows only things you did not create), and billing displays no running charges.
- The Episode 6 account and pool are removed, and **Deploy demo** is disabled.
- (The resource names depend on your project and tech stack.)

If something fails

Symptom	Likely cause	Fix
“Error: PROJECT_ID is not set”	No project is set in Cloud Shell.	Run `export PROJECT_ID=...` again, or `gcloud config set project ...`
“Aborted: confirmation did not match."	The typed project ID differs.	Run the script again and type the exact ID.
“Teardown completed," but a resource is still listed.	That command failed (permissions or a different name).	Read the error above it, then delete the resource in the console or fix the name in the script.
The Cloud Build bucket remains.	The script does not delete it.	Delete it from the bucket list in 10.4.
The deploy demo fails after the teardown.	The pipeline points at deleted resources.	Disable the workflow (10.5, step 2)